Securing Vuln-Bank: A Purple Team Exercise in Enterprise Threat Emulation and Automated Detection Engineering

Class: SPR708 – Cybersecurity Engineering

Group 1

Haleemah Anwary

Sharisse Esguerra

Govind Sharma

Joshua Petrone

August 12th, 2026

Table of Contents

1. Executive Summary **6**
1.1 Overview 6
2. Introduction **7**
2.1 Project Background 7
2.2 Project Objectives 7
3. Market Segment Analysis **8**
4. Infrastructure **10**
5. Exercise Methodology **12**
5.1 Purple Team Lifecycle 12
5.2 Framework Usage 13
5.3 Exercise Approach 13
5.4 Methodology Summary 14
6. Attack Scenarios **14**
6.1 Scenario 1: API Version Downgrade and Password Reset Bypass 14
6.1.1 Objective 14
6.1.2 Target and Tools Used 14
6.1.3 Attack Chain and Execution 15
Phase 1 – Initiating the Password Reset 15
Phase 2 – Intercepting and Examining the API Request 17
Phase 3 – API Version Manipulation and PIN Disclosure 18
Phase 4 – Establishing the Original Reset Request 19
Phase 5 – Modifying the API Version and Reset PIN 21
Phase 6 – Successful Password Reset 24
Phase 7 – Account Access Validation 24
6.1.4 Attack Flow Summary 26
6.1.5 TTPs and Attacker Objective 26
6.1.6 MITRE ATT&CK Mapping 27
6.1.7 Impacted Systems and Attack Outcome 27
6.2 Scenario 2 - Broken Object Level Authorization (BOLA) 28
6.2.1 Objective 28
6.2.2 Target and Tools Used 28
6.2.3 Attack Chain and Execution 28
6.2.4 Attack Flow Summary 30
6.2.5 TTPs and Attacker Objective 30
6.2.6 Security Framework Mapping 30
6.2.7 Impacted Systems and Attack Outcome 30
6.3 Scenario 3 Business Logic Abuse (Negative Amount Transfer) 30
6.3.1 Objective 30
6.3.2 Target and Tools Used 31
6.3.3 Attack Chain and Execution 31

Phase 1 Establishing Baseline Balances 31
Phase 2 Capturing the Legitimate Transfer Request 32
Phase 3 Testing a Negative Transfer Amount 35
Phase 4 Repeated Exploitation of the Negative-Amount Transfer 37
Phase 5 Confirming Impact via Final Account Balances 38
6.3.4 Attack Flow Summary 40
6.3.5 TTPs and Attacker Objective 40
6.3.6 MITRE ATT&CK Mapping 40
6.3.7 Impacted Systems and Attack Outcome 40
6.4 Scenario 4- GraphQL Trust Boundary Failure 41
6.4.1 Objective 41
6.4.2 Target and Tools Used 41
6.4.3 Attack Chain and Execution 42
6.4.4 Attack Flow Summary 51
6.4.5 TTPs and Attacker Objective 52
6.4.6 MITRE ATT&CK Mapping 52
6.4.7 Impacted Systems and Attack Outcome 53
7. Detection and Response **53**
7.1 Scenario 1 – API Version Downgrade and Password Reset Bypass 54
7.1.1 Detection Method 54
7.1.2 Baseline Detection Results 55
7.1.3 Response Actions 56
7.1.4 Detection and Response Gaps 56
7.2 Scenario 2 - Broken Object Level Authorization 57
7.2.1 Detection Method 57
7.2.2 Baseline Detection Results 57
7.2.3 Response Actions 57
7.2.4 Detection and Response Gaps 57
7.3 Scenario 3 - Business Logic Abuse (Negative Amount Transfer) 57
7.3.1 Detection Method 57
7.3.2 Baseline Detection Results 58
7.3.3 Response Actions 59
7.3.4 Detection and Response Gaps 59
7.4 Scenario 4 – GraphQL Trust Boundary Failure 60
7.4.1 Detection Method 60
7.4.2 Baseline Detection Results 60
7.4.3 Response Actions 61
7.4.4 Detection and Response Gaps 61
8. Findings **62**
8.1 Scenario 1 – API Version Downgrade and Password Reset Bypass 62
8.1.1 API Version Downgrade Vulnerability 62

8.1.2 Exposure of Sensitive Reset Information 62
8.1.3 Insufficient Server-Side Protection 62
8.1.4 Weak Logging and Missing Detection 62
8.1.5 Overall Finding 63
8.3 Scenario 3 - Business Logic Abuse (Negative Amount Transfer) 63
8.3.1 Missing Server-Side Validation of Transfer Direction 63
8.3.2 Flawed Funds-Validation Logic 63
8.3.3 Unbounded, Repeatable Exploitation 63
8.3.4 Weak Logging and Missing Detection 64
8.3.5 Overall Finding 64
8.4 Scenario 4 – GraphQL Trust Boundary Failure 64
8.4.1 Unsanitized Input Reaching a Raw SQL Context 64
8.4.2 Excessive Database Service Account Privileges 65
8.4.3 Credential Scope Violation 65
8.4.4 Missing Network Segmentation Enforcement 65
8.4.5 Weak JWT Signing Secret 65
8.4.6 Overall Finding 65
9. Recommendations 66
9.1 Scenario 1 – API Version Downgrade and Password Reset Bypass 66
9.1.1 Disable or Restrict Vulnerable API Versions 66
9.1.2 Strengthen Server-Side Password Reset Validation 66
9.1.3 Improve Monitoring and SIEM Detection 66
9.1.4 Add a Reverse Proxy or Application-Layer Control 67
9.1.5 Apply Rate Limiting and Account-Recovery Protections 67
9.1.6 Protect Privileged Accounts 67
9.1.7 Centralize Application and Security Logs 67
9.1.8 Defense-in-Depth Summary 67
9.3 Scenario 3 — Business Logic Abuse (Negative Amount Transfer) 67
9.3.1 Enforce Sign Validation on the Transfer Amount 67
9.3.2 Redesign the Funds-Validation Logic 68
9.3.3 Apply Transaction Rate Limiting and Anomaly Thresholds 68
9.3.4 Improve Monitoring and SIEM Detection 68
9.3.5 Enable Docker Container Log Ingestion 68
9.3.6 Add Transaction Integrity Checks 68
9.3.7 Defense-in-Depth Summary 69
9.4 Scenario 4 – GraphQL Trust Boundary Failure 69
9.4.1 Parameterize Database Queries 69
9.4.2 Restrict Database Service Account Privileges 69
9.4.3 Scope and Restrict SSH Automation Credentials 69
9.4.4 Enforce Network Segmentation Between DMZ and Internal Zones 69
9.4.5 Strengthen JWT Secret Management and Rotate Keys 69

9.4.6 Improve Monitoring and SIEM Detection 70
10. Remediation 70
10.1 Scenario 1 – API Version Downgrade and Password Reset Bypass 70
10.1.1 Changes Implemented 70
10.1.2 NGINX Reverse Proxy Configuration 71
10.1.3 Reverse Proxy Deployment 71
10.1.4 Custom Wazuh Detection Rule 73
10.1.5 Remediation Summary 74
10.2 Scenario 2 - Broken Object Level Authorization 74
10.2.1 Changes Implemented 74
10.2.2 Custom Suricata Detection Rule 74
10.2.3 Suricata Validation and Deployment 75
10.3 Scenario 3 — Business Logic Abuse (Negative Amount Transfer) 75
10.3.1 Changes Implemented 75
10.3.2 Docker Log Ingestion Configuration 75
10.3.3 Custom Wazuh Detection Rules 76
10.3.4 Remediation Summary 77
10.4 Scenario 4 – GraphQL Trust Boundary Failure 78
10.4.1 Changes Implemented 78
10.4.2 Network Segmentation 80
10.4.3 SSH Credential Restriction 81
10.4.4 Custom Detection Rules 81
10.4.5 Remediation Summary 84
11. Validation 84
11.1 Scenario 1 – API Version Downgrade and Password Reset Bypass 84
11.1.1 Detection Validation 84
11.1.2 Preventive Control Validation 85
11.1.3 Detection of the Blocked Attack Attempt 88
10.3 Scenario 3 — Business Logic Abuse (Negative Amount Transfer) 90
11.3.1 Detection Validation 90
11.3.2 Preventive Control Validation 90
11.3.3 Detection of the Attack Despite Lack of Prevention 90
11.4 Scenario 4 – GraphQL Trust Boundary Failure 90
11.4.1 Detection Validation 90
11.4.2 Preventive Control Validation 91
11.4.3 Detection of Post-Exploitation Activity 92
12. Conclusion 94
13. References 96
14. Video Recordings 97

1. Executive Summary

1.1 Overview

This project presents a Purple Team exercise conducted against Vuln-Bank, a vulnerable banking application deployed to represent a mock financial institution operating at Security Maturity Level 1. The exercise was designed to simulate realistic cyberattacks against an intentionally vulnerable environment and evaluate the organization's ability to detect, respond to, and prevent malicious activity.

The Vuln-Bank application was deployed in a segmented virtual lab alongside internal enterprise and security systems. The environment included separate network zones for the public-facing application, internal servers, workstations, security management, and an external attacker. Wazuh was used for centralized security monitoring, with additional tools supporting endpoint monitoring, network visibility, security automation, and Purple Team tracking. This reflects the segmented multi-system architecture established in the project plan.

The exercise followed a structured test-remediate-retest approach. Attack scenarios were first executed against the baseline environment to identify vulnerabilities and gaps in existing security controls. The findings were then used to develop defensive controls and detection rules before the attacks were repeated to validate the implemented changes.

2. Introduction

2.1 Project Background

This project focuses on securing Vuln-Bank, a deliberately vulnerable banking application used to represent a mock financial institution operating at Security Maturity Level 1. The application was deployed within a controlled virtual lab environment to provide a realistic target for conducting offensive and defensive cybersecurity testing. The project guideline defines a Level 1 environment as having characteristics such as weak monitoring, minimal logging, limited endpoint protection, weak security policies, and poor network segmentation.

To support the exercise, a segmented network environment was created consisting of a DMZ, Internal Server Zone, Workstation Zone, Security Management Zone, and external attacker network. The environment included the Dockerized Vuln-Bank application, a Windows Domain Controller and file server, an enterprise workstation, Wazuh SIEM, Shuffle SOAR, VECTR/MISP, and an edge gateway with Suricata network monitoring. The original project design assigns these systems to separate network zones to simulate the structure of a small enterprise environment.

Purple teaming was used to bring offensive and defensive security activities together throughout the exercise. Attack scenarios were performed against the environment while the defensive side reviewed available logs, monitored security events, and identified gaps in existing controls. Findings from the attacks were then used to guide remediation and subsequent validation testing. This collaborative process supports continuous improvement by allowing attack behaviour to directly inform defensive changes. The project guideline specifically requires Red and Blue Team collaboration, shared findings, detection tuning, and defensive improvements throughout the exercise.

2.2 Project Objectives

The primary objective of this project was to design and conduct a structured Purple Team exercise against the simulated financial institution and evaluate its ability to detect and respond to cyberattacks. The project followed a test-remediate-retest approach in which attacks were first executed against the baseline environment, security weaknesses were identified, defensive controls were implemented, and the attacks were repeated to validate the changes.

The main objectives were to:

- Build a segmented multi-node lab representing a financial organization.
- Execute realistic attack scenarios against the Vuln-Bank environment.
- Map adversary behaviour to MITRE ATT&CK tactics and techniques.
- Collect and analyze security telemetry using centralized monitoring.
- Identify vulnerabilities, security-control weaknesses, and detection gaps during baseline testing.
- Develop detection rules based on observed attack behaviour.
- Implement defensive controls based on the findings of the Purple Team exercise.

- Use security orchestration and automation where implemented within the environment.
- Retest attack scenarios following remediation to evaluate changes in detection and attack success.
- Demonstrate collaboration between Red and Blue Team activities throughout the Purple Team lifecycle.

These objectives align with the overall project requirement to perform attack simulation, detection and monitoring, incident response, defensive improvement, and validation testing.

3. Market Segment Analysis

The group selected the financial services sector as the market segment for this exercise. Vuln-Bank represents a mid-size financial institution operating at Security Maturity Level 1, hosting customer accounts, fund-transfer capabilities, and an AI-assisted support interface. The financial sector was chosen because it is among the most heavily targeted industries globally, faces stringent regulatory obligations, and the specific vulnerabilities present in Vuln-Bank, API manipulation, business logic flaws, and injection-based exploitation, reflect real-world attack patterns documented in recent breach investigations against banking and payment platforms.

Industry Threats
Financial institutions face a distinct threat landscape driven by the direct monetary value of their assets and the regulatory sensitivity of customer financial data. The Verizon 2024 Data Breach Investigations Report identified the financial sector as the second most targeted industry, with financially motivated attacks accounting for the majority of incidents.

The threats most relevant to this exercise include:

1. API and Web Application Exploitation. Financial institutions increasingly expose functionality through APIs and GraphQL endpoints. Poorly secured API versions, missing input validation, and excessive data exposure enable attackers to bypass authentication, inject malicious queries, and exfiltrate customer records, directly mirroring Scenarios 1 through 4 in this exercise.

2. Business Logic Abuse. Unlike traditional injection attacks, business logic flaws exploit the intended functionality of an application. Negative-amount transfers, parameter tampering, and workflow manipulation allow attackers to steal funds without triggering conventional intrusion-detection signatures.

3. Credential Theft and Account Takeover. Weak password-reset mechanisms, guessable JWT signing secrets, and exposed API keys enable attackers to forge authentication tokens or hijack privileged accounts without brute-forcing passwords.

4. Lateral Movement and Privilege Escalation. Once an initial foothold is established on a public-facing server, attackers exploit weak network segmentation and over-scoped service credentials to pivot into internal zones, ultimately targeting domain infrastructure and high-value workstations.

5. Ransomware and Data Destruction. Although not directly simulated in this exercise, ransomware remains the highest-impact threat to financial institutions. The lateral movement and credential-access techniques demonstrated in Scenario 4 represent the exact precursor activity that typically precedes a ransomware deployment.

Threat Actors

Financial institutions are targeted by a wide range of threat actors with varying capabilities and motivations:

1. Financially Motivated Cybercriminal Groups. Organized groups such as FIN7 and Carbanak have historically targeted banks, payment processors, and financial institutions using a combination of phishing, web application exploitation, and lateral movement to authorize fraudulent transfers or deploy ransomware. Their operational patterns, initial access via public-facing applications followed by internal reconnaissance and privilege escalation, closely match the attack chain demonstrated in Scenario 4.

2. Nation-State Advanced Persistent Threats (APTs). Groups such as Lazarus Group have conducted multi-year campaigns targeting financial institutions for revenue generation. These actors typically combine custom malware with legitimate administrative tools, making detection dependent on behavioral monitoring rather than signature-based controls.

3. Opportunistic Attackers and Script Kiddies. Weak JWT secrets, exposed debug endpoints, and missing input validation are routinely discovered by automated vulnerability scanners. The API downgrade and business logic vulnerabilities in Vuln-Bank represent the type of low-complexity flaws that opportunistic actors exploit at scale.

4. Insider Threats. Employees or contractors with legitimate access to financial systems may abuse business logic, manipulate transactions, or exfiltrate customer data. The negative-amount transfer vulnerability in Scenario 3 could equally be exploited by an insider familiar with the application's transfer workflow.

4. Infrastructure

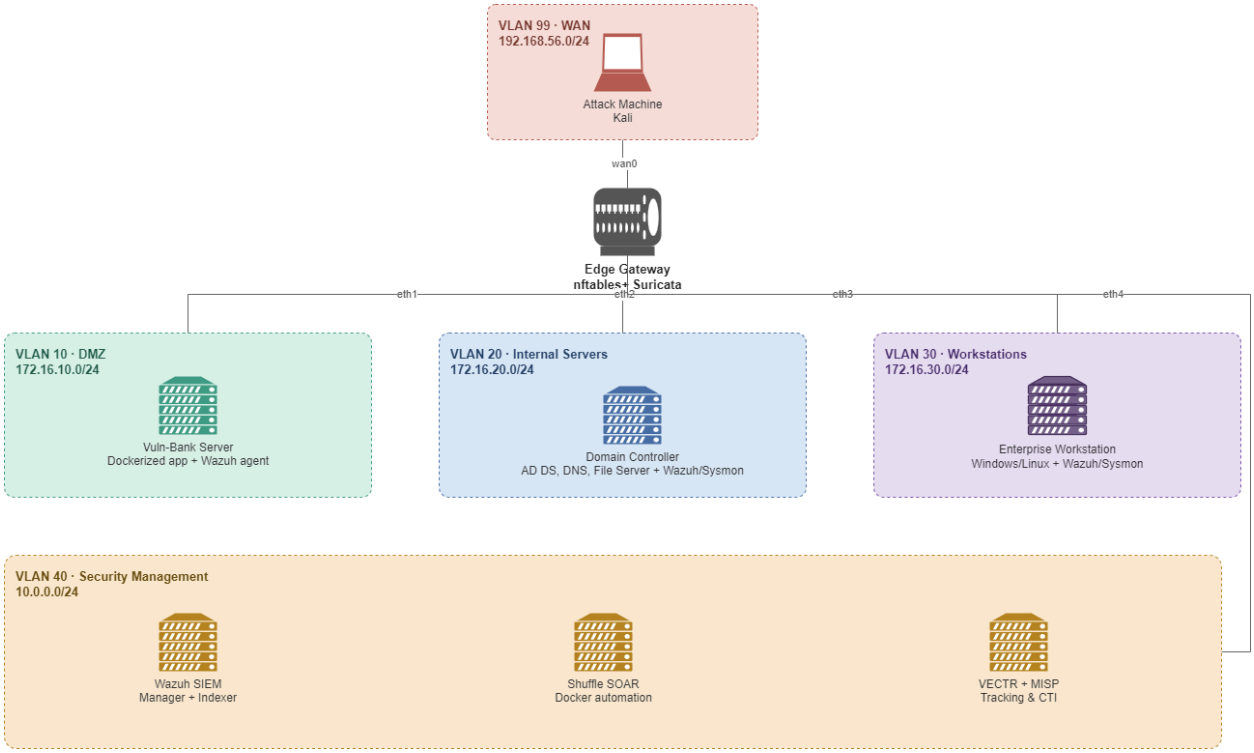


Figure 1: Network Topology

Segment Name	**Subnet**	**Type**	**Occupants**	**Purpose**
Fakenet (VLAN 99)	192.168.1.0/24	Internal Net	VM2 attack-machine, VM1 gateway	Simulates untrusted public Internet; attacker must pass through gateway security
DMZ (VLAN 10)	172.16.10.0/24	Internal Network	VM3 vuln-bank	Public-facing services; isolated to prevent easy pivoting

Internal Server Zone **(VLAN 20)**	172.16.20.0/24	Internal Network	VM4 domain-controller	Core enterprise services (AD, DNS, File Server)
Workstation Zone **(VLAN 30)**	172.16.30.0/24	Internal Network	VM5 workstation	Corporate workstation segment; routes through gateway
Security Mgmt Zone **(VLAN 40)**	10.0.0.0/24	Internal Network	VM6 SIEM, VM7 SOAR, VM8 CTI	Out-of-band monitoring & response; telemetry flows inward

Table 1: Network

VM Name	Role / Function	Subnet	IP Address
VM 1: Gateway	Edge Router + Suricata IDS	Multiple	192.168.1.1 (WAN) / 172.16.10.1 (DMZ) / 172.16.20.1 (Internal) / 172.16.30.1 (Workstation) / 10.0.0.1 (Sec-Mgmt)
VM 2: Attack Machine	Red Team / Kali	192.168.1.0/24	192.168.1.20
VM 3: Vuln-Bank	Public Web App (DMZ)	172.16.10.0/24	172.16.10.10
VM 4: Domain Controller	AD / DNS / File Server	172.16.20.0/24	172.16.20.10
VM 5: Workstation	Enterprise Windows Client	172.16.30.0/24	172.16.30.10
VM 6: SIEM (Wazuh Manager)	Security Monitoring	10.0.0.0/24	10.0.0.10

VM 7: SOAR (Shuffle)	Automated Response	10.0.0.0/24	10.0.0.20
VM 8: CTI (MISP / VECTR)	Threat Intelligence	10.0.0.0/24	10.0.0.30

Table 2: IP Addressing Scheme

5. Exercise Methodology

The Purple Team exercise followed a structured methodology designed to connect offensive testing with defensive monitoring, remediation, and validation. Rather than performing penetration testing as a standalone activity, each attack scenario was used to evaluate the existing security controls, identify weaknesses, implement defensive changes, and then retest the environment.

5.1 Purple Team Lifecycle

The exercise followed a test-remediate-retest lifecycle across three main testing stages.

During the baseline execution, the Red Team performed the planned attack scenarios against the Vuln-Bank environment before scenario-specific defensive controls were introduced. The Blue Team monitored available application, endpoint, network, and security logs to determine what activity was visible and whether existing controls generated meaningful detections. This phase established the initial security posture and exposed detection and prevention gaps. The original project plan specifically defines this phase as running the attacks without custom defenses to determine what succeeds and identify logging gaps.

During the remediation phase, findings from the baseline attacks were reviewed and used to develop additional defensive controls. Depending on the attack scenario, these included custom detection rules, application or network-level controls, and security automation where applicable. The purpose of this phase was to directly address the weaknesses identified during baseline testing.

Finally, during the validation phase, the attack scenarios were repeated against the remediated environment. Detection events, logs, alerts, and attack outcomes were compared with the baseline results to determine whether the implemented controls increased visibility and reduced attack success. This follows the project's requirement to rerun attacks after remediation and demonstrate improved detection, visibility, and reduced attack success.

The overall lifecycle can therefore be summarized as:

Baseline Attack → Observe & Detect → Identify Gaps → Remediate → Retest → Validate

5.2 Framework Usage

The exercise used the MITRE ATT&CK Framework to provide a consistent method for describing and categorizing adversary behaviour. Each attack scenario was mapped to relevant ATT&CK tactics and techniques based on the actions performed and the objective of the simulated attacker.

ATT&CK mapping was used to connect offensive activity with defensive monitoring opportunities. Rather than documenting attacks only as technical steps, the framework allowed the group to describe the behaviour being tested, its security objective, and where detection opportunities existed.

The project guideline specifically requires each scenario to document its tactic, technique, sub-technique, and detection opportunities, and ATT&CK mapping was maintained throughout attack execution and analysis.

The exercise also followed the Purple Team Exercise Framework and Purple Team Lifecycle required by the project. These provided the overall structure for planning, execution, collaborative analysis, remediation, and validation.

5.3 Exercise Approach

The exercise used a collaborative workflow between the Red and Blue Team functions.

The Red Team was responsible for executing the planned attack scenarios against the lab environment, documenting the actions performed, recording attack evidence, and mapping relevant behaviour to MITRE ATT&CK.

The Blue Team monitored the environment during the attacks using the available logging and security-monitoring infrastructure. Application logs, Wazuh events, Sysmon telemetry, and other available evidence were reviewed to determine whether malicious behaviour was visible and whether existing detections responded appropriately.

Following each baseline test, the Red and Blue Team findings were reviewed together. Successful attack activity was compared with the available defensive evidence to identify where controls were effective and where detection or prevention gaps remained. These findings were then used to determine appropriate remediation.

After the defensive changes were implemented, the same attack behaviour was repeated. The resulting logs, alerts, and attack outcomes were compared with the baseline results to validate the controls and document any change in detection or attack success.

This collaborative workflow reflects the project requirement that Red and Blue Teams communicate throughout the exercise and participate in shared findings, joint analysis, detection tuning, and defensive improvements.

5.4 Methodology Summary

The methodology connected each stage of the exercise rather than treating attack simulation, detection, and remediation as separate activities. Each scenario progressed through the same general process:

Execute → Monitor → Analyze → Identify Gap → Remediate → Retest → Validate

This provided a consistent methodology across the group's attack scenarios and allowed the final results to demonstrate both the vulnerabilities present in the baseline environment and the effect of the defensive controls implemented during the Purple Team exercise.

6. Attack Scenarios

6.1 Scenario 1: API Version Downgrade and Password Reset Bypass

6.1.1 Objective

The objective of this attack scenario was to evaluate the security of VulnBank's password-reset functionality and determine whether differences between API versions could be abused to bypass the application's intended account-recovery controls. The scenario focused on the admin@vulnbank.org account and tested whether information exposed through an alternate API version could be used to reset the account password without knowing the existing password.

A successful attack was defined as obtaining a valid reset PIN through the vulnerable API behavior, using that PIN to successfully reset the administrator password, and confirming access to the account using the newly configured password.

6.1.2 Target and Tools Used

The target of the scenario was the VulnBank web application, which was hosted within the project's isolated lab environment. The attack originated from the Kali Linux attacker machine.

Component	Purpose
Kali Linux	Attacker workstation used to conduct the controlled attack
Burp Suite	Interception and examination of VulnBank HTTP/API requests and responses
Burp Repeater	Used to modify and resend captured requests during API-version testing
Web Browser	Used to interact with the VulnBank application and verify account access

VulnBank	Target vulnerable banking application
admin@vulnbank.org	Test administrator account targeted during the scenario

The attacker machine was separated from the VulnBank server according to the segmented architecture developed for the exercise. The report skeleton identifies VulnBank as the public-facing application in the DMZ and the attack machine as the Red Team/Kali system.

6.1.3 Attack Chain and Execution

Phase 1 – Initiating the Password Reset

The scenario began by accessing VulnBank's Forgot Password functionality and initiating a password reset for the admin@vulnbank.org account. The application generated the password-recovery workflow and instructed the user to provide a reset PIN.

The normal web interface specifically expected a four-digit PIN. This established the expected behavior of the application before manipulation of the underlying API requests.

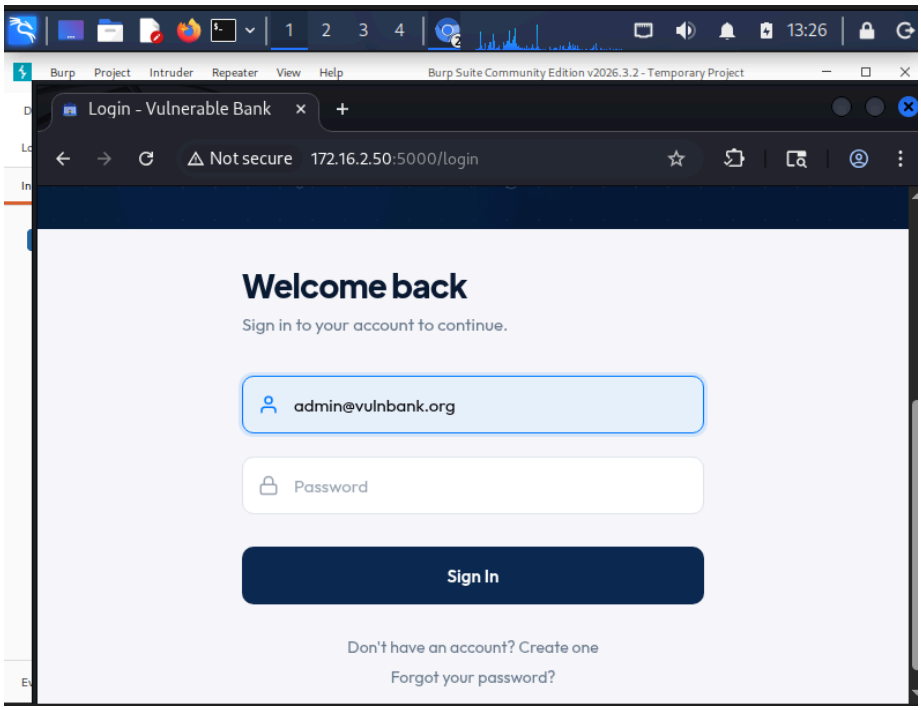


Figure 2 : VulnBank Forgot Password page used to initiate the administrator password-reset process.

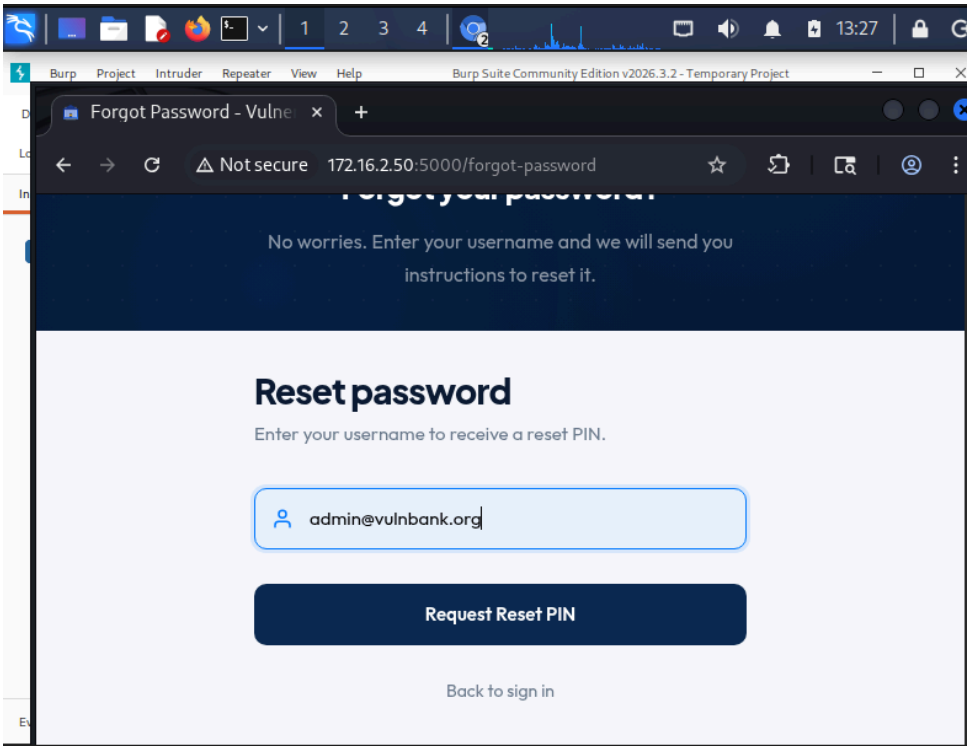


Figure 3 : Password-reset PIN requested for the administrator account.

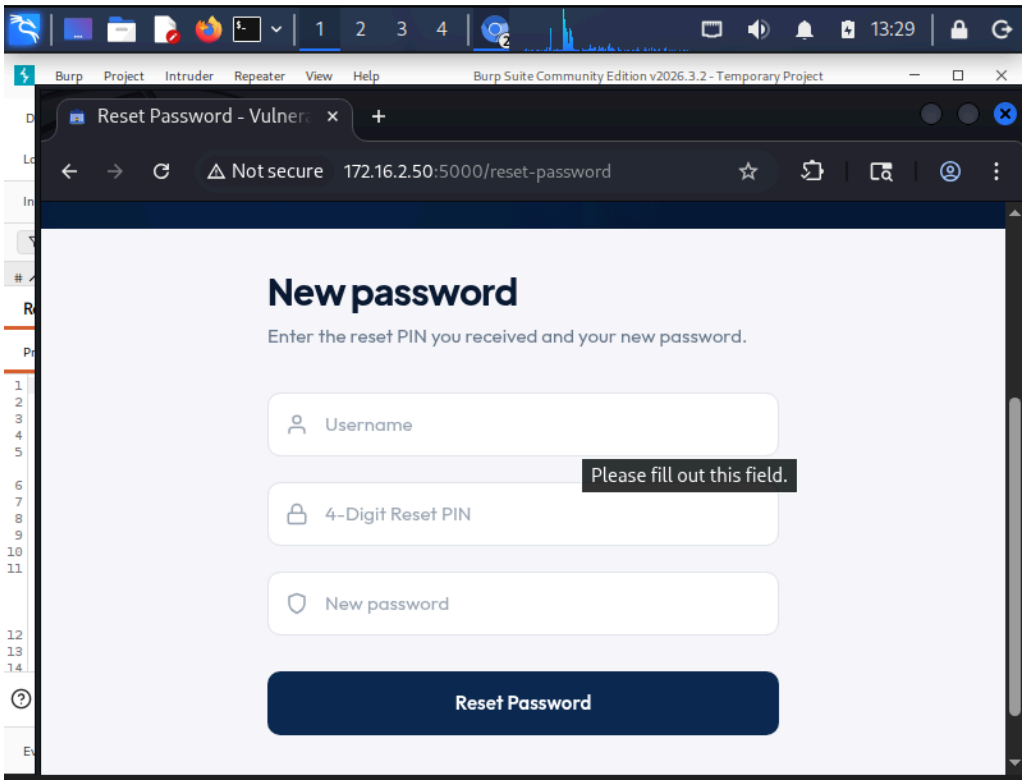


Figure 4: VulnBank password-reset interface requesting a four-digit PIN.

Phase 2 – Intercepting and Examining the API Request

Burp Suite was used to observe the HTTP/API traffic generated during the password-reset process. The relevant request was captured and examined to identify how the application communicated with its backend password-reset API.

The captured request was transferred to Burp Repeater, allowing the request to be examined and modified without changing the normal browser workflow.

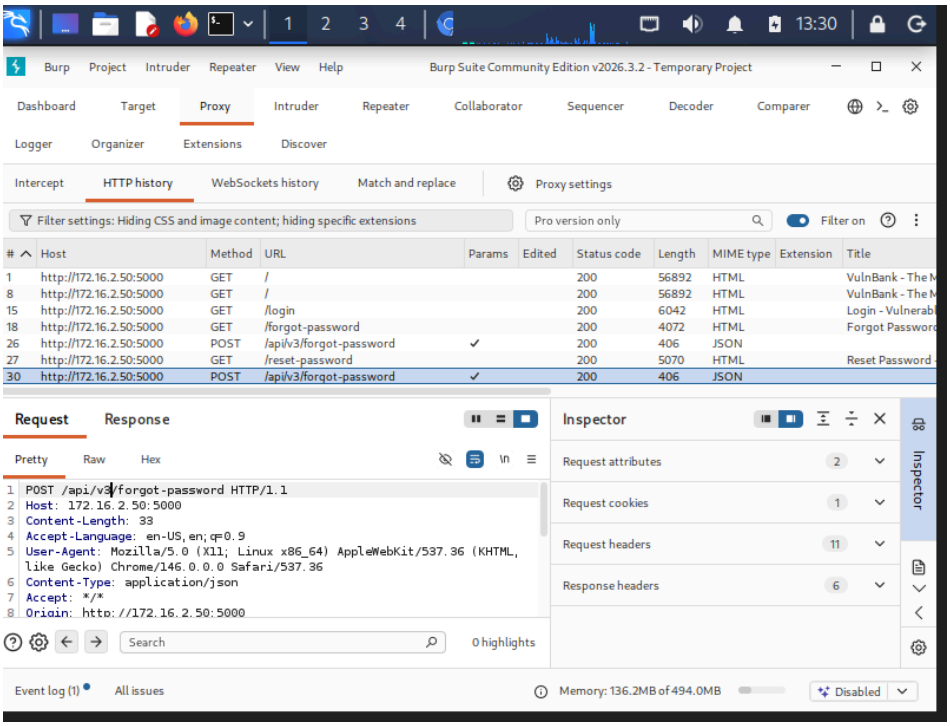


Figure 5: VulnBank password-reset API request captured in Burp Suite.

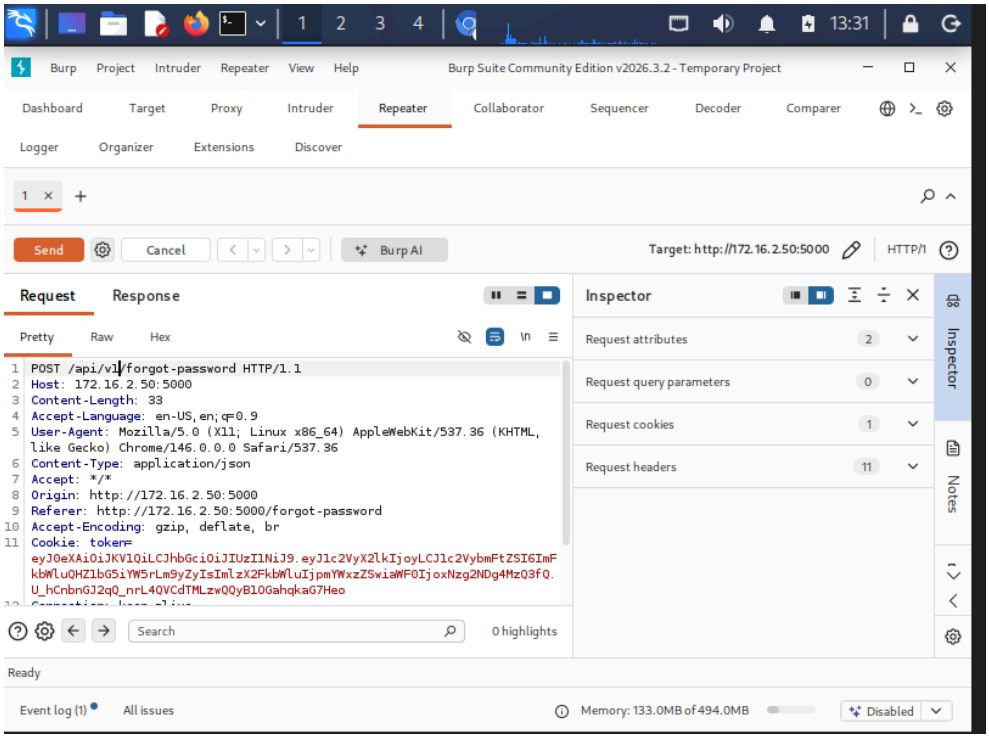


Figure 6: Captured API request transferred to Burp Repeater for controlled API-version testing.

Phase 3 – API Version Manipulation and PIN Disclosure

During testing, the API version referenced by the captured request was changed to interact with an alternate version of the VulnBank API. This produced different behavior from the application's normal password-reset workflow.

Most importantly, the resulting API response exposed a three-digit reset PIN (484). This was inconsistent with the normal web interface, which expected a four-digit PIN.

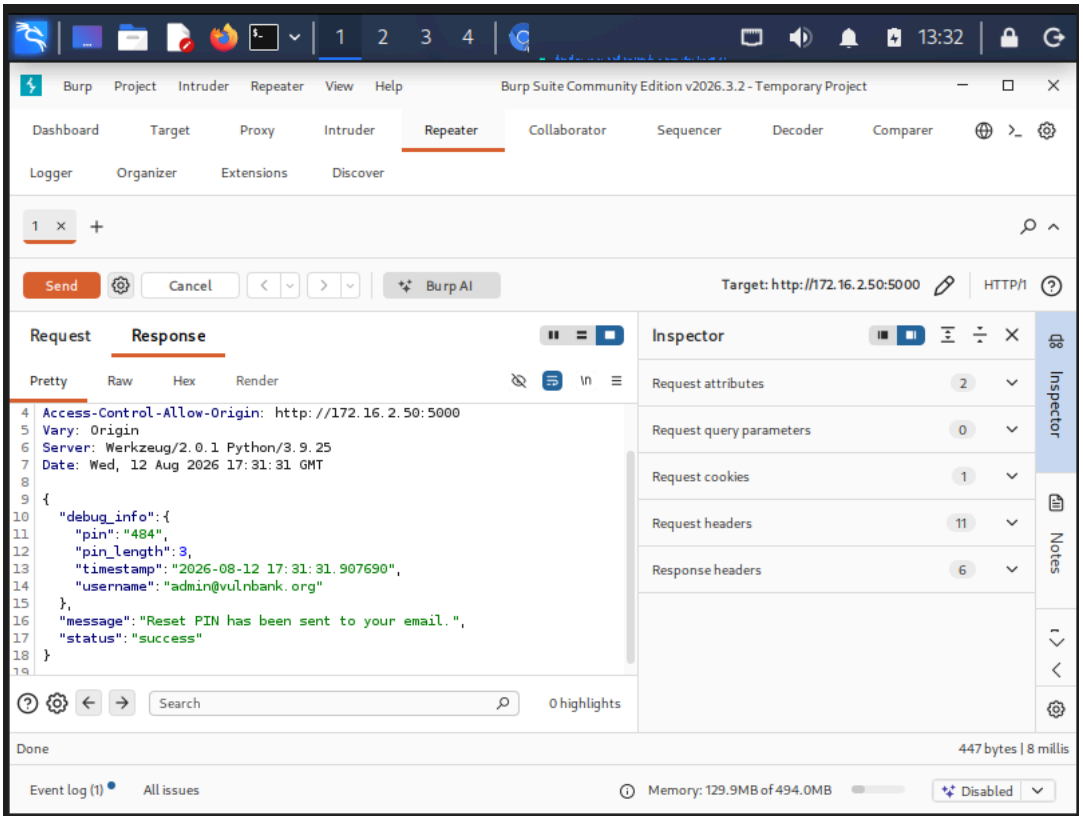


Figure 7: Alternate API response exposing the three-digit password-reset PIN (484).

This represented the main vulnerability discovered during this stage of the scenario. Information that should have remained protected during the password-recovery process was exposed through the API response and could subsequently be used in another password-reset request.

Phase 4 – Establishing the Original Reset Request

To test whether the exposed PIN could actually be used, the password-reset workflow was continued through the web application. An incorrect four-digit value (0000) was entered initially.

The resulting password-reset request was captured and examined. The request contained the administrator username, the incorrect reset PIN, and the new password.

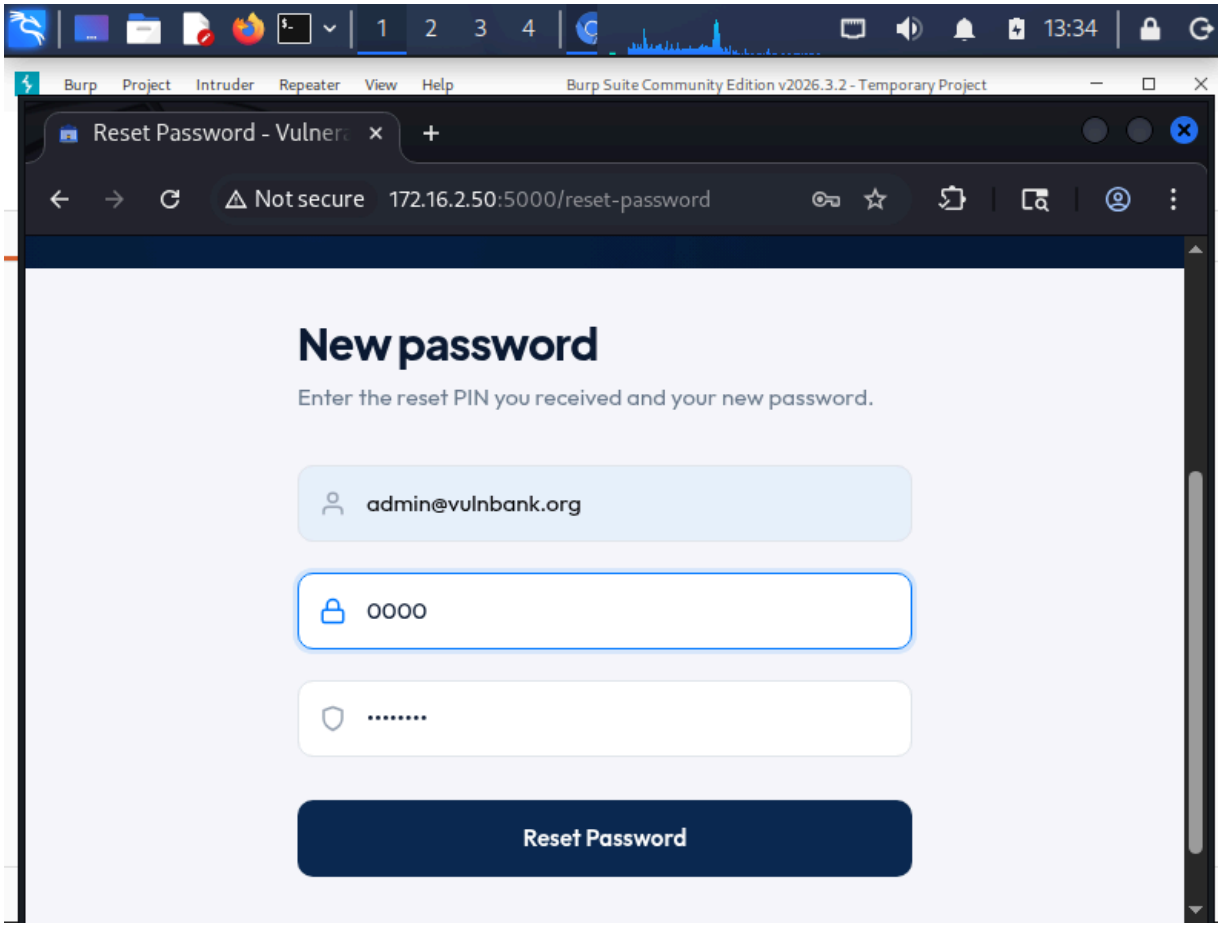


Figure 8: Incorrect four-digit PIN (0000) entered through the VulnBank password-reset interface.

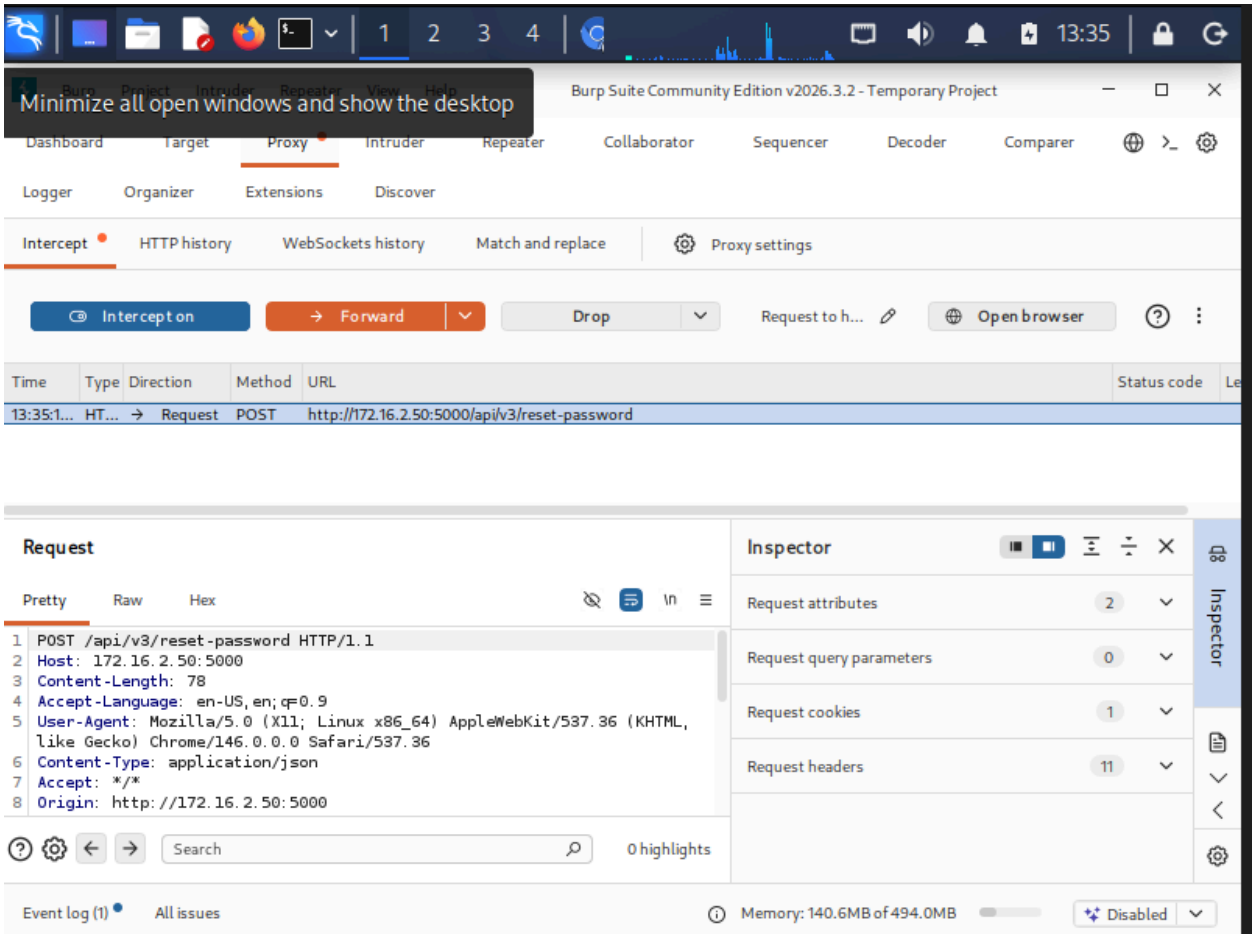


Figure 9: Password-reset request captured after submission through the web application.

Phase 5 – Modifying the API Version and Reset PIN

The captured reset request was then sent to Burp Suite Repeater so it could be tested against the alternate API version used earlier in the scenario. As shown in Figure 12, the API version in the request was changed again before the reset request was sent.

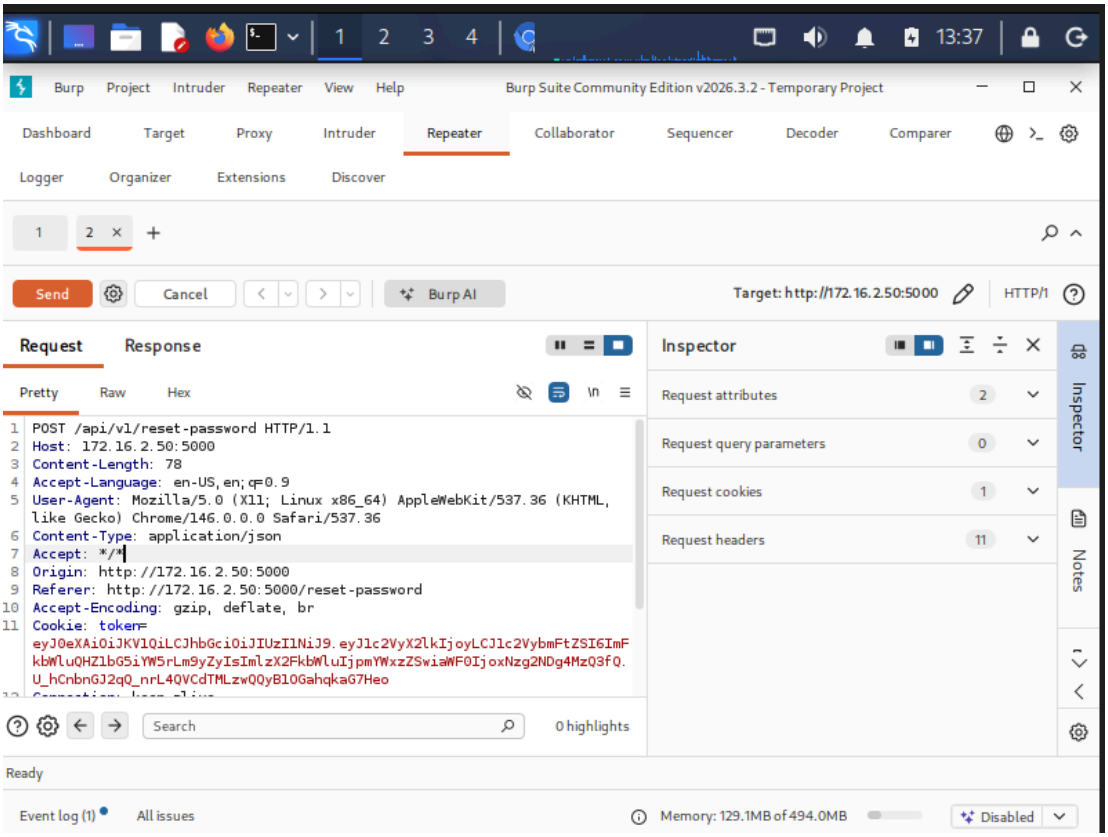


Figure 10: Password-reset request sent to Burp Suite Repeater and modified to use the alternate API version before submission.

The request body still contained the incorrect four-digit reset PIN originally entered through the web interface. Figure 13 shows the request containing:

"reset_pin": "0000"

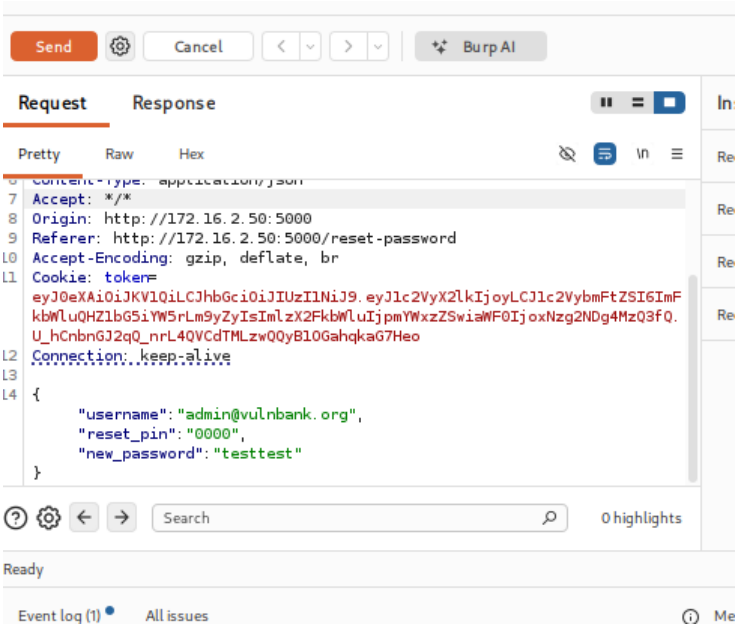


Figure 11: Burp Suite Repeater displaying the password-reset request containing the incorrect four-digit PIN (0000).

The incorrect value was then replaced with the valid three-digit PIN (484) obtained from the earlier API response.

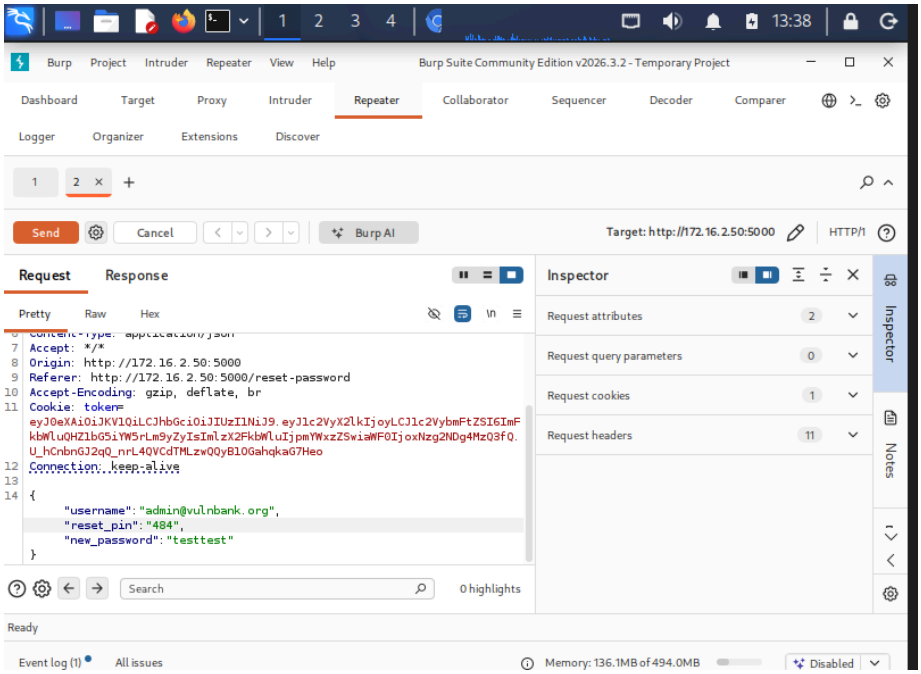


Figure 12: Password-reset request modified by replacing the incorrect PIN with the valid three-digit PIN (484) disclosed by the alternate API response.

Phase 6 – Successful Password Reset

The modified request, now using the alternate API version and the valid reset PIN, was submitted to VulnBank. The application returned a successful response containing "reset_success": true and confirmed that the password had been reset.

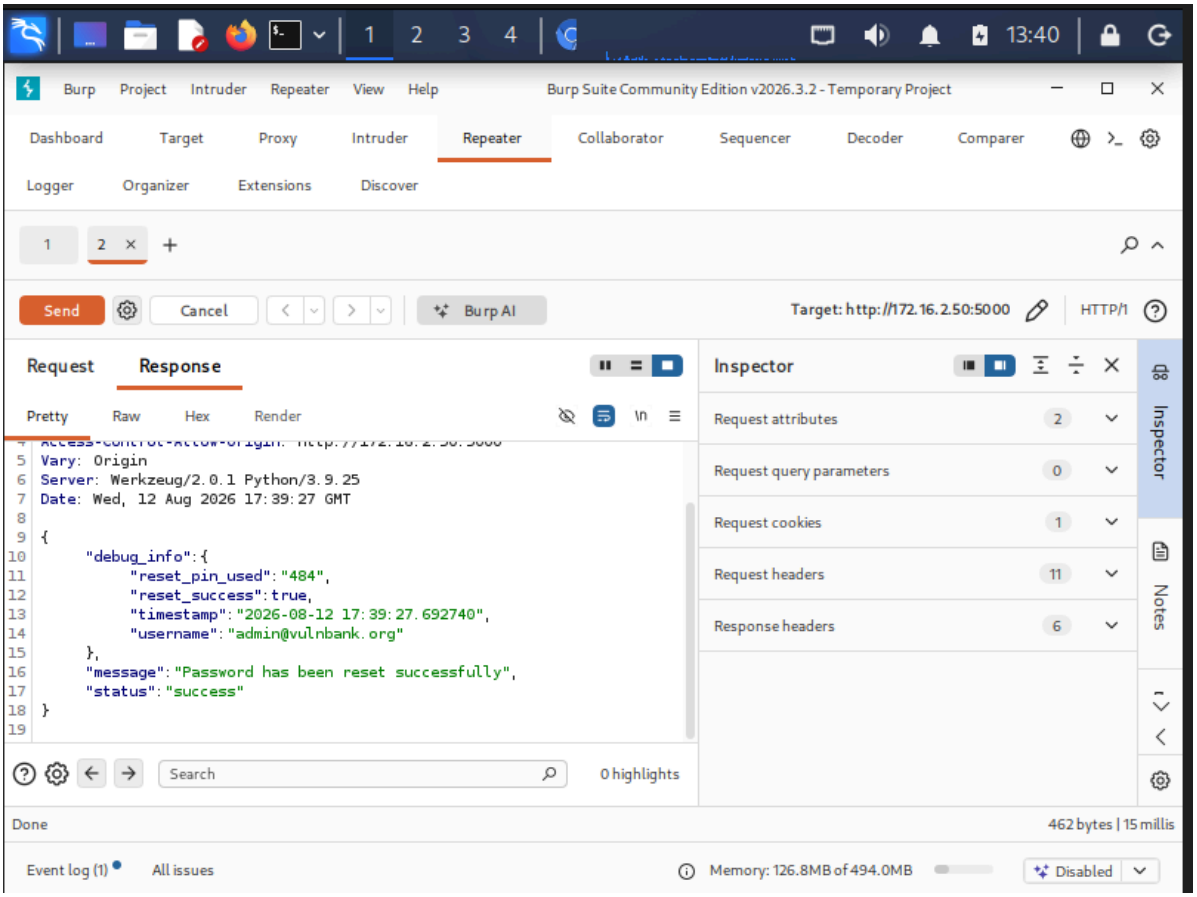


Figure 13: VulnBank API response confirming successful password reset after submission of the modified request.

Phase 7 – Account Access Validation

The newly configured password was then used to authenticate to the admin@vulnbank.org account. The login succeeded, and access to the VulnBank dashboard confirmed that the password-reset bypass resulted in successful administrator account access.

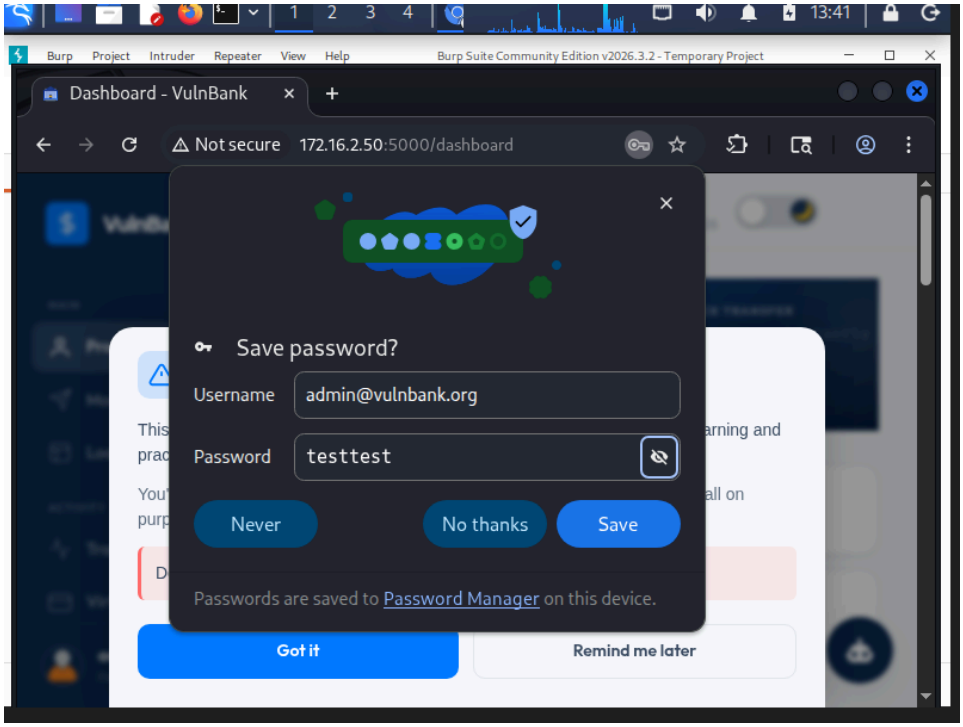


Figure 14: Login attempt using the newly reset administrator password.

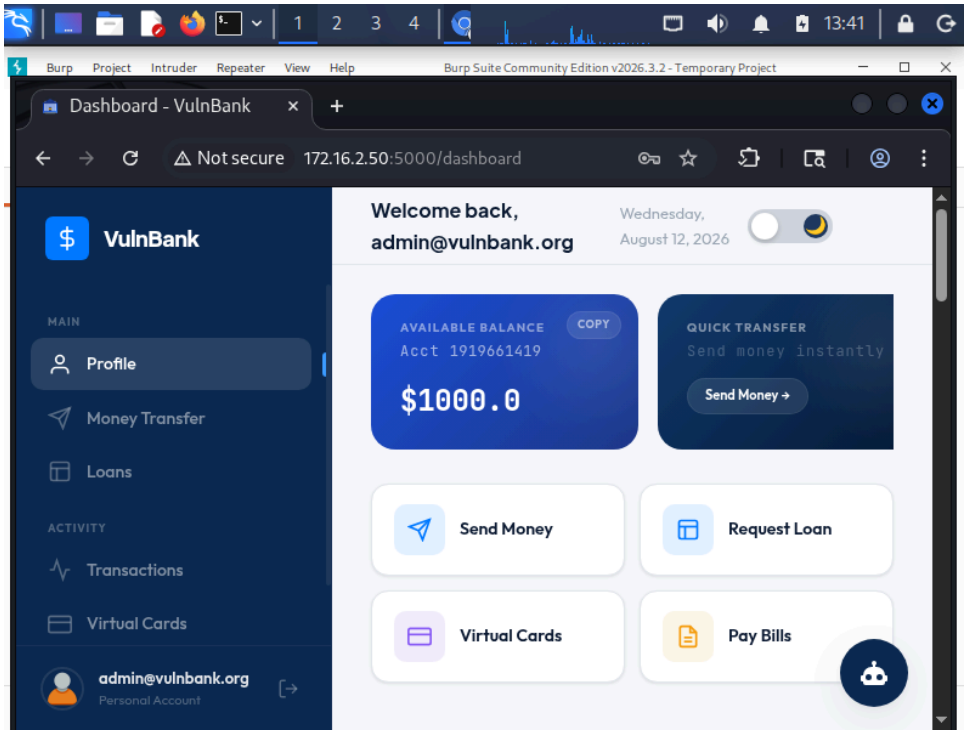


Figure 15: Successful access to the VulnBank administrator dashboard following the password-reset bypass.

6.1.4 Attack Flow Summary

The attack followed a structured sequence beginning with the normal VulnBank password-reset process. The application initially required a four-digit reset PIN for the administrator account. After the password-reset request was captured in Burp Suite, the API version was modified and the response exposed a valid three-digit reset PIN. A separate password-reset request was then captured, sent to Burp Repeater, and modified to use the alternate API version. The incorrect PIN originally submitted through the web application was replaced with the valid three-digit PIN obtained earlier. When the modified request was sent, the server accepted the PIN and successfully reset the administrator password. The new password was then used to log in to the admin@vulnbank.org account, confirming successful account access.

6.1.5 TTPs and Attacker Objective

The primary attacker objective in this scenario was to gain access to the VulnBank administrator account without knowing the account's existing password. Rather than attempting to obtain the original password, the attack targeted weaknesses in the application's account-recovery process.

The main adversary behavior involved interacting with the public-facing VulnBank application, inspecting its API communications, manipulating the API version used by password-reset requests, and taking advantage of the reset PIN disclosed by the alternate API response. The

disclosed PIN was then used to modify a subsequent password-reset request, allowing the administrator password to be changed. The newly established credentials were ultimately used to authenticate successfully to the account.

This scenario therefore demonstrated how weaknesses within an exposed application and its API could be chained together to obtain unauthorized access to a privileged account.

6.1.6 MITRE ATT&CK Mapping

The attack was mapped to the MITRE ATT&CK framework based on the main techniques demonstrated during the password-reset bypass.

Attack Activity	**MITRE Tactic**	**Technique**	**ID**
Exploiting the vulnerable VulnBank application and API	Initial Access	Exploit Public-Facing Application	T1190
Obtaining and using the exposed reset PIN	Credential Access	Credential-related activity	N/A
Logging into the administrator account after resetting the password	Initial Access / Persistence	Valid Accounts	T1078

6.1.7 Impacted Systems and Attack Outcome

The primary system impacted during this scenario was the VulnBank web application and its password-reset API, hosted within the project's DMZ. The account affected during the controlled exercise was the test administrator account, admin@vulnbank.org. The report's infrastructure design places the VulnBank application within the DMZ as the project's public-facing service.

The attack was successful. The alternate API behavior exposed a three-digit reset PIN despite the normal web interface expecting a four-digit value. After the disclosed PIN was inserted into the modified password-reset request, VulnBank returned a successful password-reset response. The newly configured password was then used to authenticate successfully to the administrator account.

The final dashboard access shown in Figure 15 confirmed the outcome of the scenario and demonstrated that the weakness could progress beyond information disclosure to successful access to the targeted account.

6.2 Scenario 2 - Broken Object Level Authorization (BOLA)

6.2.1 Objective

The objective of this scenario was to determine whether VulnBank enforced object-level authorization when a user requested account-balance information. The test evaluated whether changing the account number supplied to the balance endpoint would allow one user to view information belonging to another controlled test account. A successful attack was defined as VulnBank returning the second account's username or balance without verifying that the requester owned or was authorized to access that account. The activity was limited to read-only requests and did not modify balances or transaction records.

6.2.2 Target and Tools Used

The target was the VulnBank balance endpoint hosted at 172.16.10.10 in the DMZ. The test originated from the Kali red-team VM at 192.168.56.10 and passed through the IDS-ROUTER at 192.168.56.1/172.16.10.1. Kali Firefox was used to submit the requests. Suricata on IDS-ROUTER and Wazuh were used later to validate network detection and centralized alerting.

6.2.3 Attack Chain and Execution

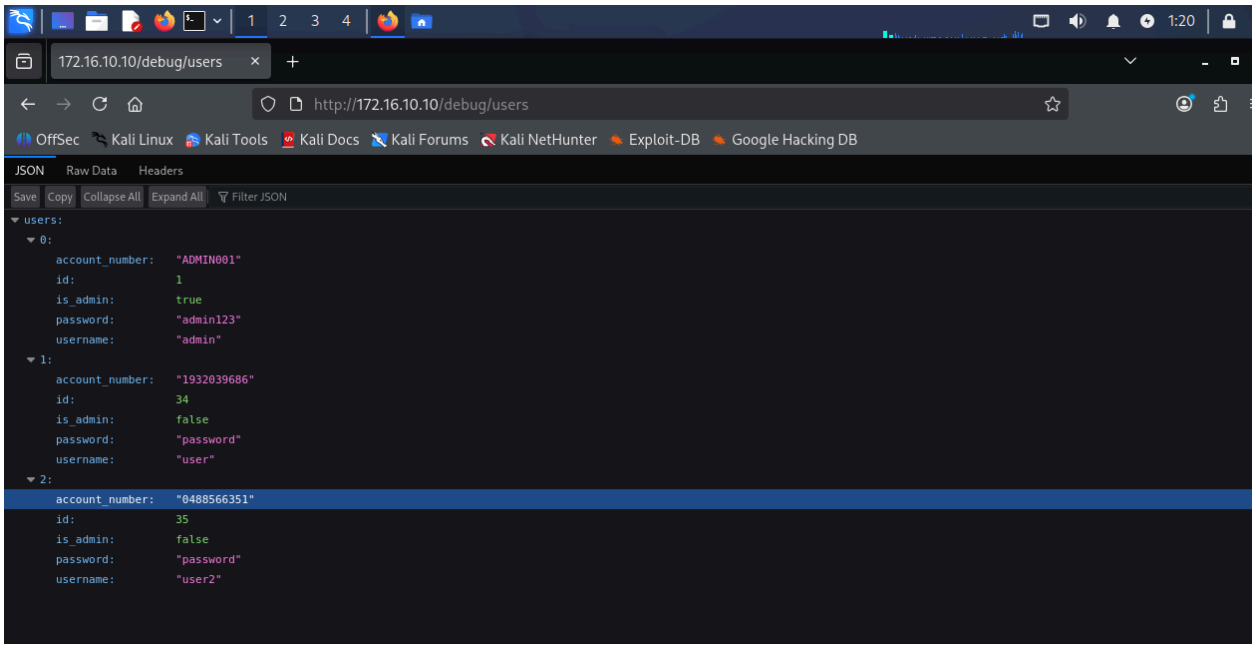


Figure 16: *VulnBank debug endpoint exposing usernames and account identifiers*

As you can see here the attacker hits an endpoint which is /debug/user. This is publicly accessible and it exposes the valid usernames and account numbers. This is information which allows the attacker to identify another controlled account for authorization breaching in this case.

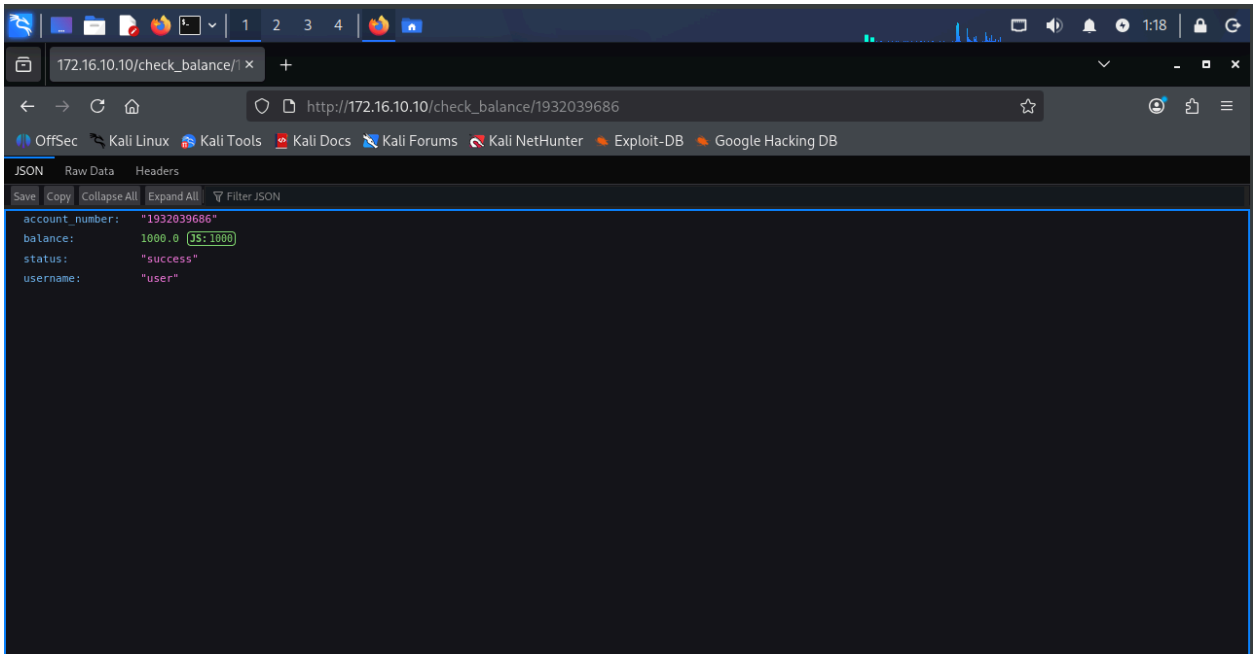


Figure 17: *Authorized balance request for the tester's original controlled account*

The tester first requested an account 1932039686. VulnBank returned the expected username and balance, establishing the normal authorized baseline.

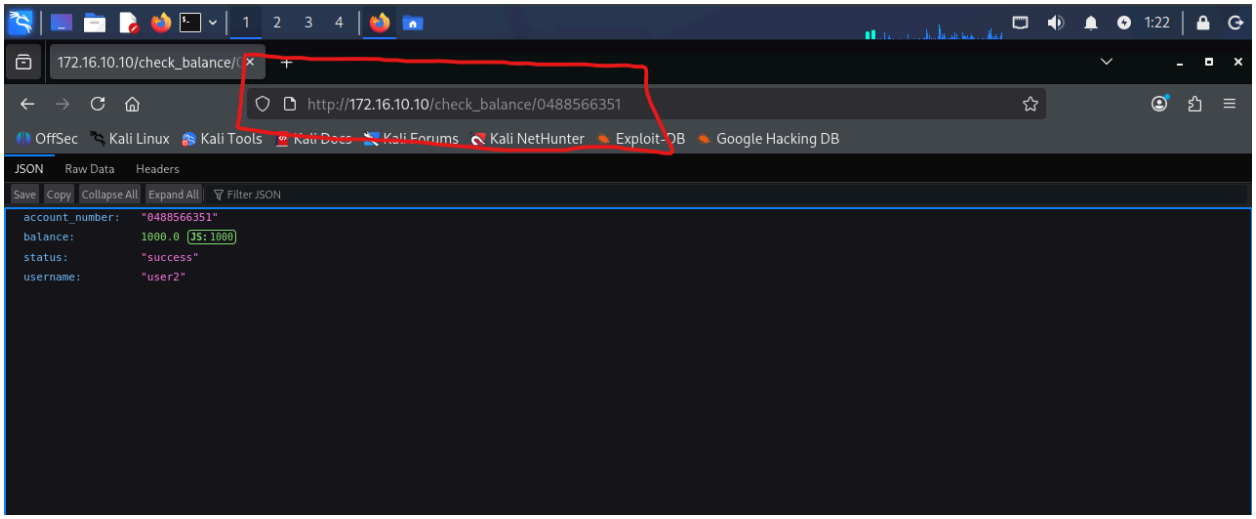


Figure 18: *Modified account identifier returning another user's balance without authorization*

The account number in the URL was changed to 0488566351 without changing the active session. VulnBank returned User 2's balance without verifying ownership, confirming the BOLA vulnerability

6.2.4 Attack Flow Summary

The attack began with an authenticated session and an authorized request for the first controlled account. The tester identified the account number in the request path, substituted a second controlled account number and submitted the modified request without changing sessions. Because VulnBank did not perform an ownership check, the application returned the second account's information. The sequence can be summarized as follows, authenticate as user1-, then record the authorized baseline, identify the object ID, substitute user2's ID, and receive user2's data.

6.2.5 TTPs and Attacker Objective

Now the attackers main objective was to bypass the horizontal authorization controls and then access the sensitive info in place. Which belongs to another customer at the same privilege level. All this behavior depended on manipulation of the user controlled object identifier rather than stealing credentials or exploiting the underlying OS.

6.2.6 Security Framework Mapping

The vulnerability maps directly to OWASP API1:2023 - Broken Object Level Authorization and CWE-639 - Authorization Bypass Through User-Controlled Key. At the adversary-behavior level, the exploitation of the externally reachable VulnBank application was mapped to MITRE ATT&CK T1190 - Exploit Public-Facing Application. OWASP and CWE provide the most precise classifications for the application flaw.

6.2.7 Impacted Systems and Attack Outcome

Looking at the impacted system which was the VulnBank Application and users. The overall outcome was successful from the attacker side and defensive side. With the request containing a different test account number returned information belonging to that account. As well it showcased that the attacker if gathered account numbers that work, they could access sensitive customer information overall.

6.3 Scenario 3 Business Logic Abuse (Negative Amount Transfer)

6.3.1 Objective

The objective of this attack scenario was to evaluate whether the VulnBank's fund transfer functionality properly validated the amount field on the server side and to determine if submitting a negative transfer value could be abused to illegally move money across accounts. The Focused on transfer endpoint used to move money between two accounts and tested the application To see if it relied on client side or UI level restrictions to prevent negative amounts rather than enforcing the validation in the back end.

A successful attack was defined as submitting a modified transfer request containing a negative amount having the server accept and process that request, and confirming that the resulting account balance reflects an illegitimate transfer of funds, showing that the intended direction of the transaction can be reversed without authorization.

6.3.2 Target and Tools Used

The target of the scenario was the VulnBank web applications fund transfer functionality, hosted in the project's isolated lab environment. The attack started from the Kali attacker machine.

Component	**Purpose**
Kali Linux	Attacker workstation used to conduct the controlled attack
Burp Suite	Interception and examination of VulnBank's transfer request
Burp Repeater	Used to modify and resend captured requests with a negative amount
Web Browser	Used to interact with the VulnBank application and initiate the legitimate transfer
VulnBank	Target vulnerable banking application
rhettmclaughlin	Test attacker account, intended recipient of the illegitimately transferred funds
linkneal	Test victim account, targeted for unauthorized fund transfer

6.3.3 Attack Chain and Execution

Phase 1 Establishing Baseline Balances

Before initiating any transfers, the starting balances of both accounts were recorded. The attacker controlled account rhettmclaughlin with account number 5757713009 showed a balance of $1000 and the victim account, linkneal with account number 0724206189 also showed a balance of $1000.

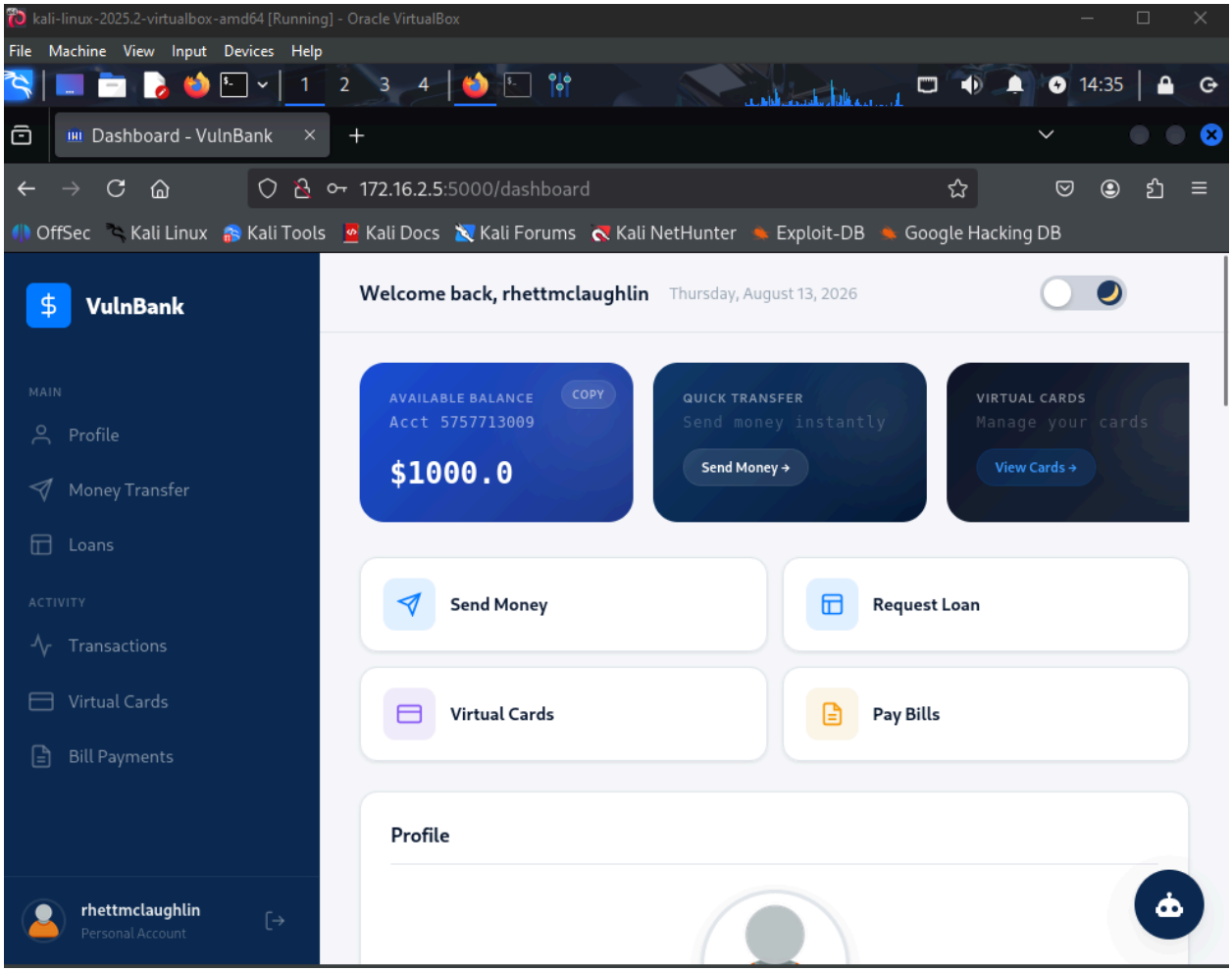


Figure 19: *VulnBank dashboard confirming rhettmclaughlin's starting balance of $1000.00.*

Phase 2 Capturing the Legitimate Transfer Request

To establish the normal request format used by VulnBank's transfer functionality, a legitimate transfer of $708 was initiated from linkneal's account to rhettmclaughlin's account (5757713009) through the standard Money Transfer form. The resulting POST /transfer request was captured in Burp Suite's HTTP history.

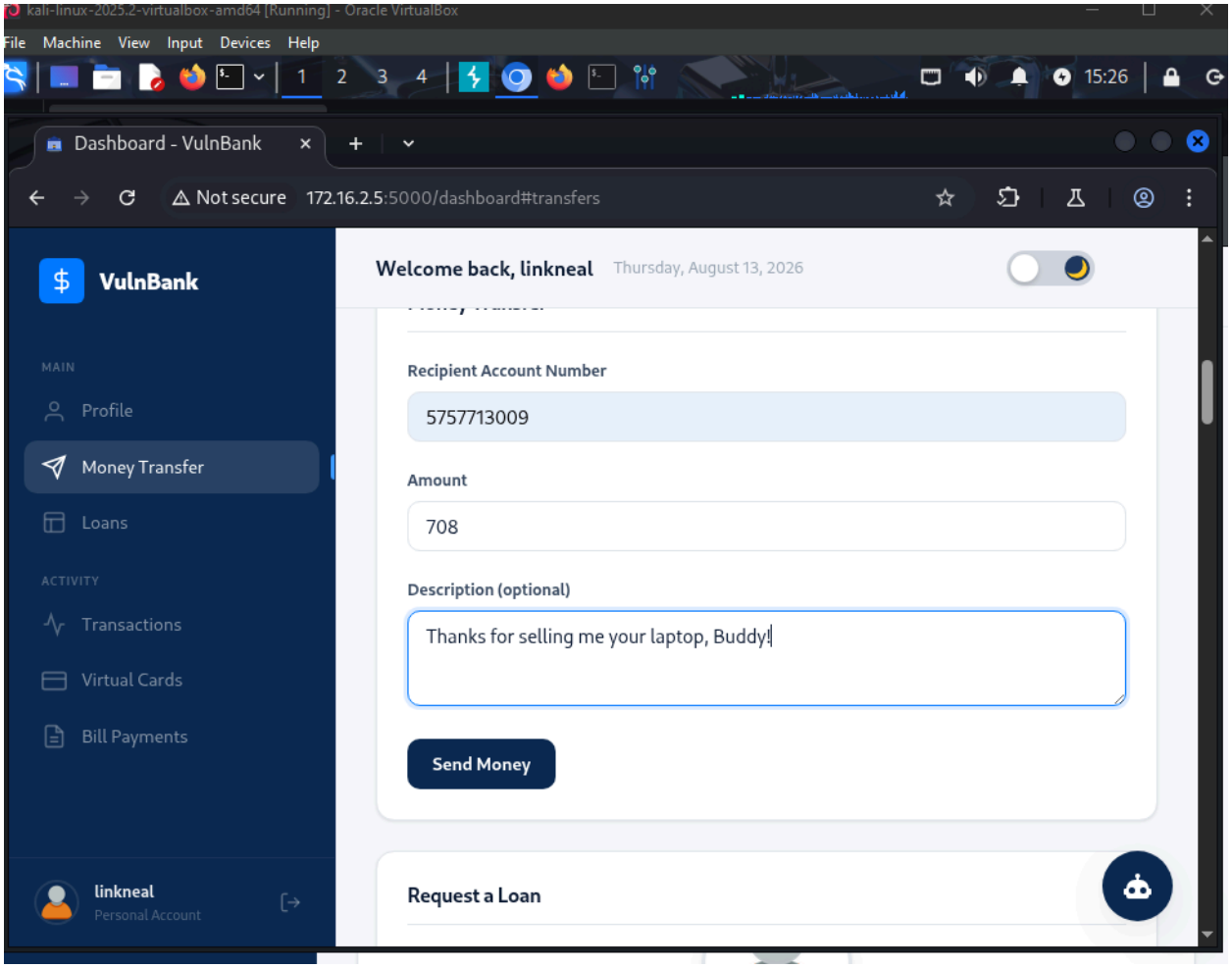


Figure 20: *VulnBank Money Transfer form used to send $708 from linkneal to rhettmclaughlin.*

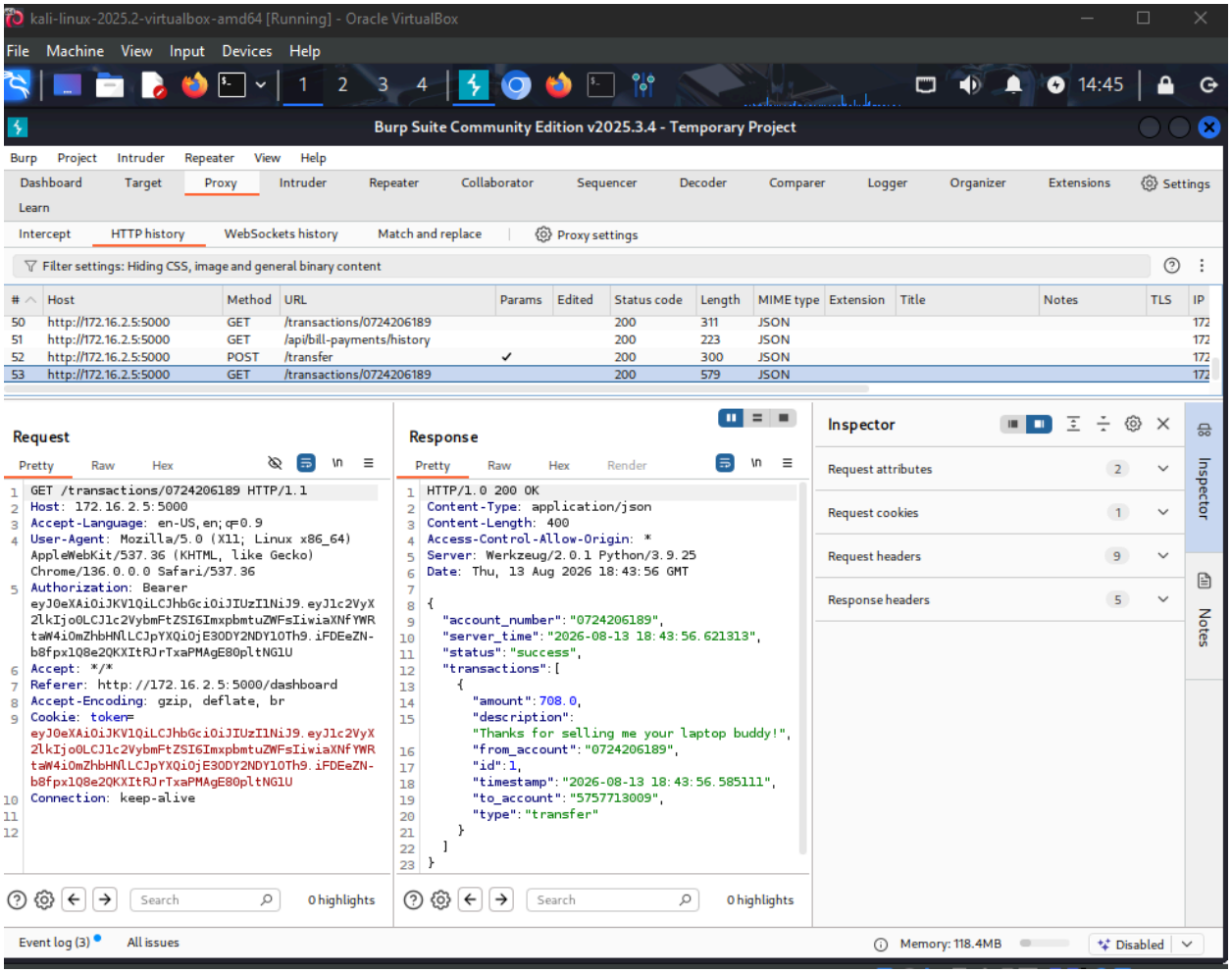


Figure 21 : *Burp Suite HTTP history and transaction record confirming the completed legitimate transfer.*

Phase 3 Testing a Negative Transfer Amount

The captured POST /transfer request was sent to Burp Repeater to test whether the amount field was validated server-side. An initial attempt to resend the request with "amount": "708" unmodified was rejected with HTTP 400 Bad Request and the message "Insufficient funds", confirming that linkneal's balance (reduced to $292 after the earlier legitimate transfer) was checked against the requested amount. A follow-up attempt using "amount": "-708" was also rejected with the same "Insufficient funds" message, showing the server compared the amount against the available balance regardless of how the money was going to move.

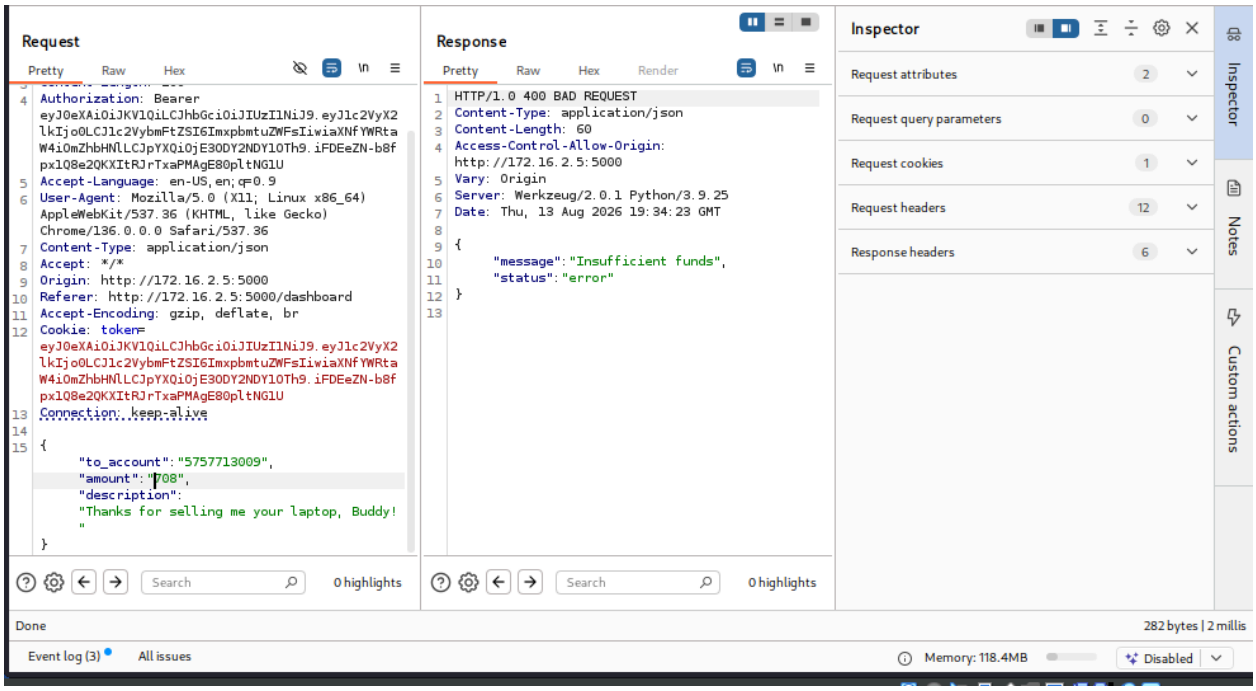


Figure 22: *Burp Repeater response showing rejection of the unmodified $708 resend ("Insufficient funds").*

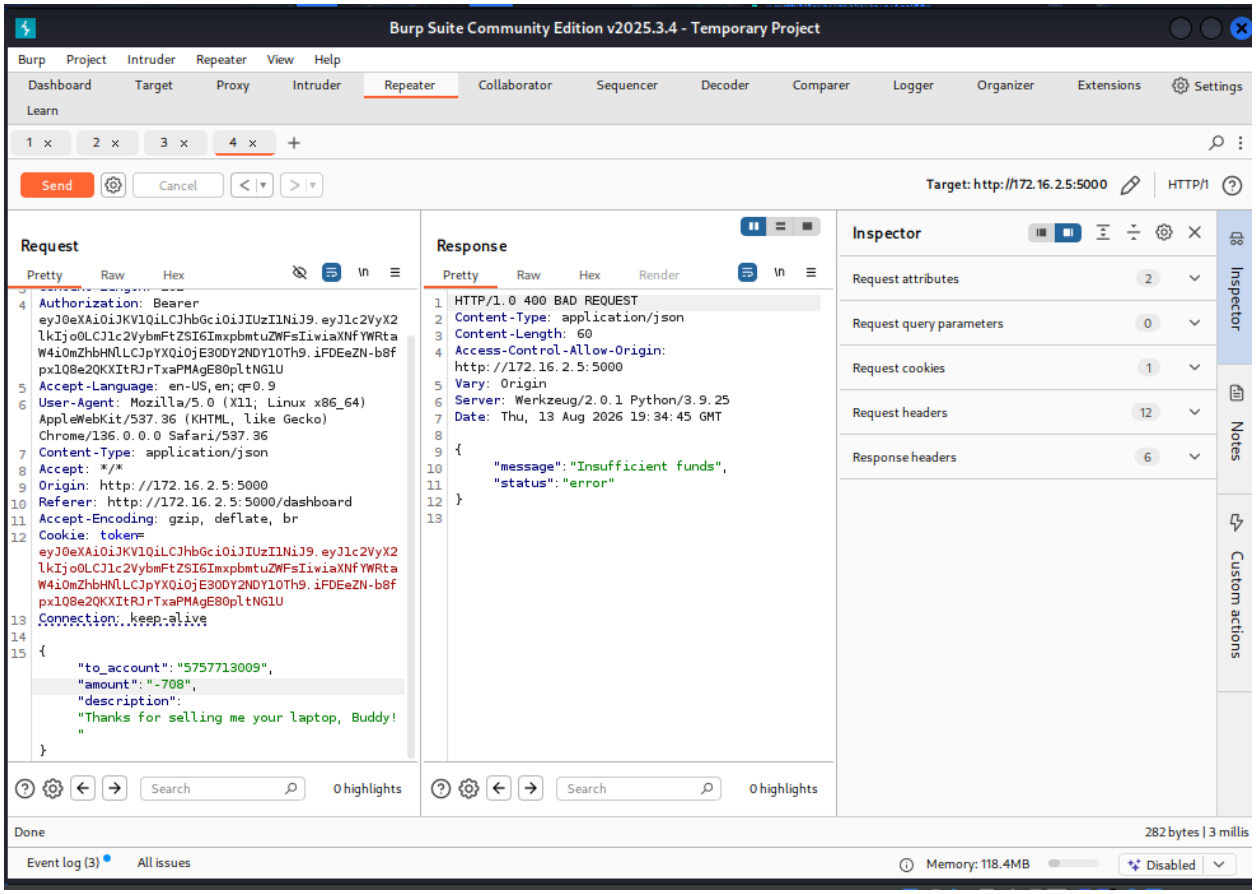


Figure 23: *Burp Repeater response showing rejection of a -708 resend ("Insufficient funds").*

Phase 4 Repeated Exploitation of the Negative-Amount Transfer

A different tactic was used to manipulate account transfer by sending a smaller amount. A packet was manipulated where the “to_account”: “ 5757713009” value (rhettmclaughlin) and “amount”: “-100” values were resent through Burp Repeater many times in succession. Each successful resend continued to credit linkneal's account by $100 per request rather than debiting it, demonstrating that the vulnerability could be exploited repeatedly to inflate an account balance without limit, rather than being a one-time logic error. After the repeated resends, the response confirmed a cumulative balance of $1,992.00 in linkneal's account.

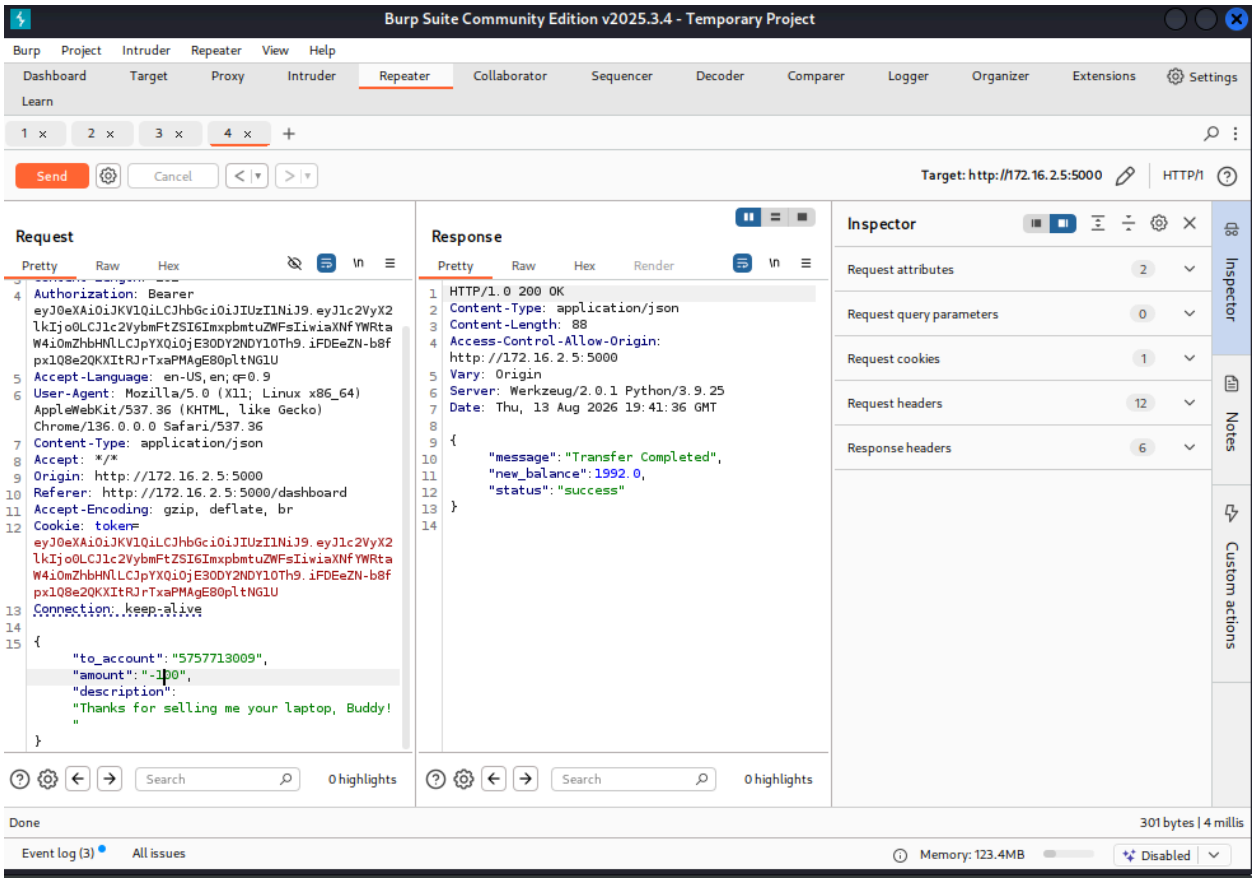


Figure 24 : *Burp Repeater response confirming acceptance of the -100 request and the resulting balance increase to $1992.00.*

Phase 5 Confirming Impact via Final Account Balances

Following repeated exploitation, both account balances were checked to confirm the real-world impact of the attack. linkneal's balance had risen to $1,992.00, while rhettmclaughlin's balance had fallen to $8.00, confirming that the repeated negative-amount transfers drained value from rhettmclaughlin's account and credited it to linkneal, rather than the reverse implied by the transfer request.

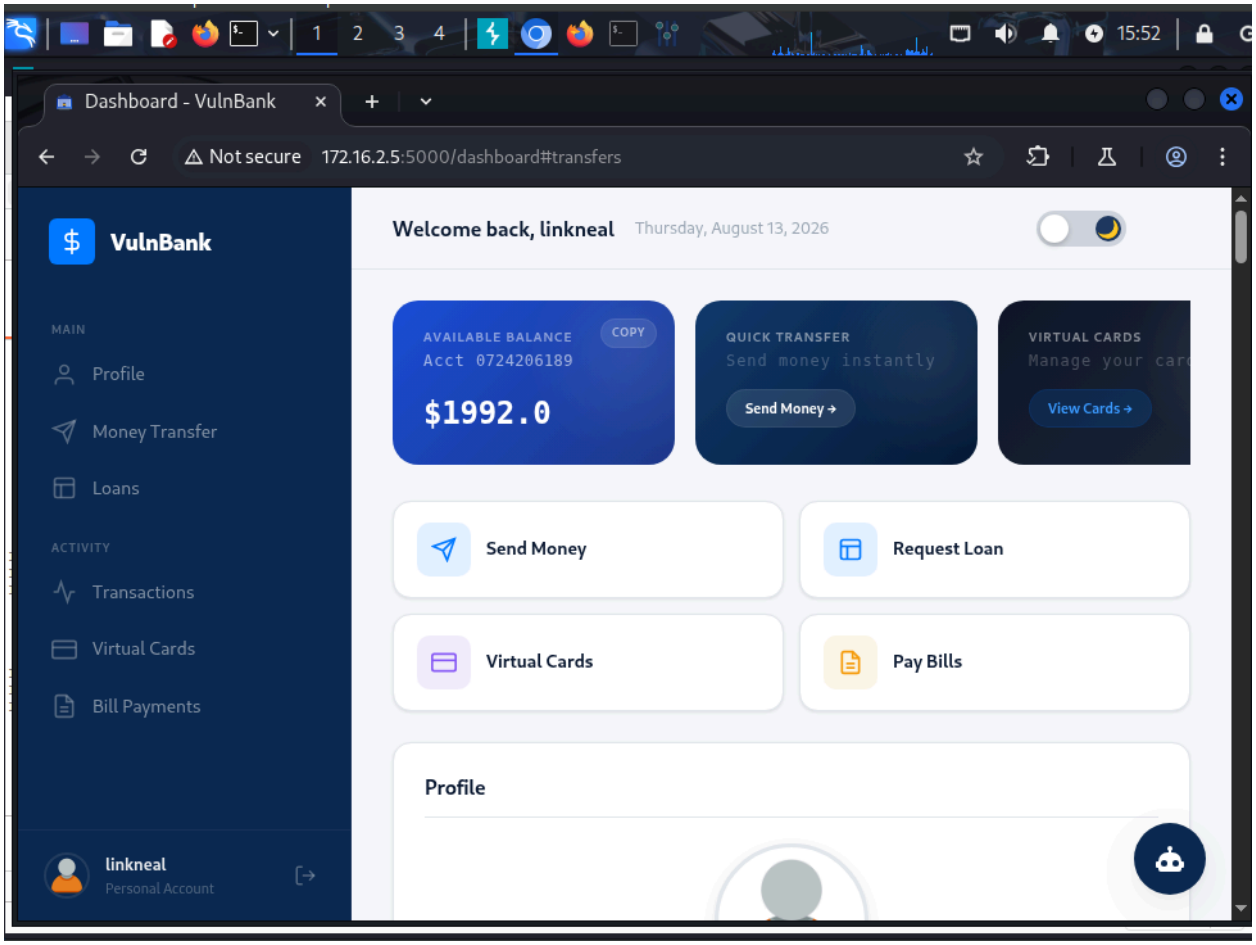


Figure 25: *linkneal's dashboard showing a final balance of $1,992.00.*

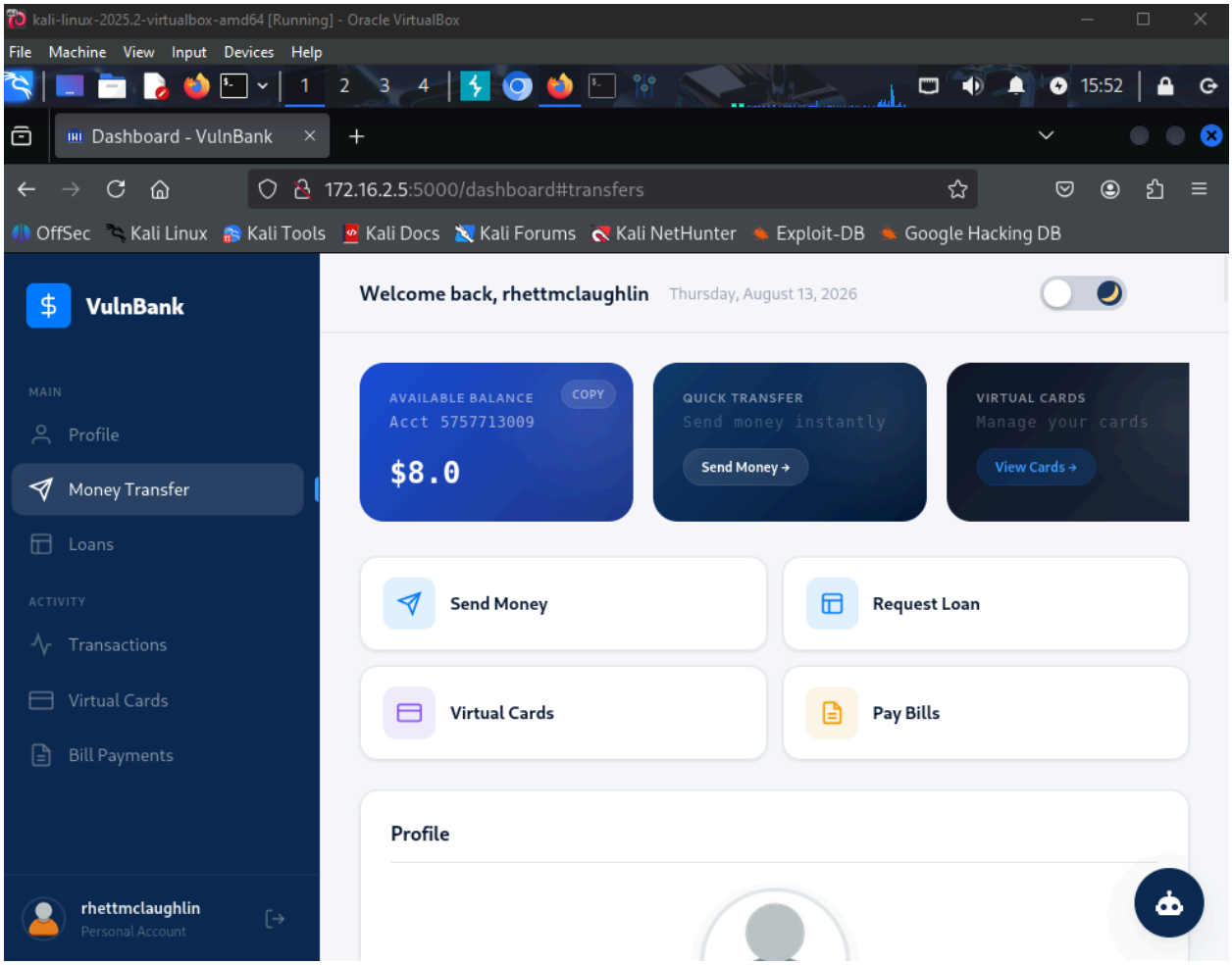


Figure 26: *rhettmclaughlin's dashboard showing a final balance of $8.00.*

6.3.4 Attack Flow Summary

The attack began with a legitimate $708 transfer from linkneal to rhettmclaughlin, captured in Burp Suite to establish the normal request format used by VulnBank's transfer endpoint. The captured request was sent to Burp Repeater, where an unmodified resend and a large negative resend were both rejected due to insufficient funds, showing that the server checked the magnitude of the requested amount against the sender's balance. A smaller negative amount, -100, was then submitted and accepted, revealing that the server did not validate the sign of the transfer amount and instead credited the sending account rather than debiting it. This request was resent repeatedly, each time increasing linkneal's balance, until linkneal's account held $1,992.00 and rhettmclaughlin's account had fallen to $8.00. This confirmed that the vulnerability could be exploited repeatedly to arbitrarily manipulate account balances between the two parties.

6.3.5 TTPs and Attacker Objective

The primary attacker objective in this scenario was to manipulate VulnBank's fund-transfer logic to move money in the attacker's favor without authorization. The adversary intercepted a legitimate transfer request, identified that the amount field lacked sign validation on the server, and repeatedly submitted negative-amount transfers to credit the attacker-controlled account at the expense of the victim account.

This scenario demonstrated how a business-logic flaw in a financial application rather than a traditional authentication bypass could be used to directly manipulate account integrity and siphon values between accounts.

6.3.6 MITRE ATT&CK Mapping

The attack was mapped to the MITRE ATT&CK framework based on the manipulation of transaction data through the exploited transfer endpoint.

Attack Activity	MITRE Tactic	Technique	ID
Submitting a modified transfer request with a negative amount value	Impact	Data Manipulation	T1565
Repeated exploitation to inflate account balance	Impact	Data Destruction / Manipulation	T1496

6.3.7 Impacted Systems and Attack Outcome

The primary system impacted during this scenario was the VulnBank web application's fund-transfer endpoint and the account balances of the two test accounts involved, rhettmclaughlin and linkneal.

The attack was successful. A negative transfer amount was accepted by the server and reversed the direction of the transaction, crediting the sending account instead of debiting it. Repeated exploitation of this behavior increased linkneal's balance to $1,992.00 and reduced

rhettmclaughlin's balance to $8.00, demonstrating that the flaw could be used to arbitrarily manipulate account balances well beyond a single transaction.

6.4 Scenario 4- GraphQL Trust Boundary Failure

6.4.1 Objective

The objective of this attack scenario was to evaluate whether the VulnBank GraphQL API's `transactionSummary` resolver could be abused beyond simple data exposure, and to determine how far an attacker could progress using only that single injection point. The scenario tested whether unauthenticated schema discovery could be chained into authentication bypass, remote code execution on the DMZ host, and lateral movement into the internal workstation zone, without ever cracking or brute-forcing a credential. A successful attack was defined as achieving code execution on the VulnBank backend through the GraphQL resolver, using artifacts discovered during that execution to pivot into a separate network zone, and confirming privileged domain access on the pivot target.

6.4.2 Target and Tools Used

The target of the scenario was the VulnBank GraphQL endpoint, hosted within the project's DMZ. The attack originated from the Kali Linux attacker machine using a custom Python driver script rather than manual Burp Suite interception, since the attack chain involved many sequential dependent requests.The attacker machine was separated from VulnBank according to the project's segmented architecture, and the pivot target (172.16.30.10) sat in a completely different network zone reached only through the compromised DMZ host.

Component	Purpose
Kali Linux	Attacker workstation used to conduct the controlled attack
Python 3 (requests library)	Custom driver script (terminal_chain.py) automating the full GraphQL attack chain
Netcat (nc)	Listener used to catch the reverse shell
VulnBank GraphQL API	Target resolver (172.16.10.10: 5000/graphql)
PostgreSQL COPY ... TO PROGRAM	Backend database feature abused for OS command execution
Burp Suite (GraphQL tab)	Construction and submission of GraphQL queries and JWT manipulation
Burp JWT Editor	Forgery of administrator tokens using the recovered signing secret
SSH (id_ed25519)	Credential discovered on the DMZ host and reused for lateral movement
Atomic Red Team	Post-exploitation technique execution for detection telemetry

Table X: Tools Used for 6.4's Scenario

6.4.3 Attack Chain and Execution

The chain ran in ten stages against a forged administrator JWT. Stages 1–5 and 7–9 are shown with representative screenshots; the repetitive discovery stages (6) are consolidated into Table 4.1, since each used the identical write-then-readback pattern described below.

Phase 1: Schema Reconnaissance and Endpoint Discovery

An unauthenticated `__schema` query confirmed a single exposed resolver, `transactionSummary`; a follow-up `__type` query against `TransactionRecordType` disclosed the field structure (`fromUsername`, `toUsername`, `fromAccount`, `toAccount`) and gave enough information to construct targeted queries without any prior documentation of the API.

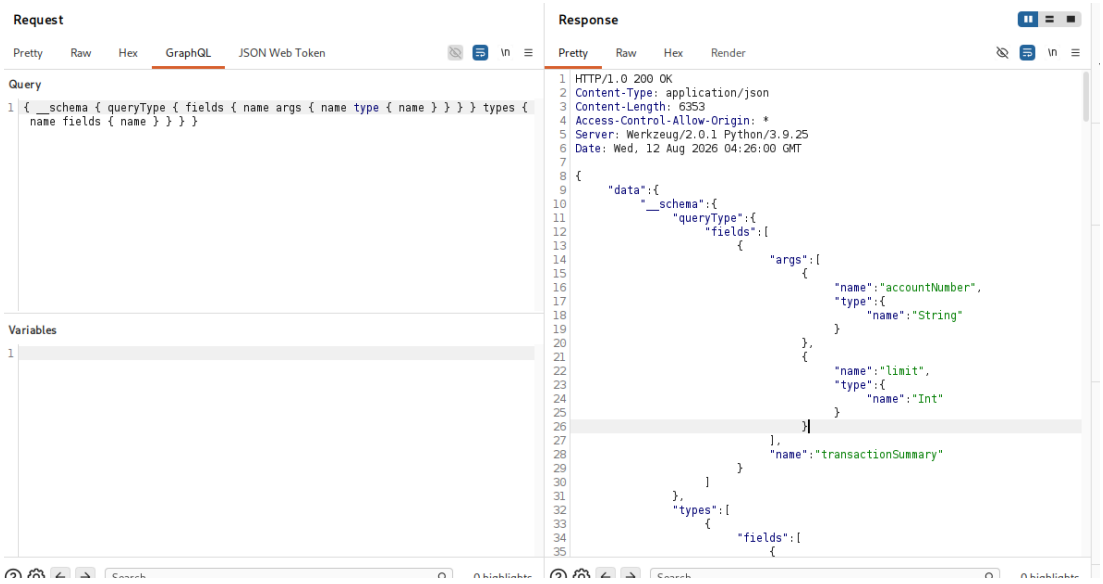


Figure 27: GraphQL `__schema` and `__type` introspection responses exposing the resolver and field structure.

Phase 2: Forging Administrator Authentication

Rather than attempting to steal or brute-force a credential, the JWT signing secret was recovered as a weak, guessable value (secret123). A token was forged offline with user_id: 1, username: admin, and is_admin: true, producing a valid, correctly-signed administrator token with no interaction with the actual login flow.

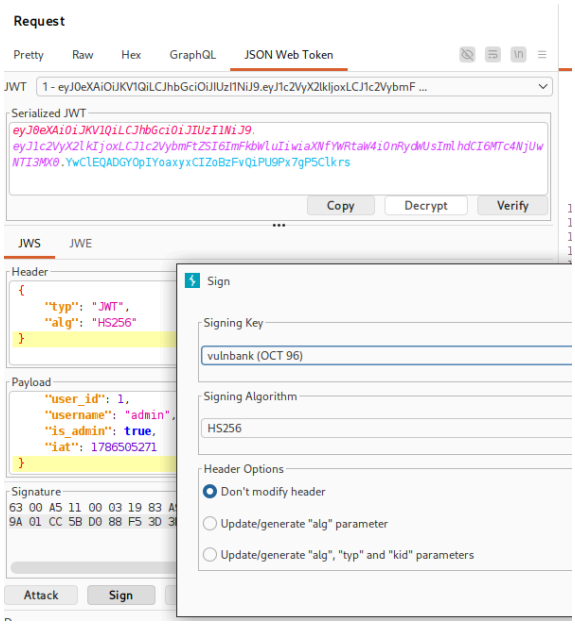


Figure 28: Forged administrator JWT generated using the recovered signing secret(secret123)

Phase 3: Authenticated Data Exfiltration

Using the forged token, a transactionSummary(limit: 15) query returned a full transaction ledger, usernames, account numbers, and transfer amounts across the customer base. This confirmed that the resolver's authorization was based entirely on JWT claims, and that forging those claims was sufficient to bypass the intended access control.

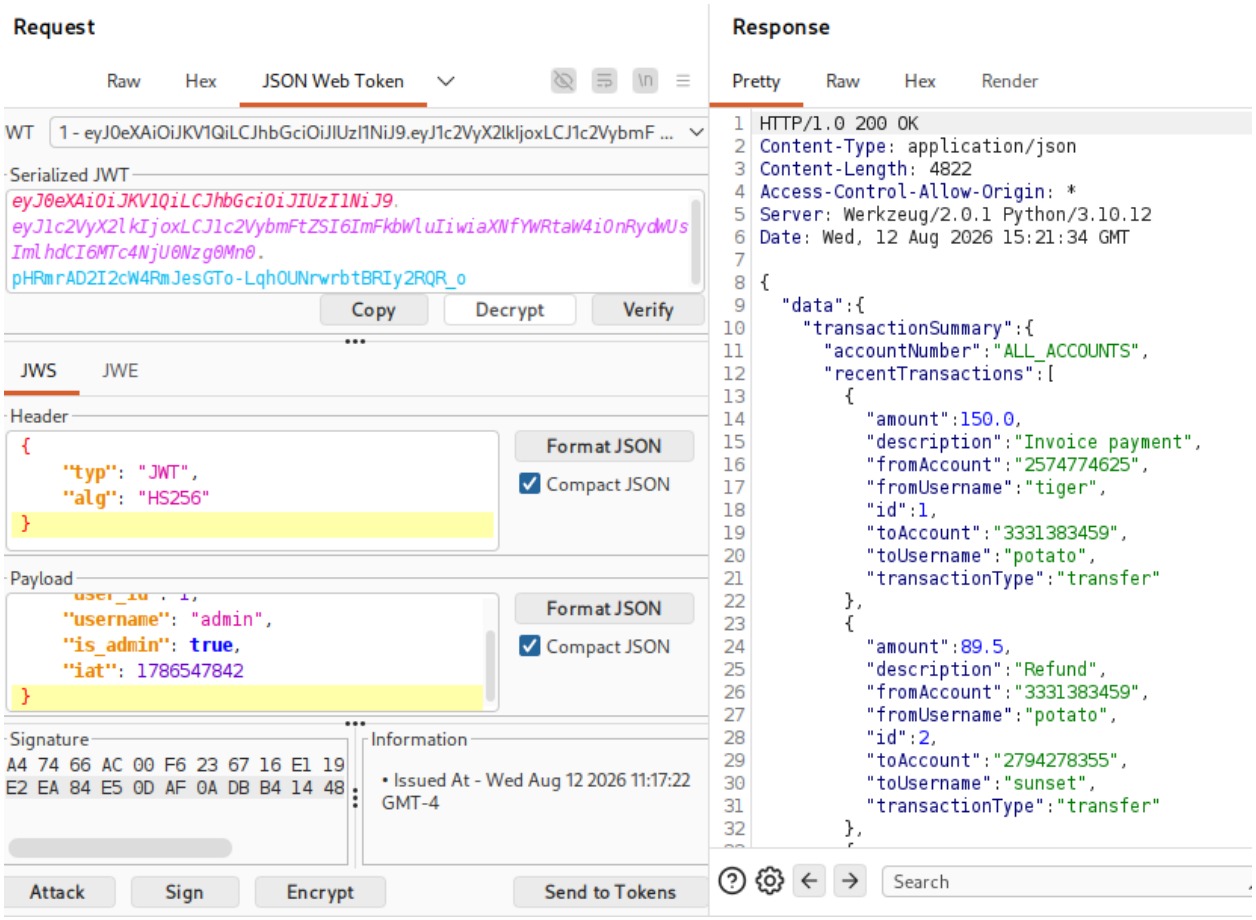


Figure 29: Authenticated GraphQL response returning full transaction records using the forged token.

Phase 4: SQL Injection and Arbitrary File Read

The accountNumber argument was found to be concatenated directly into a backend SQL query without sanitization. A UNION SELECT payload using PostgreSQL's pg_read_file() function was submitted through the same field used for legitimate account lookups, returning the full contents of /etc/passwd inside the accountNumber response field. This confirmed injectable SQL and enumerated local system accounts, including the postgres service account and the dmz monitoring agent account.

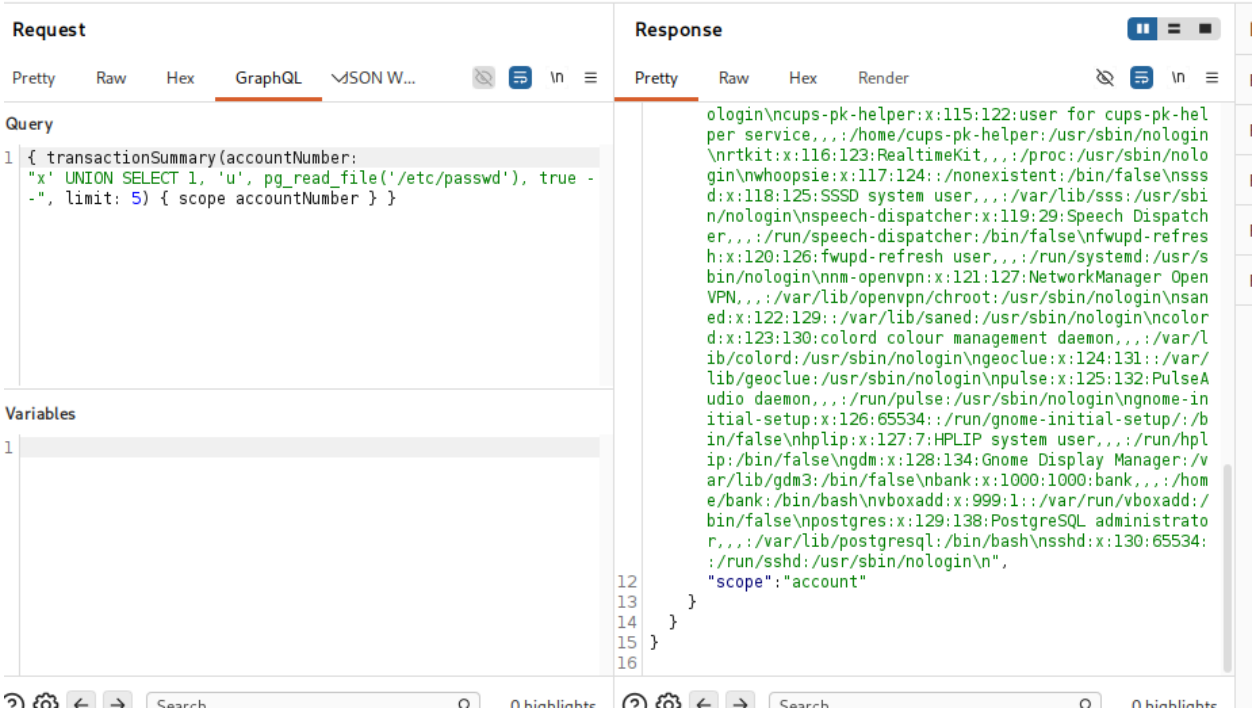


Figure 30: GraphQL response returning the contents of /etc/passwd via the injected UNION SELECT payload.

Phase 5: Remote Code Execution via COPY ... TO PROGRAM

The same injection point was used to invoke PostgreSQL's COPY (SELECT 1) TO PROGRAM, which executes an arbitrary operating-system command under the database service account. Because the API returns an error rather than command output, a two-step pattern was used for the remainder of the chain: run the command with output redirected to a file under /tmp, then issue a second injected query to read that file back. The first command executed, id, confirmed code execution as uid=129(postgres) gid=138(postgres).

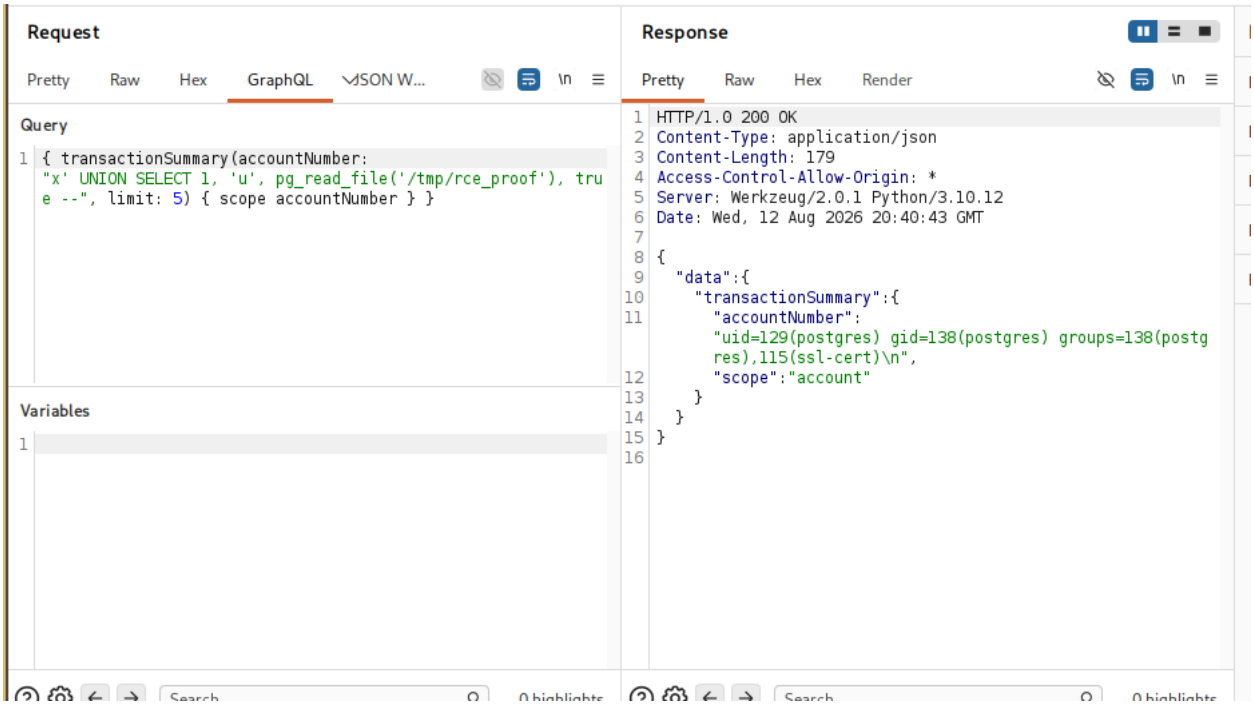


Figure 31: RCE query readback query confirming code execution as the postgres service account.

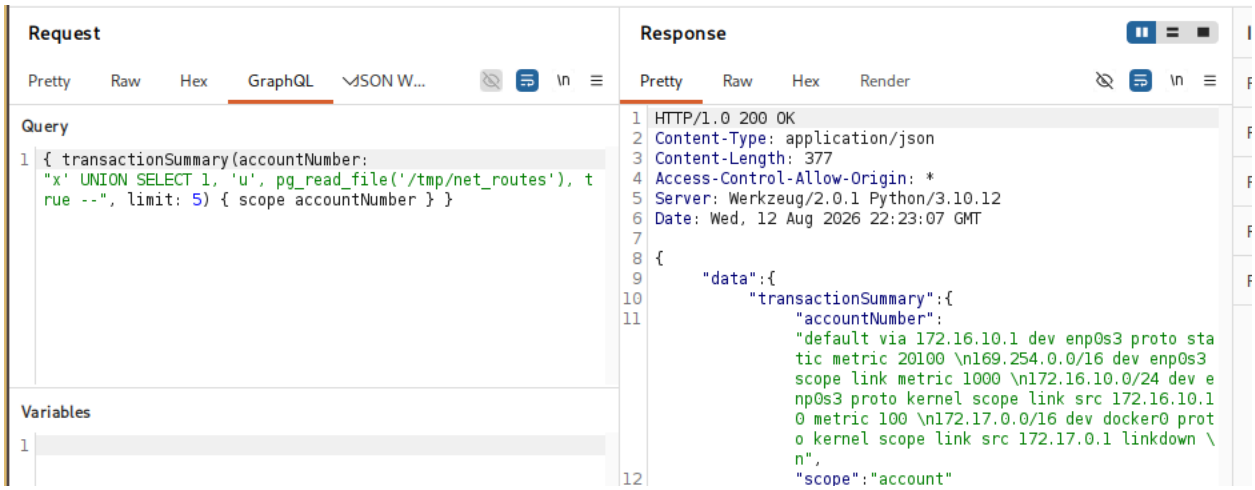
Phase 6: Host and Network Discovery

Using the same write-then-readback pattern, the chain enumerated login history (last -a), network interfaces (ip addr), and the routing table (ip route). This confirmed the DMZ host's address (172.16.10.10/24), a Docker bridge interface, and a default route with no visible restriction preventing outbound connections to other internal subnets.

Stage	Command / read target	What it showed
Login history	`last -a`	Sessions for `bank`, including two from `172.16.30.10`
Network interfaces	`ip addr`	DMZ host at `172.16.10.10/24` plus a Docker bridge
Routing table	`ip route`	Default gateway `172.16.10.1`; no egress restriction to internal subnets

Service account	`getent passwd postgres`	`postgres` (uid 129) with an interactive `/bin/bash` shell
postgres home	`ls -la /var/lib/postgresql/`	A `.ssh` directory with restrictive permissions
SSH directory	`ls -la .../.ssh/`	A live `id_ed25519` private key usable by the service account
Backup script	`pg_read_file('/opt/vuln-bank/backup.sh')`	Nightly `scp` of DB dumps to `adminloyee@172.16.30.10` using the same key
Backup log	`pg_read_file('/var/log/vuln-bank-backup.log')`	A recurring, scheduled, credentialed cross-segment transfer

Table X: Discovery and credential-harvesting stages



. **Figure 32:** Routing table retrieved via ip route.

Phase 7: Credential and Application Discovery

Further RCE calls enumerated the postgres OS account and confirmed its home directory contained a live SSH key pair (id_ed25519) with restrictive file permissions. Application-level reconnaissance recovered /opt/vuln-bank/backup.sh, VulnBank's own nightly backup script, which revealed that this same key was already used by a scheduled job to scp database dumps to adminloyee@172.16.30.10, a Windows host in the workstation zone. The associated backup log corroborated this as a recurring, credentialed, cross-segment connection that already existed before the attack began.

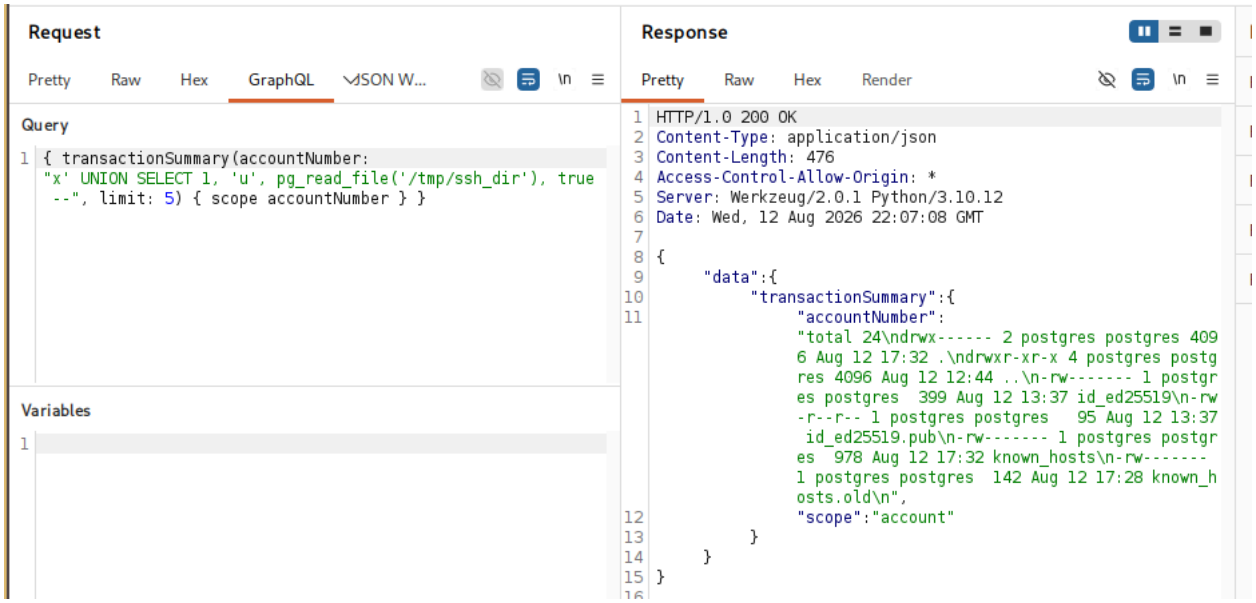


Figure 33: Directory listing of .ssh/ confirming a live private key.

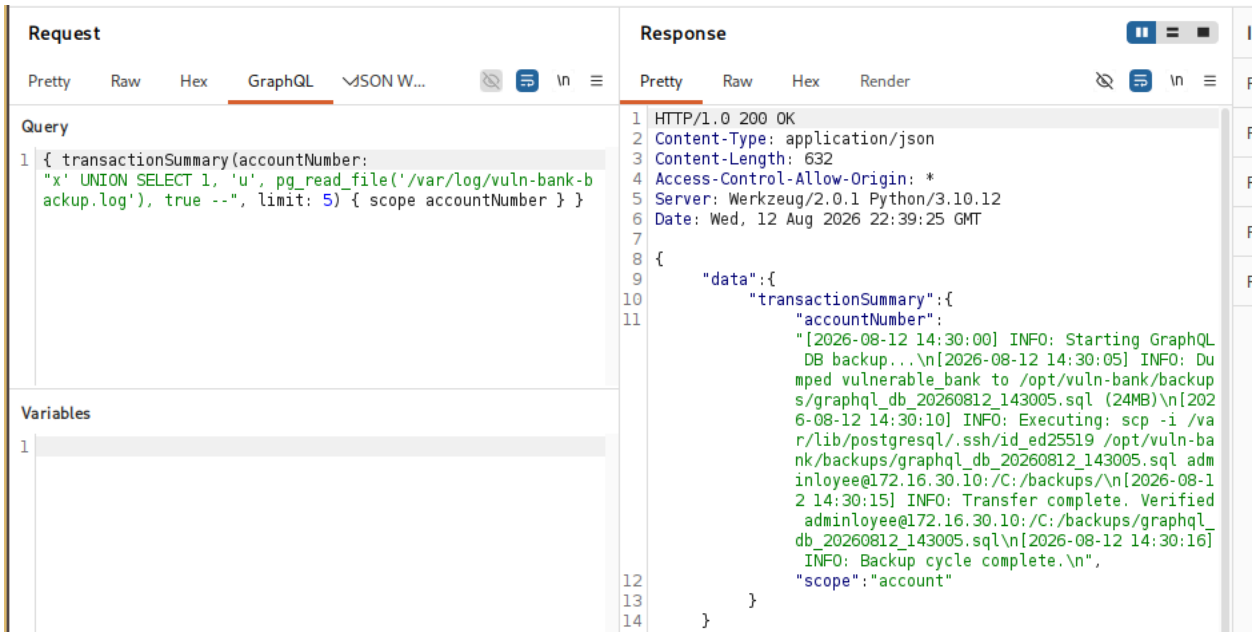
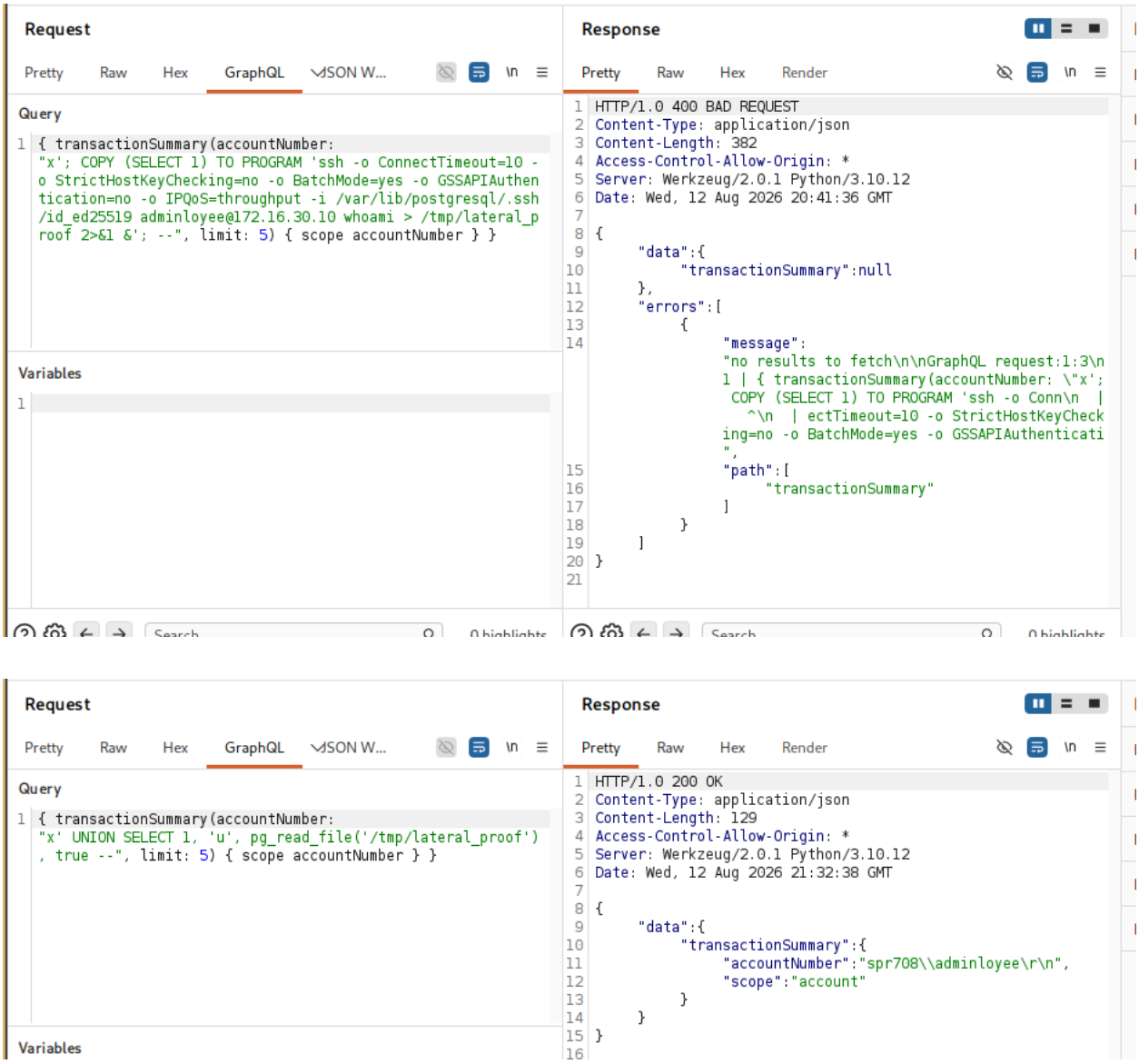


Figure 34: Backup log confirming the scheduled job's recurring execution and destination.

Phase 8: Lateral Movement to the Workstation Zone

Rather than needing to steal or crack a new credential, the same key and destination already used by the backup job were reused directly. An RCE-triggered ssh command from the DMZ host, authenticating as adminloyee to 172.16.30.10, returned spr708\adminloyee, confirming interactive command execution on the internal Windows workstation from the DMZ with no brute-forcing or credential theft required.



. **Figure 35:** Query & Readback confirming successful command execution as spr708\adminloyee.

Phase 9: Privilege Discovery on the Compromised Host

A follow-up whoami /all over the same SSH channel confirmed the compromised account is a member of SPR708\Domain Admins, with SeDebugPrivilege and SeImpersonatePrivilege enabled, both commonly abused for further credential access on Windows domain environments.

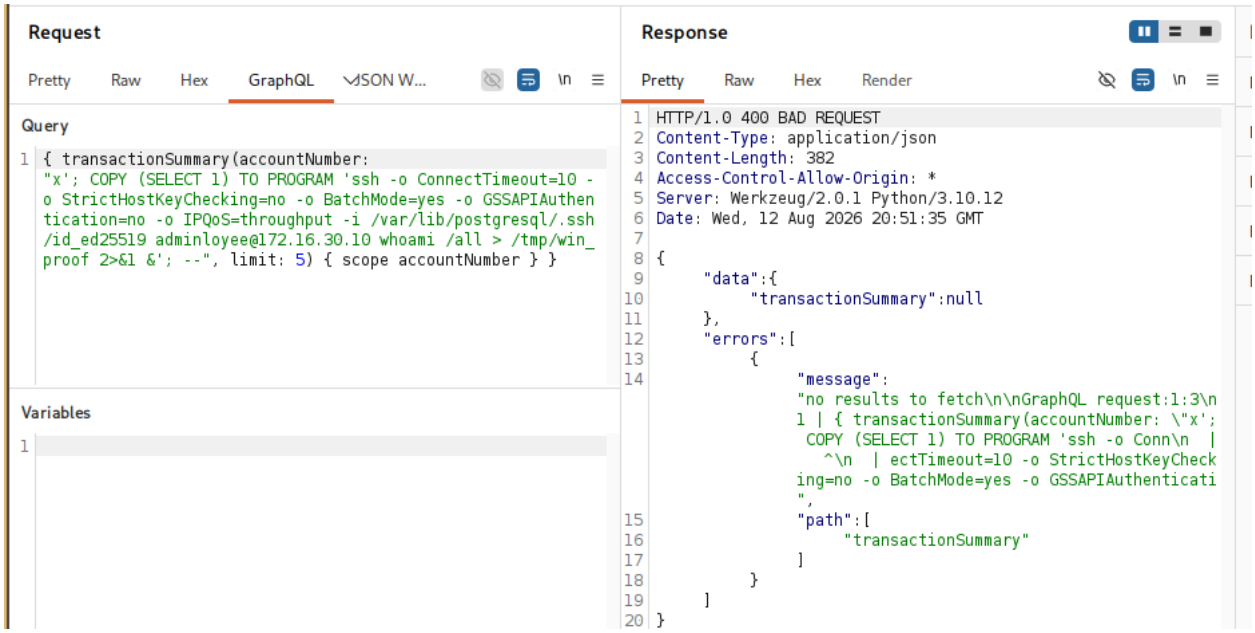


Figure 36: whoami /all command issued over the established SSH channel.

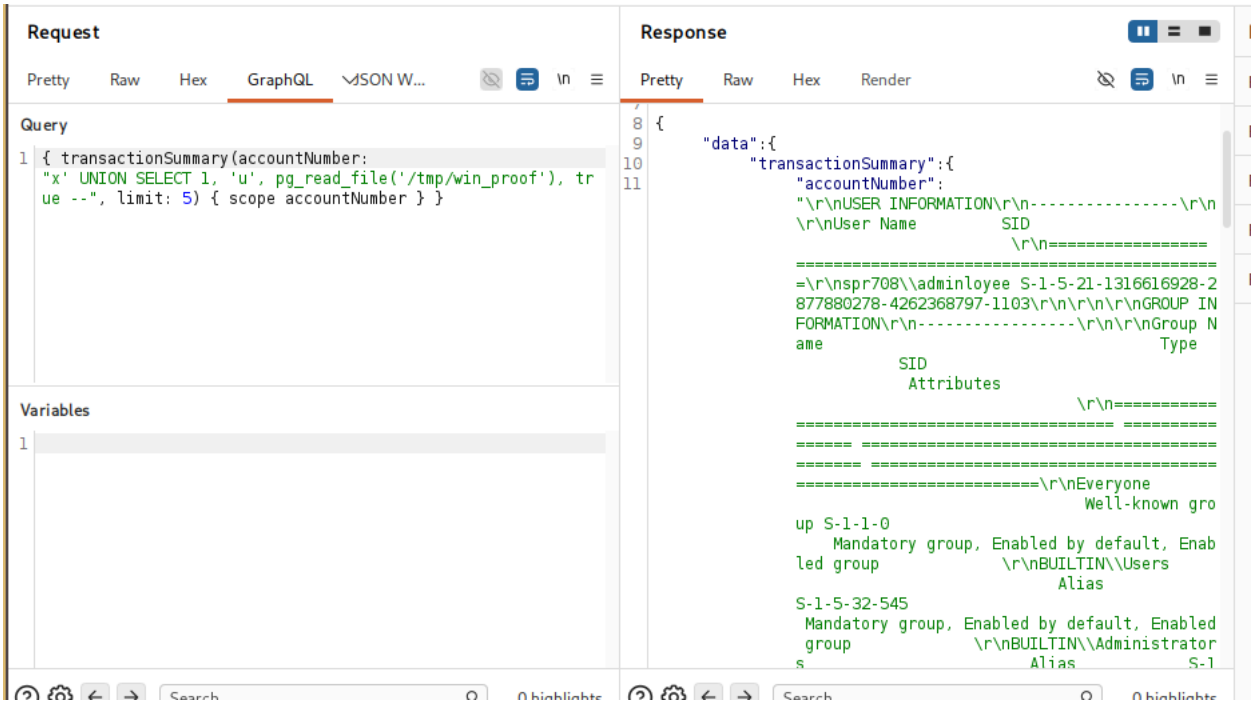


Figure 37: Readback confirming Domain Admins group membership and enabled privileges.

Phase 10: Reverse Shell Delivery Attempt

As a final step, the SSH RCE chained to curl.exe (pulling `rev.ps1` from an attacker HTTP server) and `powershell.exe -ep bypass -f`, catching an interactive shell and then utilizing Atomic Red Team to execute persistence, discovery, and credential-access techniques.

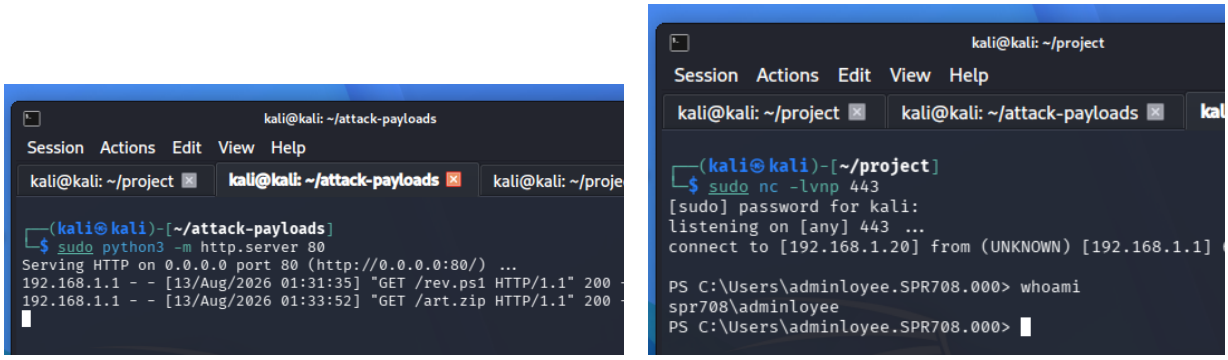


Figure 38: Attacker HTTP server staging payloads and the Netcat listener catching the workstation shell.

	Aug 13, 2026 @ 03:12:38.4...	workstation	CRITICAL: LSASS Dump via rundll32 comsvcs.dll - Credential Dumping	14	100305
	Aug 13, 2026 @ 03:12:37.1...	workstation	Scheduled Task Created - Persistence	10	100304
	Aug 13, 2026 @ 03:12:37.1...	workstation	Scheduled Task Created - Persistence	10	100304
	Aug 13, 2026 @ 03:10:18.5...	workstation	CRITICAL: LSASS Dump via rundll32 comsvcs.dll - Credential Dumping	14	100305
	Aug 13, 2026 @ 03:09:32.2...	workstation	CRITICAL: LSASS Dump via rundll32 comsvcs.dll - Credential Dumping	14	100305
	Aug 13, 2026 @ 03:09:32.0...	workstation	Scheduled Task Created - Persistence	10	100304
	Aug 13, 2026 @ 03:09:32.0...	workstation	Scheduled Task Created - Persistence	10	100304
	Aug 13, 2026 @ 03:08:49.8...	workstation	CRITICAL: LSASS Dump via rundll32 comsvcs.dll - Credential Dumping	14	100305
	Aug 13, 2026 @ 03:08:33.4...	workstation	Domain Account Enumeration - Discovery	8	100307
	Aug 13, 2026 @ 03:08:33.3...	workstation	Domain Account Enumeration - Discovery	8	100307

Figure 39: Atomic Red Team post-exploitation output, scheduled task (T1053.005), domain discovery (T1087.002), LSASS dump (T1003.001).

6.4.4 Attack Flow Summary

The attack began with unauthenticated schema introspection against the GraphQL endpoint, which was used to forge a valid administrator token from a weak signing secret. The forged token exposed the full transaction ledger and revealed an injectable parameter, which was escalated from file read to full remote code execution using PostgreSQL's COPY ... TO PROGRAM. Discovery commands run through this RCE surfaced network configuration, a live SSH private key belonging to the database service account, and the application's own backup script, which showed that key was already used for a scheduled connection into the internal workstation zone. That same credential and destination were reused directly to pivot into the

workstation, where a further discovery command confirmed the compromised account held Domain Admin privileges. A reverse shell was delivered for interactive access, and Atomic Red Team executed post-exploitation techniques to validate detection coverage.

6.4.5 TTPs and Attacker Objective

The primary attacker objective was to progress from unauthenticated access to a public-facing GraphQL API all the way to privileged access in the internal workstation zone. The adversary chained API reconnaissance, token forgery, injection-based code execution, and credential reuse rather than relying on credential theft or brute force. The scenario demonstrated how a single unsanitized input in a public-facing application, combined with an over-scoped automation credential, could bridge the boundary between the DMZ and the internal network.

The main adversary behavior involved interacting with the public-facing GraphQL API, forging authentication rather than stealing it, escalating an injectable parameter into OS-level command execution, and reusing an existing application credential to move laterally into a separate, more sensitive network zone. This scenario demonstrated how a single unsanitized parameter in a public-facing application can be chained into full cross-segment compromise when combined with excessive service account privileges and unrestricted credential scope.

6.4.6 MITRE ATT&CK Mapping

Attack Activity	MITRE Tactic	Technique	ID
Schema introspection	Reconnaissance	Active Scanning: Vulnerability Scanning	T1595.002
JWT forgery / auth bypass	Credential Access	Forge Web Credentials	T1606
Authenticated data dump	Collection	Data from Information Repositories	T1213
SQL injection	Initial Access	Exploit Public-Facing Application	T1190
Arbitrary file read	Collection	Data from Local System	T1005
RCE via `COPY TO PROGRAM`	Execution	Command and Scripting Interpreter: Unix Shell	T1059.004
Host/network discovery	Discovery	System Network Configuration Discovery	T1016
Account enumeration	Discovery	Account Discovery: Local Account	T1087.001
SSH private key reuse	Credential Access	Unsecured Credentials: Private Keys	T1552.004
SSH pivot to workstation	Lateral Movement	Remote Services: SSH	T1021.004

Reused backup credential	Initial Access / Lateral Movement	Valid Accounts	T1078
`whoami /all` on pivot host	Discovery	Permission Groups Discovery: Domain Groups	T1069.002
PowerShell payload staging	Command and Control	Ingress Tool Transfer	T1105
PowerShell execution	Execution	Command and Scripting Interpreter: PowerShell	T1059.001
Scheduled task persistence (ART)	Persistence	Scheduled Task/Job: Scheduled Task	T1053.005
LSASS memory dump (ART)	Credential Access	OS Credential Dumping: LSASS Memory	T1003.001
Domain account discovery (ART)	Discovery	Account Discovery: Domain Account	T1087.002

6.4.7 Impacted Systems and Attack Outcome

The primary systems impacted during this scenario were the VulnBank GraphQL API and its underlying PostgreSQL database, hosted within the DMZ, and the enterprise workstation in the internal workstation zone reached through lateral movement. The report's infrastructure design places both systems in separate, supposedly isolated network segments.

The attack was fully successful through Phase 9. A single unauthenticated injection point was escalated to full remote code execution on the DMZ host, and a credential intended for a narrow, automated backup task was reused to gain interactive access to a Domain Admin account on a host in a different network zone entirely, without cracking or exfiltrating a single password. The final reverse-shell delivery step did not produce a confirmed outcome and should be resolved before this section is finalized.

7. Detection and Response

During the Purple Team exercise, the Blue Team monitored the available security and application logs to determine whether each attack could be identified during the baseline phase. The purpose of this phase was to document what evidence was available before remediation, determine whether the existing monitoring controls generated an alert, and identify gaps that needed to be addressed.

7.1 Scenario 1 – API Version Downgrade and Password Reset Bypass

7.1.1 Detection Method

During the baseline execution of the API Version Downgrade and Password Reset Bypass scenario, the Blue Team reviewed the VulnBank web application logs and monitored Wazuh for corresponding security alerts.

The VulnBank application generated log entries for the API requests made during the attack. These logs provided visibility into the requested API endpoints and the HTTP responses returned by the application. Wazuh was also monitored to determine whether the existing SIEM configuration recognized the activity as suspicious.

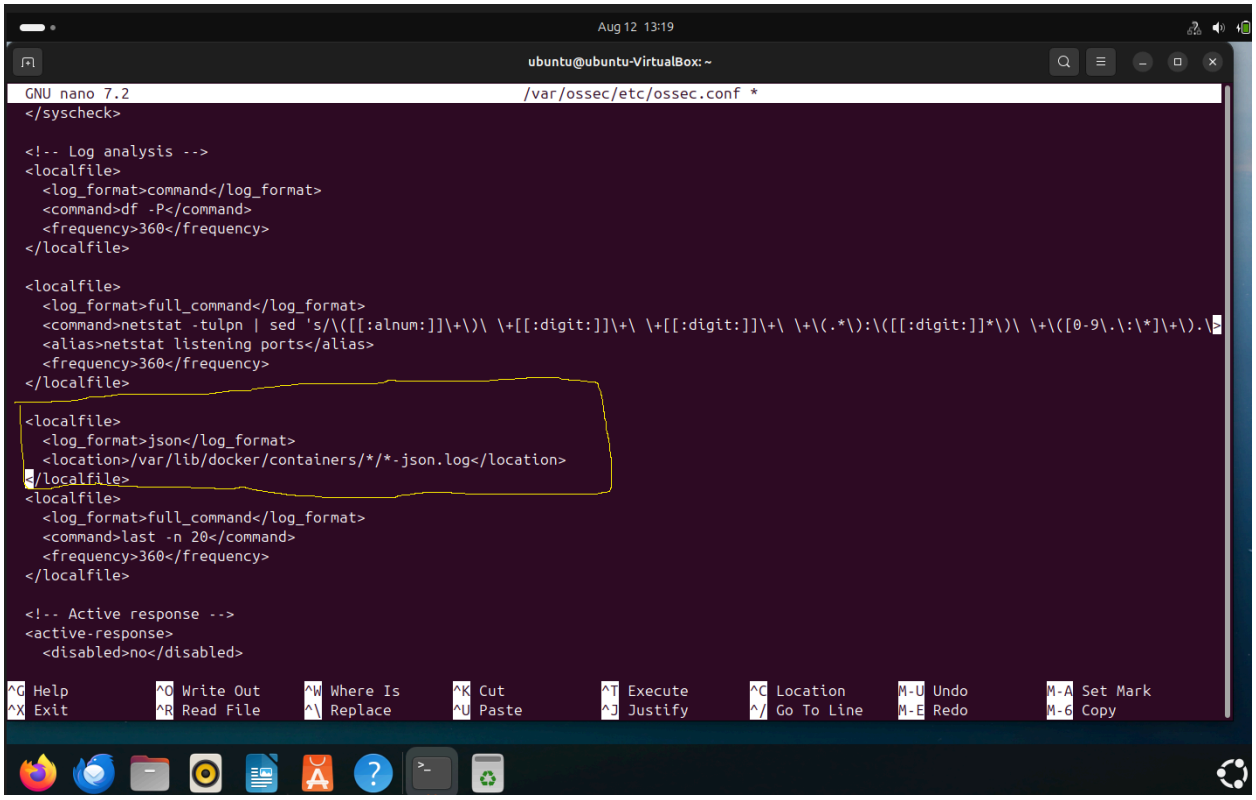


Figure 40: VulnBank web application logs added to the agent to digest

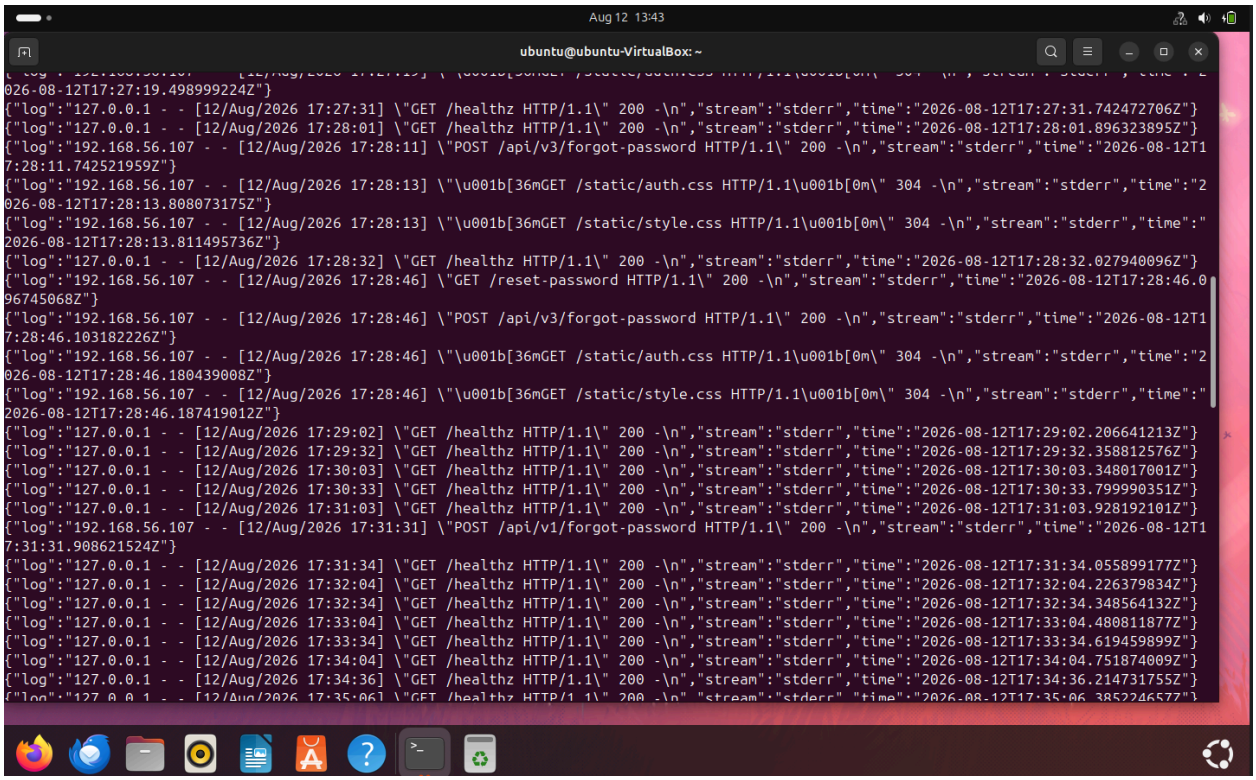


Figure 41: VulnBank web application logs showing API activity generated during the baseline password-reset attack.

7.1.2 Baseline Detection Results

The baseline application logs successfully recorded the API activity associated with the attack. This provided evidence that the underlying activity was visible at the application level.

However, no corresponding Wazuh alert was generated during the initial attack. Although the required telemetry existed within the VulnBank logs, the existing Wazuh configuration did not contain detection logic capable of identifying the API manipulation as suspicious activity.

As a result, the password-reset bypass was successfully completed without generating a SIEM alert. The administrator password was reset and the account was accessed, as demonstrated previously in Section 6.

Baseline Control	Result
VulnBank application logging	Activity recorded
Wazuh log monitoring	No relevant alert generated
Suspicious API activity identified automatically	No
Password-reset attack prevented	No
Overall baseline attack result	Successful

7.1.3 Response Actions

After the baseline attack was completed, the Blue and Purple Teams reviewed the VulnBank application logs to determine whether evidence of the attack was available.

The review confirmed that the suspicious API activity was present in the application logs even though Wazuh had not generated an alert. This showed that the issue was not a complete lack of logging. Instead, there was a gap between the available application telemetry and the SIEM's ability to recognize the activity as suspicious.

Based on this analysis, the team determined that both detection and preventive controls needed improvement. The specific controls implemented to address these weaknesses are documented later in Section 10 – Remediation.

7.1.4 Detection and Response Gaps

The main detection gap identified during Scenario 1 was the absence of a Wazuh rule capable of recognizing the suspicious VulnBank API activity.

The application produced logs containing evidence of the requests, but those events did not result in a meaningful Wazuh alert. Therefore, an analyst relying primarily on the SIEM would not have been automatically notified of the password-reset bypass during the baseline exercise.

A second gap was the lack of a preventive control protecting the vulnerable API path. Because the request was allowed to reach the application, the password-reset workflow could be manipulated and the attack completed successfully.

The Purple Team therefore identified two areas requiring improvement:

- **Detection gap:** Available VulnBank telemetry was not being converted into a Wazuh security alert.
- **Prevention gap:** The vulnerable API request was not being blocked before reaching the application.

These findings provided the basis for the detection and containment improvements implemented during the remediation phase.

7.2 Scenario 2 - Broken Object Level Authorization

7.2.1 Detection Method

The following blue team relied on the IDS router which allowed it to observe HTTP traffic between the kali attacker network but as well the VulnBank DMZ. Now suricata generated JSON events in /var/log/suricata/eve.json, and the wazuh agent installed on the IDS router which then forwarded suricata alerts to Wazuh manager 10.0.0.10.

7.2.2 Baseline Detection Results

Looking into the original suricata ruleset it did not contain any specific scenario requests in checking the /check_balance/ endpoint overall. So at first it did not produce a clear BOLA alert in wazuh.

7.2.3 Response Actions

After confirming the attack, the Red and Blue Teams reviewed the request path, the Kali source address, the VulnBank destination address and the available Suricata telemetry. The activity was stopped after sufficient evidence was collected because the scenario used controlled test accounts and did not modify financial records. The team then created a targeted network detection for the vulnerable endpoint.

7.2.4 Detection and Response Gaps

Now in the first gap it was missing authorization within the application. The second was missing a specific scenario detection rule. A further limitation was that network monitoring could ilook into access to /check_balance/ but could not see whether the requested account belonged to the requester. Higher-confidence detection would require application audit logs containing the authenticated user, requested account number and authorization decision.

7.3 Scenario 3 - Business Logic Abuse (Negative Amount Transfer)

7.3.1 Detection Method

During baseline testing, detection relied on the default Wazuh agent configuration installed on the host. The agent's ossec.conf configuration file was reviewed to determine which log sources were being monitored, and it was confirmed that no Docker listener module or container-log monitoring was configured. The only active localfile monitors covered system-level data — disk usage, listening ports, login history, journald, active-response logs, and dpkg logs — none of which capture VulnBank's application-level HTTP traffic or transaction activity, which is written to the VulnBank Docker container's console output.

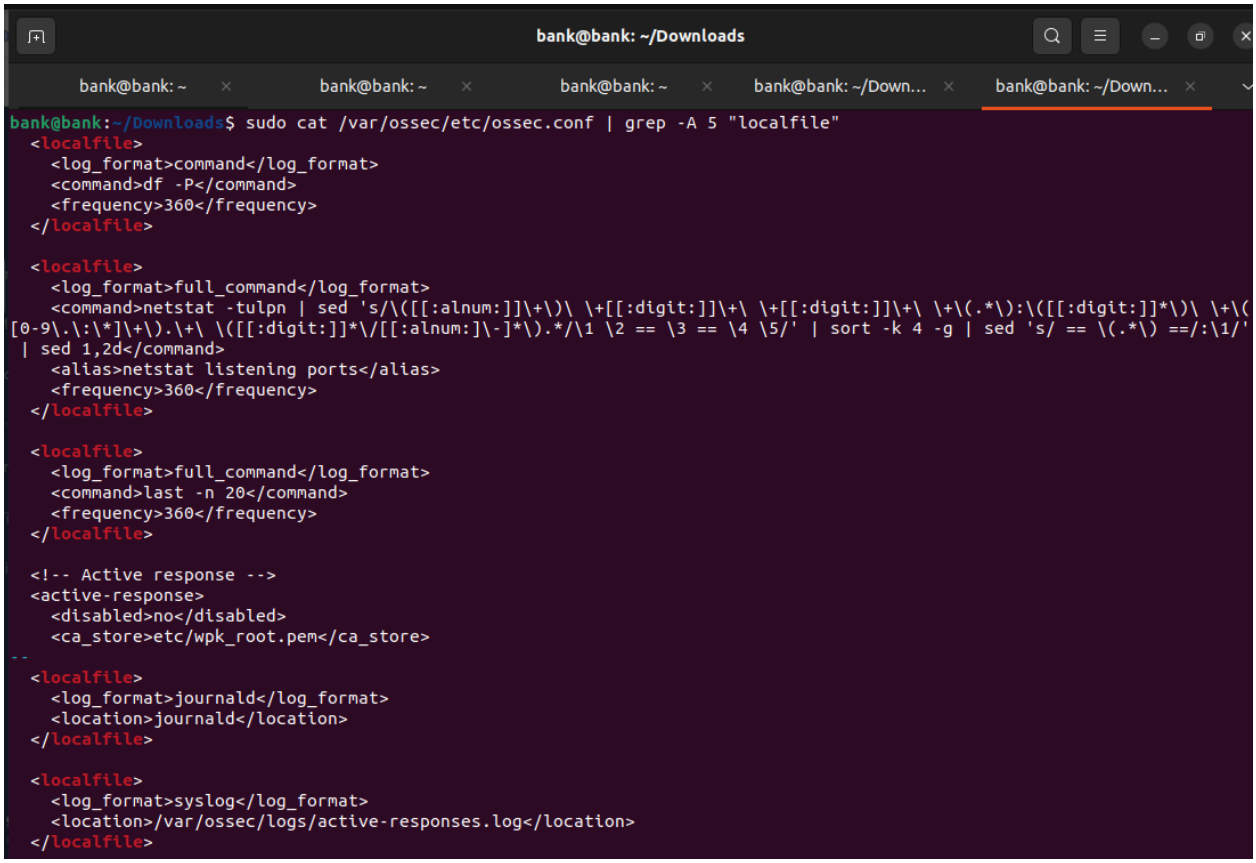


Figure 42: *Wazuh agent ossec.conf configuration confirming only default system-level localfile monitors were active with no VulnBank or Docker log source configured.*

7.3.2 Baseline Detection Results

Baseline Control	**Result**
Wazuh agent installed and running	Yes
Docker container log ingestion configured	No
VulnBank transfer requests visible to Wazuh	No
Custom detection rule for negative-amount transfers	Placeholder, not functional
Malicious negative-amount transfers detected	No
Attack prevented	No
Overall baseline attack result	Fully successful, undetected

7.3.3 Response Actions

Because the malicious transfer activity was never ingested by Wazuh, no alert was generated and no automated or manual response action was triggered during the baseline test. The attack was only identified after the fact, by manually reviewing VulnBank's transaction history and the Burp Suite request/response logs captured during testing not through any Blue Team monitoring capability.

7.3.4 Detection and Response Gaps

- **Logging gap:** VulnBank's application activity, including fund-transfer requests, was not being collected by the Wazuh agent. No Docker listener or equivalent log-forwarding mechanism was configured to capture the container's console output, meaning the SIEM had no data source for this activity at all.
- **Detection gap:** Even if logs had been ingested, no functional detection rule existed to flag a negative transfer amount as anomalous; only a placeholder rule had been created at the time of baseline testing.
- **Response gap:** With no alert generated, there was no automated or manual response the repeated exploitation of the transfer endpoint was allowed to continue unchecked throughout baseline testing.

7.4 Scenario 4 – GraphQL Trust Boundary Failure

7.4.1 Detection Method

During the baseline execution of the GraphQL Trust Boundary Failure scenario, the Blue Team monitored Wazuh for corresponding security alerts and reviewed telemetry from the Wazuh agent on the DMZ host, Sysmon on the workstation, and Suricata on the edge gateway. Each stage of the Scenario 4 attack chain was tracked in VECTR as a distinct procedure. The resulting pie chart shows the proportion of steps detected versus undetected during baseline execution.

7.4.2 Baseline Detection Results

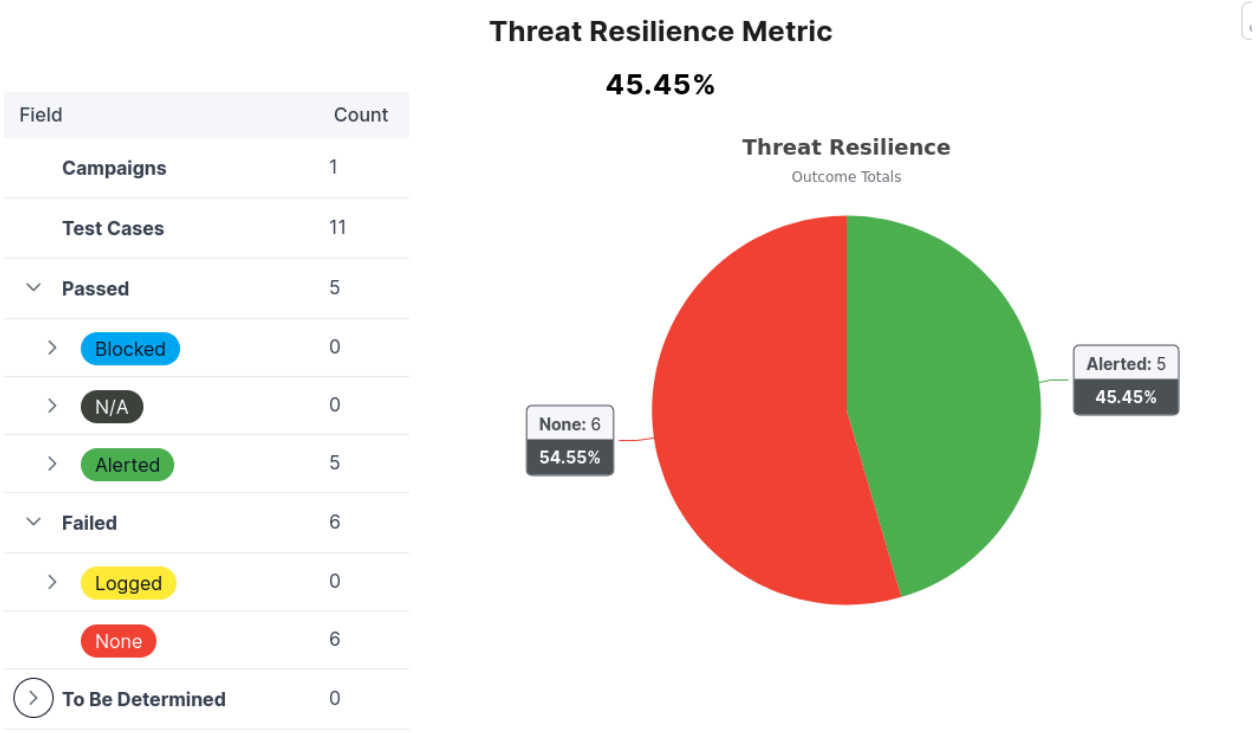


Figure 43: VECTR Baseline Assessment Resilience Metric (no custom rules)

The vast majority of the chain, GraphQL introspection, JWT forgery, SQL injection, PostgreSQL RCE, and the cross-segment SSH pivot, completed without generating any security alert. The small "Detected" slice does not represent purpose-built detection for this scenario. It consists solely of default Wazuh rules fired by generic Sysmon telemetry at the very end of the chain: Rule 92052 ("Windows command prompt started by an abnormal process") when the SSH session spawned cmd.exe, and Rule 92203 ("Executable file created by PowerShell") when the reverse shell staged Atomic Red Team on disk. These were low-fidelity, single-host indicators that did not identify the GraphQL injection, the database-level code execution, or the lateral

movement path. An analyst reviewing them in isolation could not have reconstructed the attack chain. The baseline environment was effectively blind to the primary attack vector.

7.4.3 Response Actions

During the baseline execution of Scenario 4, no automated response was triggered because the attack progressed entirely undetected. The Blue Team manually reviewed the VulnBank application logs, PostgreSQL logs, and Sysmon telemetry on the DMZ host after the Red Team confirmed successful lateral movement. The manual review confirmed that the `postgres` OS account had spawned an `ssh` child process, and that the authenticated session to `172.16.30.10` originated from the DMZ server. Because no alert had fired, containment was performed manually: the Red Team terminated the SSH session, and the Blue Team temporarily removed the `id_ed25519` private key from the DMZ host to prevent further pivoting. These actions were reactive rather than proactive, highlighting the absence of real-time detection.

7.4.4 Detection and Response Gaps

1. No GraphQL-aware network detection. Suricata's default ruleset is designed for traditional web attacks and cannot inspect GraphQL payloads embedded in JSON request bodies. As a result, the introspection queries, `UNION SELECT` injection, and `COPY TO PROGRAM` sequences all transited the gateway without triggering a single signature.
2. No application-layer SIEM rules for PostgreSQL RCE.The Wazuh agent on the DMZ host forwarded system logs but had no decoder or rule to parse PostgreSQL statement logs. The actual code execution mechanism `COPY (SELECT 1) TO PROGRAM` and `pg_read_file()`occurred entirely inside the database engine and was invisible to the SIEM.
3. No cross-host correlation for service-account abuse. Sysmon recorded the inbound SSH session and `cmd.exe` execution on the workstation, while the DMZ host logged the `postgres` account initiating outbound SSH. Because no correlation rule linked these two events, the lateral movement appeared as two isolated, low-severity entries rather than a single attack chain.
4. No network segmentation enforcement. The gateway firewall permitted outbound TCP/22 from the DMZ subnet (`172.16.10.0/24`) directly into the workstation subnet (`172.16.30.0/24`). A public-facing DMZ host should never be able to initiate interactive shell connections to internal endpoints, yet the pivot succeeded on the first attempt.
5. Weak identity and credential controls. The JWT signing secret (`secret123`) was guessable, enabling offline administrator token forgery. The backup SSH key had no `command=` restriction in `authorized_keys`, allowing the same credential provisioned for automated file transfers to grant full interactive shell access.

8. Findings

The baseline Purple Team exercise identified security weaknesses that allowed the simulated attacks to succeed or remain undetected. The findings below focus on the specific security failures demonstrated during each attack scenario.

8.1 Scenario 1 – API Version Downgrade and Password Reset Bypass

8.1.1 API Version Downgrade Vulnerability

The primary application security failure identified during Scenario 1 was inconsistent security between VulnBank's API versions. The normal password-reset workflow required a four-digit PIN, while modifying the request to use the alternate API version resulted in the disclosure of a valid three-digit reset PIN.

The weaker API version remained accessible and did not enforce the same protections as the normal application workflow. This allowed the password-reset process presented through the web interface to be bypassed.

8.1.2 Exposure of Sensitive Reset Information

The alternate API response exposed the valid password-reset PIN. This information could then be reused in the password-reset request to change the administrator password.

The successful reset demonstrated that sensitive account-recovery information was exposed by the application and could directly contribute to unauthorized account access.

8.1.3 Insufficient Server-Side Protection

The password-reset requirements were not consistently enforced by the backend. Although the normal interface expected a four-digit PIN, the backend accepted the three-digit PIN obtained through the alternate API version.

This allowed a manipulated API request to bypass the behavior enforced through the normal web interface and successfully change the administrator password.

8.1.4 Weak Logging and Missing Detection

The VulnBank application generated logs containing evidence of the suspicious API activity, meaning the activity was not completely invisible. However, the baseline monitoring configuration did not provide sufficient centralized detection of this behavior.

Most importantly, Wazuh did not generate an alert during the baseline attack. The relevant activity existed within the application logs, but there was no detection rule capable of identifying the API manipulation and alerting the Blue Team.

Therefore, the primary monitoring failure was not a complete absence of logs, but rather insufficient use of the available logging and missing SIEM detection logic.

8.1.5 Overall Finding

Scenario 1 identified weaknesses in both VulnBank's application security and its monitoring capabilities. A weaker API version exposed sensitive password-reset information, the backend accepted the manipulated reset request, and the baseline Wazuh configuration did not detect the suspicious activity.

Together, these failures allowed the administrator password to be reset and the admin@vulnbank.org account to be accessed successfully without generating a corresponding Wazuh alert.

8.3 Scenario 3 - Business Logic Abuse (Negative Amount Transfer)

8.3.1 Missing Server-Side Validation of Transfer Direction

The core application security failure identified during Scenario 3 was the absence of sign validation on the transfer amount field. VulnBank's fund-transfer endpoint accepted negative values in the amount field without rejecting them, allowing a request nominally structured as a transfer out of an account to instead credit that same account. This meant the direction of a financial transaction could be reversed simply by modifying a single field in the request, without needing to bypass authentication or manipulate any other parameter.

8.3.2 Flawed Funds-Validation Logic

Testing also revealed that the server's funds check compared the magnitude of the requested amount against the sender's balance, regardless of the amount's sign. Resending the original request with an unmodified positive value, and separately with a large negative value, were both rejected with an "Insufficient funds" message once the sender's balance had been reduced.

This showed that a balance check existed, but it was not paired with any validation of transaction direction. An attacker only needed to keep the requested amount within the sender's current balance to bypass the check entirely, regardless of whether the transaction was legitimate.

8.3.3 Unbounded, Repeatable Exploitation

Once a negative-amount transfer was accepted, the same request could be resent an unlimited number of times with no rate limiting, transaction caps, or anomaly detection preventing repetition. Each resend independently succeeded, allowing linkneal's balance to be inflated from an initial $1,000 to $1,992.00, while rhettmclaughlin's balance was driven down to $8.00. This escalated the impact of the vulnerability significantly: rather than a single manipulated transaction, the flaw enabled effectively unlimited value transfer between accounts.

8.3.4 Weak Logging and Missing Detection

VulnBank's transfer activity was written only to the application's Docker container console output, and the Wazuh agent was not configured with a Docker listener or any equivalent log-forwarding mechanism. As a result, none of the negative-amount transfer requests were ever ingested by the SIEM. Even had ingestion been configured, no functional Wazuh detection rule existed at the time of baseline testing to flag a negative transfer amount as anomalous — only a placeholder rule had been created. The baseline monitoring configuration therefore provided no visibility into the attack at any stage.

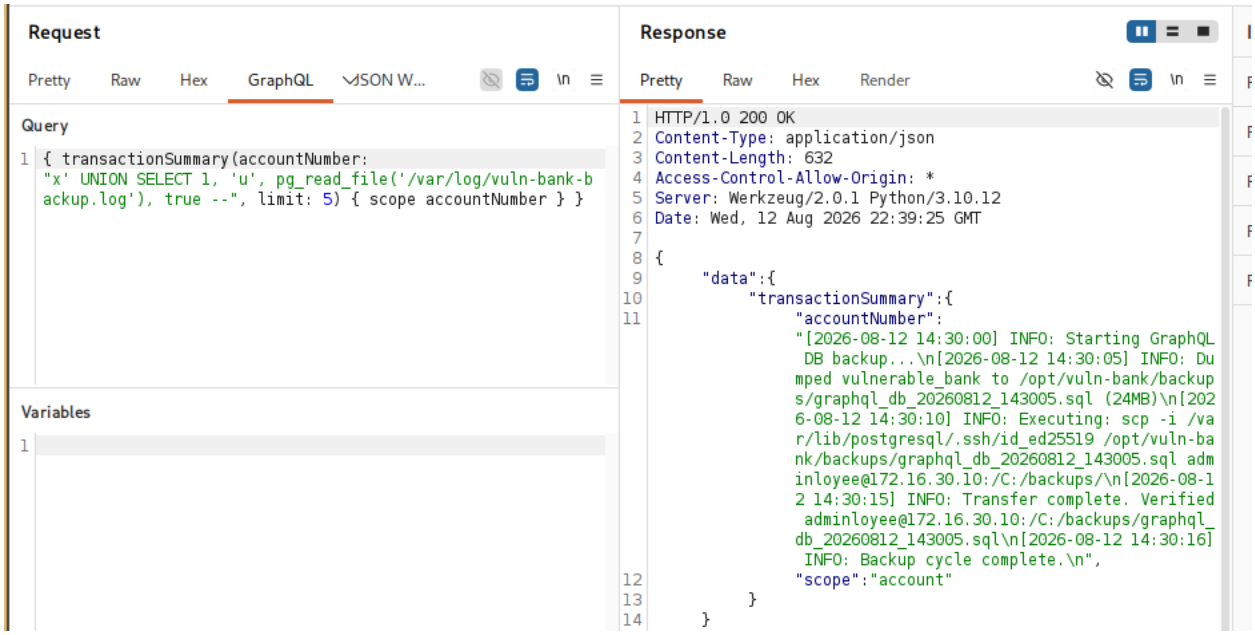
8.3.5 Overall Finding

Scenario 3 identified weaknesses in both VulnBank's transaction logic and its monitoring capabilities. The transfer endpoint failed to validate the sign of the amount field, its funds check could be satisfied regardless of transaction direction, and no rate limiting prevented the same flaw from being exploited repeatedly. Together, these failures allowed an attacker-controlled account to be credited $992.00 above its starting balance while a victim account was driven from $1,000.00 to $8.00, entirely without generating a Wazuh alert or triggering any automated response.

8.4 Scenario 4 – GraphQL Trust Boundary Failure

8.4.1 Unsanitized Input Reaching a Raw SQL Context

The `accountNumber` parameter was concatenated into the backend query without parameterization, enabling both blind data exfiltration (`UNION SELECT`) and OS-level code execution (`COPY TO PROGRAM`) through the same injection point.

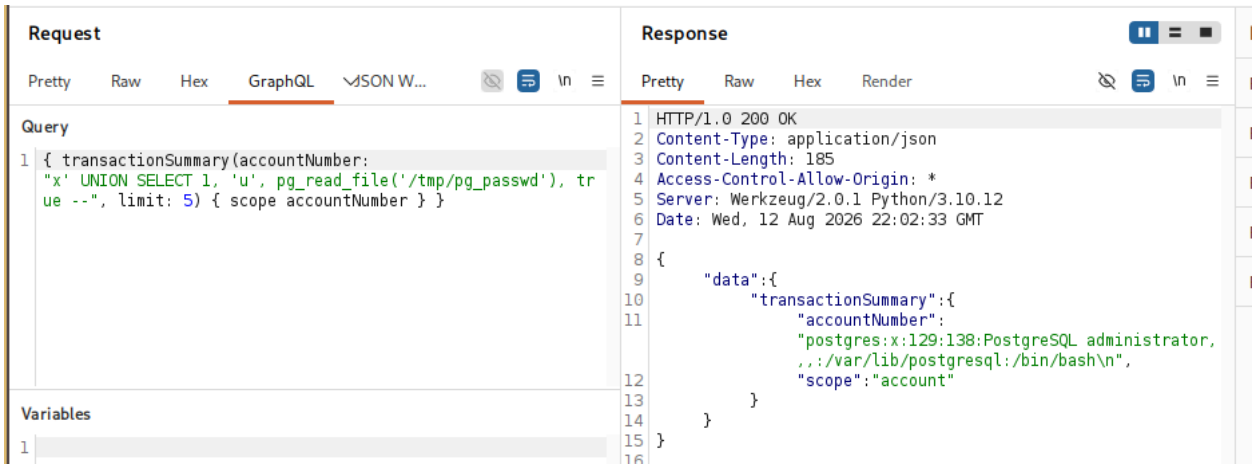


8.4.2 Excessive Database Service Account Privileges

The `postgres` service account had filesystem read access outside the database's own data directory and the ability to invoke `COPY TO PROGRAM`, a superuser-only capability that should not be enabled on an internet-facing database role

8.4.3 Credential Scope Violation

An SSH key provisioned for a single automated backup task had no restrictions, such as a forced command in `authorized_keys` preventing its use for arbitrary interactive shell access to the same destination.



8.4.4 Missing Network Segmentation Enforcement

Outbound SSH from the DMZ zone (`172.16.10.0/24`) to the workstation zone (`172.16.30.0/24`) was not blocked at the routing layer, allowing the pivot to succeed on the first attempt.

8.4.5 Weak JWT Signing Secret

The JWT signing secret was weak and guessable (`secret123`), allowing full administrator token forgery without interacting with the real authentication flow.

8.4.6 Overall Finding

Scenario 4 demonstrated that a single unsanitized input in a public-facing GraphQL resolver could be chained into full cross-segment compromise. The attack progressed from unauthenticated schema discovery to forged administrator authentication, SQL injection, OS-level command execution, and lateral movement into the internal workstation zone, all without brute-forcing a single credential. The root causes were a weak JWT signing secret, absence of parameterized queries, excessive privileges assigned to the PostgreSQL service account, a backup credential with no scope restrictions, and flat network segmentation that

allowed DMZ hosts to initiate SSH sessions into internal zones. The baseline monitoring stack provided no automated detection at any stage of this chain.

9. Recommendations

9.1 Scenario 1 – API Version Downgrade and Password Reset Bypass

9.1.1 Disable or Restrict Vulnerable API Versions

Older or weaker API versions should not remain publicly accessible if they do not enforce the same security controls as the current application workflow. Deprecated API versions should be removed where possible or restricted so that only approved versions can be accessed.

This would reduce the risk of an attacker bypassing stronger controls by manually changing the API version within a request.

9.1.2 Strengthen Server-Side Password Reset Validation

Password-reset requirements should be enforced by the backend rather than relying on the web interface. The server should validate the expected PIN format, verify that the PIN is associated with the correct account and reset request, and reject values that do not meet the required conditions.

Sensitive reset information should also never be returned directly within an API response.

9.1.3 Improve Monitoring and SIEM Detection

VulnBank application logs should be continuously forwarded to Wazuh and monitored using custom detection logic designed for suspicious API behavior.

Detection rules should identify activity such as:

- access to deprecated or restricted API versions;
- unusual password-reset requests;
- repeated reset attempts;
- unexpected PIN formats;
- suspicious modification of account-recovery requests.

This would allow the Blue Team to receive an alert instead of relying on manual review of the raw application logs.

9.1.4 Add a Reverse Proxy or Application-Layer Control

A reverse proxy should be placed in front of VulnBank to provide an additional security layer between clients and the backend application.

The proxy can restrict access to vulnerable or deprecated API endpoints and reject requests that should no longer reach the application. This provides containment even if the vulnerable application code has not yet been modified.

9.1.5 Apply Rate Limiting and Account-Recovery Protections

Password-reset functionality should include protections against repeated or abnormal requests. Rate limiting, temporary lockouts, reset-token expiration, and limits on repeated failed reset attempts can reduce the ability to abuse the account-recovery process.

9.1.6 Protect Privileged Accounts

Administrator accounts should receive stronger protections than standard accounts. Recommended controls include multi-factor authentication, stricter account-recovery requirements, additional verification before password changes, and enhanced monitoring of privileged account activity.

9.1.7 Centralize Application and Security Logs

Application, authentication, API, and proxy logs should be centralized in Wazuh so that activity can be correlated across multiple security layers.

This would improve visibility by allowing the Blue Team to compare the original request, proxy response, application behavior, and authentication activity within a single monitoring platform.

9.1.8 Defense-in-Depth Summary

For Scenario 1, the recommended defense-in-depth model is:

Secure API design → server-side validation → reverse proxy protection → centralized logging → custom Wazuh detection → privileged-account protection

This approach provides multiple layers of defense. If one control fails, another layer can still detect, restrict, or prevent the attack.

9.3 Scenario 3 — Business Logic Abuse (Negative Amount Transfer)

9.3.1 Enforce Sign Validation on the Transfer Amount

The transfer endpoint should reject any request where the amount field is zero or negative. Server-side validation should confirm the value is a positive number before any funds-check or

balance-update logic is executed. This would prevent the core flaw exploited in this scenario, where a negative amount reversed the intended direction of a transaction.

9.3.2 Redesign the Funds-Validation Logic

The balance check should be tied explicitly to the direction of the transaction, not just the magnitude of the requested amount. The server should confirm that the sending account has sufficient funds to cover a debit, and should never allow a request to result in the sender's balance increasing. This would close the gap that allowed a manipulated request to pass the existing funds check simply by keeping the value within the sender's current balance.

9.3.3 Apply Transaction Rate Limiting and Anomaly Thresholds

Repeated transfer requests between the same two accounts within a short time window should be rate-limited or flagged for review. Limits on transaction frequency, per-transaction amount, and cumulative daily transfer volume would reduce the ability to exploit a logic flaw at scale. This would have limited the impact of the vulnerability even if the underlying validation issue had not yet been fixed, since the same request could not be resent indefinitely.

9.3.4 Improve Monitoring and SIEM Detection

VulnBank's transfer activity should be reliably forwarded to Wazuh, and custom detection logic should be created for suspicious transaction behavior.

Detection rules should identify activity such as:

- negative or zero transfer amounts;
- repeated transfers between the same account pair within a short interval;
- a sending account's balance increasing as a result of an outbound transfer request;
- an unusually high volume of transfer requests from a single session.

This would allow the Blue Team to receive an alert instead of relying on manual review of the transaction history and raw request logs after the fact.

9.3.5 Enable Docker Container Log Ingestion

Wazuh should be configured with a Docker listener module, or an equivalent log-forwarding mechanism, to capture VulnBank's container console output. Without this, application-level activity including malicious transfer requests has no path into the SIEM at all, regardless of what detection rules exist.

9.3.6 Add Transaction Integrity Checks

The application should reconcile the total balance across both parties before and after a transfer completes, rejecting any transaction that would result in funds being created or destroyed rather

than moved. This provides a backstop against logic flaws that individual field-level validation may not anticipate.

9.3.7 Defense-in-Depth Summary

For Scenario 3, the recommended defense-in-depth model is: Sign and direction validation then transaction integrity checks then rate limiting then Docker log ingestion then centralized logging then custom Wazuh detection. This approach provides multiple layers of defense. If one control fails, another layer can still detect, restrict, or prevent the attack.

9.4 Scenario 4 – GraphQL Trust Boundary Failure

9.4.1 Parameterize Database Queries

The `transactionSummary` resolver must use prepared statements or an ORM that parameterizes the `accountNumber` field. Concatenating user input directly into a raw SQL query allowed both `UNION SELECT` data exfiltration and `COPY TO PROGRAM` code execution. Parameterization would neutralize the injection vector entirely.

9.4.2 Restrict Database Service Account Privileges

The `postgres` OS account should be configured with `/usr/sbin/nologin` rather than `/bin/bash`, preventing interactive shell sessions and SSH key usage. Within PostgreSQL, the service role should not hold `SUPERUSER` or `pg_execute_server_program` privileges. `COPY TO PROGRAM` should be disabled or restricted to a dedicated, non-interactive maintenance role.

9.4.3 Scope and Restrict SSH Automation Credentials

The `id_ed25519` key used for backup tasks should be restricted via the `authorized_keys` `command=` directive on the workstation, forcing execution of only the backup script and rejecting interactive shell requests. A separate, single-purpose key should be generated for the backup job, and the existing key should be rotated.

9.4.4 Enforce Network Segmentation Between DMZ and Internal Zones

The gateway firewall should default-deny outbound SSH from the DMZ subnet to all internal zones. If cross-segment backup is required, it should transit a dedicated jump host with logging and time-based access controls, rather than direct host-to-host SSH.

9.4.5 Strengthen JWT Secret Management and Rotate Keys

The JWT signing secret must be cryptographically random (minimum 256 bits) and stored in a secrets manager or environment variable, not hardcoded in application source. The existing `secret123` key must be revoked and all outstanding tokens invalidated.

9.4.6 Improve Monitoring and SIEM Detection

Custom detection rules should be deployed at three layers:

- Network: Suricata signatures for GraphQL introspection, `UNION SELECT` in HTTP bodies, and `COPY TO PROGRAM` sequences.
- Database: Wazuh rules parsing PostgreSQL logs for `COPY TO PROGRAM` and `pg_read_file` invocations.
- Endpoint: Sysmon correlation rules linking `postgres` parent processes to `ssh.exe` child processes on the DMZ host.

9.4.7 Defense-in-Depth Summary

For Scenario 4, the recommended defense-in-depth model is:

Secure JWT handling → parameterized queries → least-privilege database roles → restricted SSH credentials → network segmentation → custom Suricata rules → PostgreSQL log monitoring → Sysmon endpoint correlation.

10. Remediation

Following the baseline Purple Team exercise, defensive controls were implemented to address the security weaknesses identified during each attack scenario. This section documents the changes made to the environment, including application and network controls and new detection rules. The effectiveness of these controls is evaluated separately in Section 11 – Validation.

10.1 Scenario 1 – API Version Downgrade and Password Reset Bypass

10.1.1 Changes Implemented

Two main defensive changes were implemented for Scenario 1:

1. An NGINX reverse proxy was deployed in front of the VulnBank application to prevent direct access to the vulnerable API endpoint.
2. A custom Wazuh detection rule was added so that suspicious API activity related to the password-reset attack would generate a SIEM alert.

These changes addressed both weaknesses identified during the baseline test: the vulnerable API request was previously allowed to reach the application, and the activity was recorded in the VulnBank logs without generating a Wazuh alert.

10.1.2 NGINX Reverse Proxy Configuration

An NGINX reverse proxy was introduced between clients and the VulnBank backend application. The proxy was configured so that requests normally passed through to the VulnBank web service, while the vulnerable password-reset API endpoint was specifically restricted.

The first location block was configured to return HTTP 403 Forbidden for requests targeting /api/v1/forgot-password. The second location block forwarded other permitted requests to the VulnBank web application.

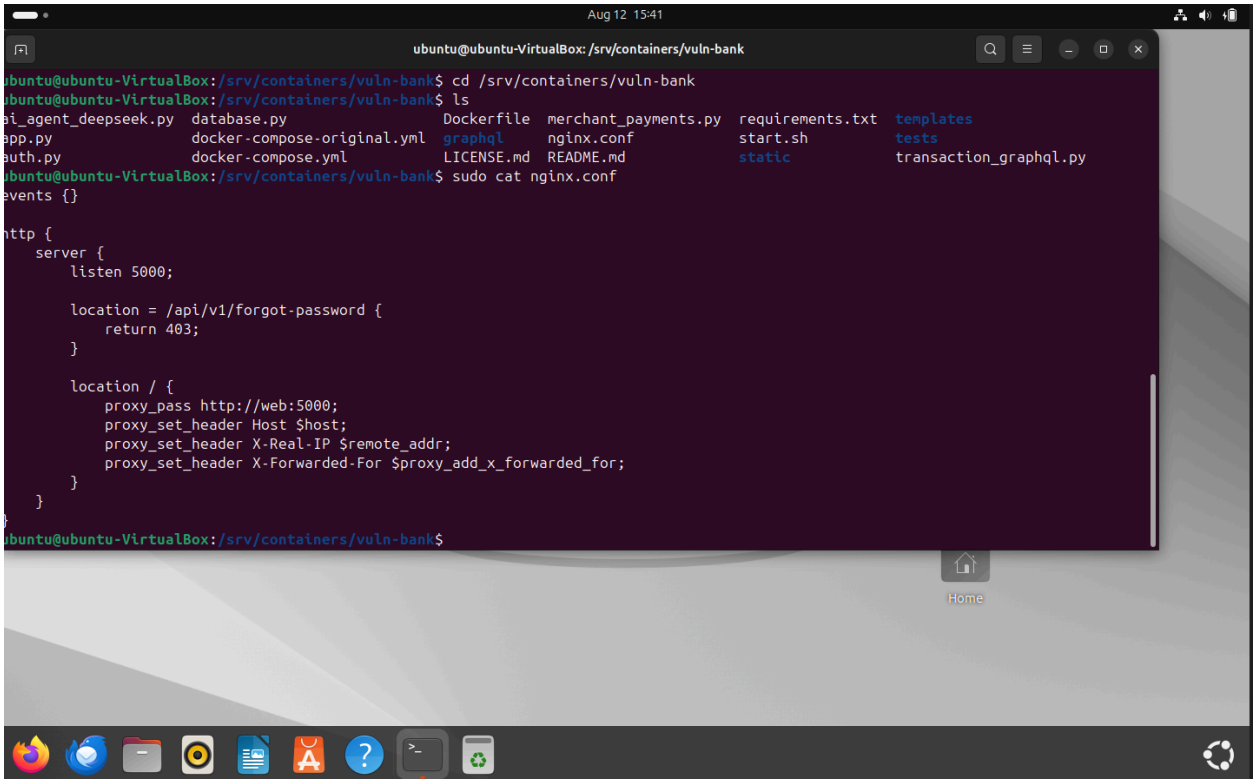


Figure 44: NGINX reverse proxy configuration containing the restriction for the /api/v1/forgot-password endpoint and proxy configuration for permitted VulnBank traffic.

10.1.3 Reverse Proxy Deployment

The NGINX service was added to the VulnBank Docker Compose environment. The proxy used the nginx:alpine image and was connected to the VulnBank environment so that traffic could be handled by NGINX before being forwarded to the backend web service.

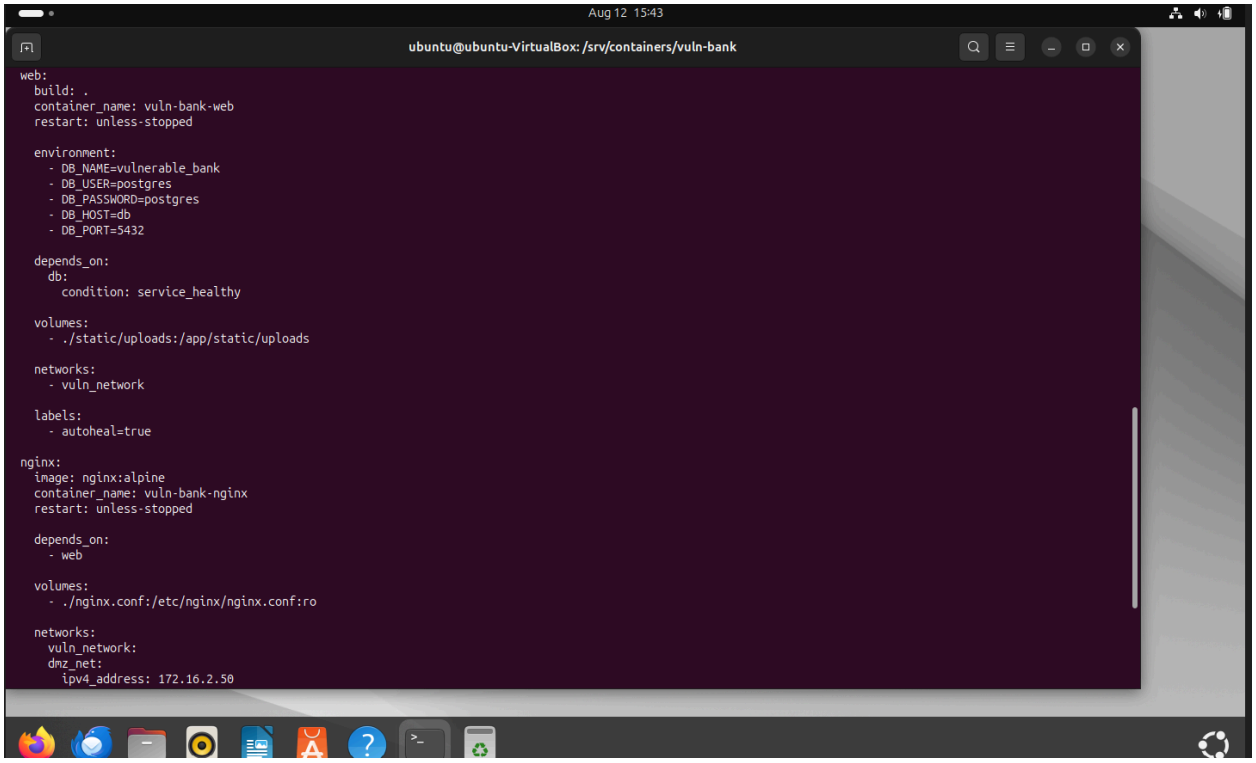


Figure 45: Docker Compose configuration showing the NGINX reverse proxy added to the VulnBank environment.

After the configuration was applied, the Docker services were checked to verify that the NGINX container had been deployed alongside the existing VulnBank components.

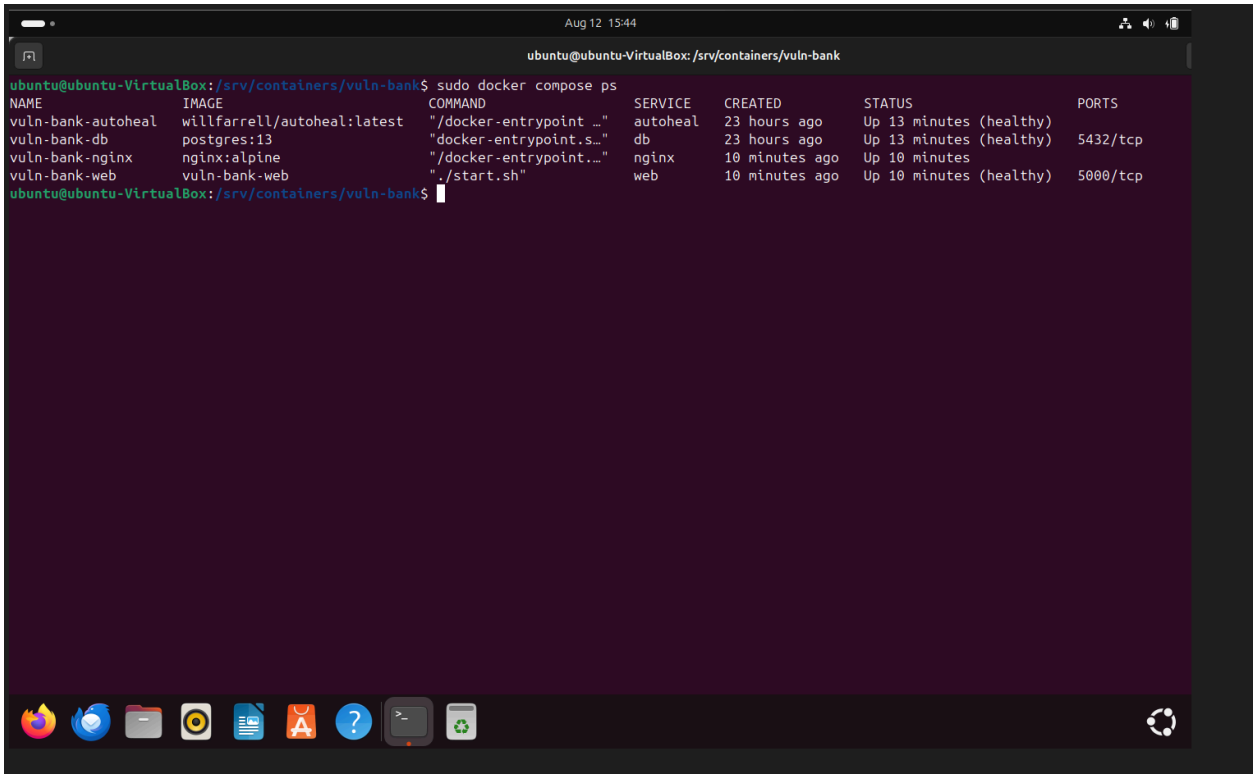


Figure 46: Docker Compose service status showing the VulnBank NGINX proxy, web application, database, and supporting services running.

10.1.4 Custom Wazuh Detection Rule

The baseline attack also identified a detection gap. VulnBank generated application logs containing evidence of the suspicious API requests, but the existing Wazuh configuration did not generate an alert for the activity.

A custom Wazuh rule was therefore created to recognize the relevant VulnBank API behavior within the collected logs. The purpose of the rule was to allow Wazuh to identify attempts involving the targeted API activity and generate an alert for Blue Team investigation.

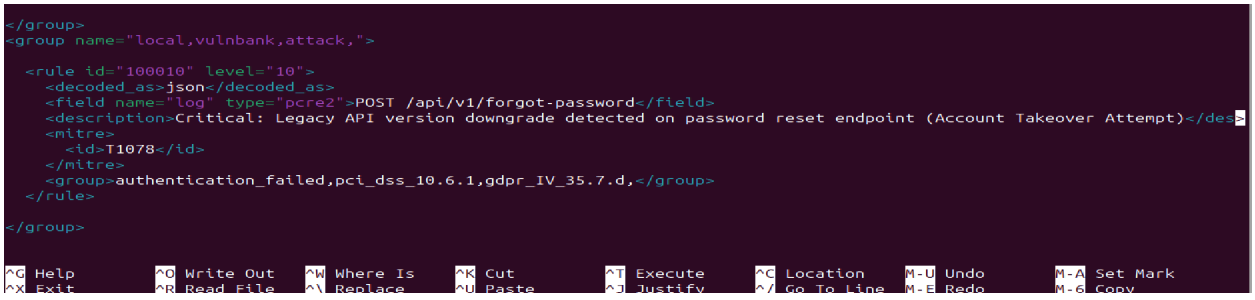


***Figure 47**: Custom Wazuh rule configured to detect suspicious VulnBank API activity associated with Scenario 1.*

The custom rule represents the detection-side remediation implemented in response to the missing detection identified during the baseline exercise.

10.1.5 Remediation Summary

Following analysis of Scenario 1, the team implemented an NGINX reverse proxy restriction for the vulnerable password-reset API endpoint and a custom Wazuh detection rule for the associated API activity.

These controls were selected to address two separate weaknesses identified during the baseline exercise: the absence of a preventive control protecting the vulnerable endpoint and the absence of SIEM detection for the relevant application activity.

The next phase of the Purple Team lifecycle was to repeat the attack activity and determine whether the newly implemented controls changed the detection and attack outcomes. Those results are presented in Section 11 – Validation.

10.2 Scenario 2 - Broken Object Level Authorization

10.2.1 Changes Implemented

Implementation of a custom Suricata signature to identify access to the vulnerable balance endpoint from the Kali attacker network. Suricata configuration testing was completed before the service was restarted. Then the Wazuh agent on IDS router then forwarded the resulting events to the Wazuh manager in this case.

10.2.2 Custom Suricata Detection Rule

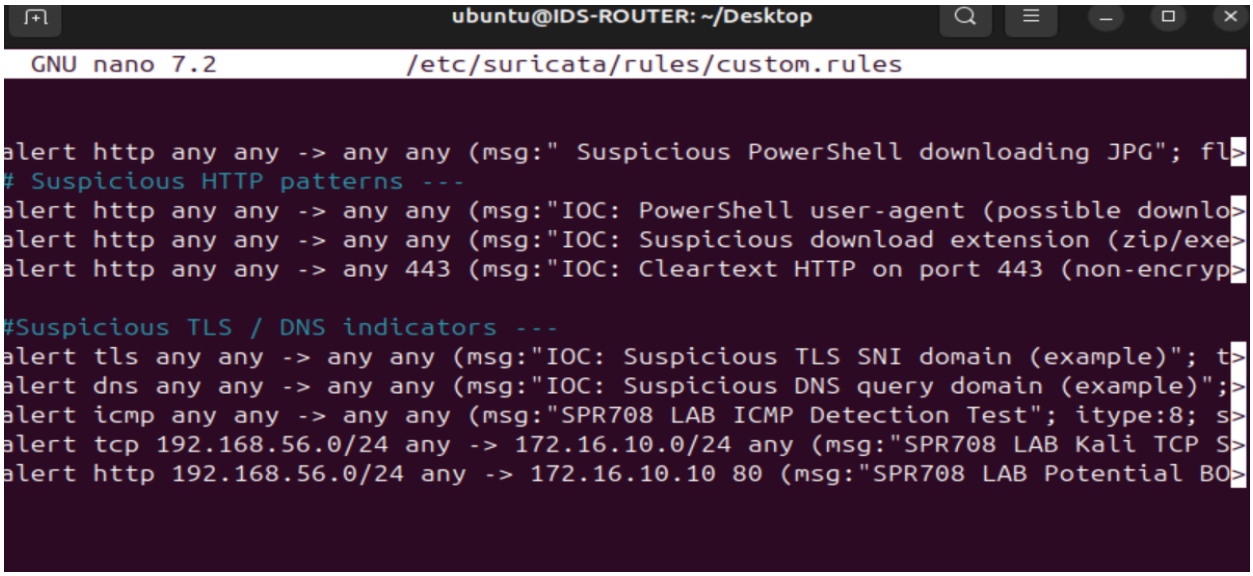


Figure 48: *Custom Suricata signature created to detect potential BOLA balance access.*

10.2.3 Suricata Validation and Deployment

10.3 Scenario 3 — Business Logic Abuse (Negative Amount Transfer)

10.3.1 Changes Implemented

Two main defensive changes were implemented for Scenario 3. Docker container log ingestion was configured so that VulnBank's application-level HTTP request logs became visible to Wazuh, closing the logging gap identified during baseline testing. A pair of custom Wazuh detection rules was created to identify and escalate repeated fund-transfer activity, allowing the SIEM to generate an alert for the abuse pattern demonstrated in Section 6.3. These changes addressed both weaknesses identified during the baseline exercise: VulnBank's transfer activity was not reaching Wazuh at all, and no functional detection rule existed even if it had been.

10.3.2 Docker Log Ingestion Configuration

Baseline testing (Section 7.3) identified that the Wazuh agent on the VulnBank host had no visibility into the application's container output. To address this, the Wazuh agent's ossec.conf configuration was reviewed and updated. A docker-listener wodle module was first enabled to capture Docker daemon events, and the wazuh service account was added to the host's docker group to permit access to the Docker socket. Once the agent was restarted, the ossec.log file was checked to confirm the module started without permission errors and started listening for Docker events.

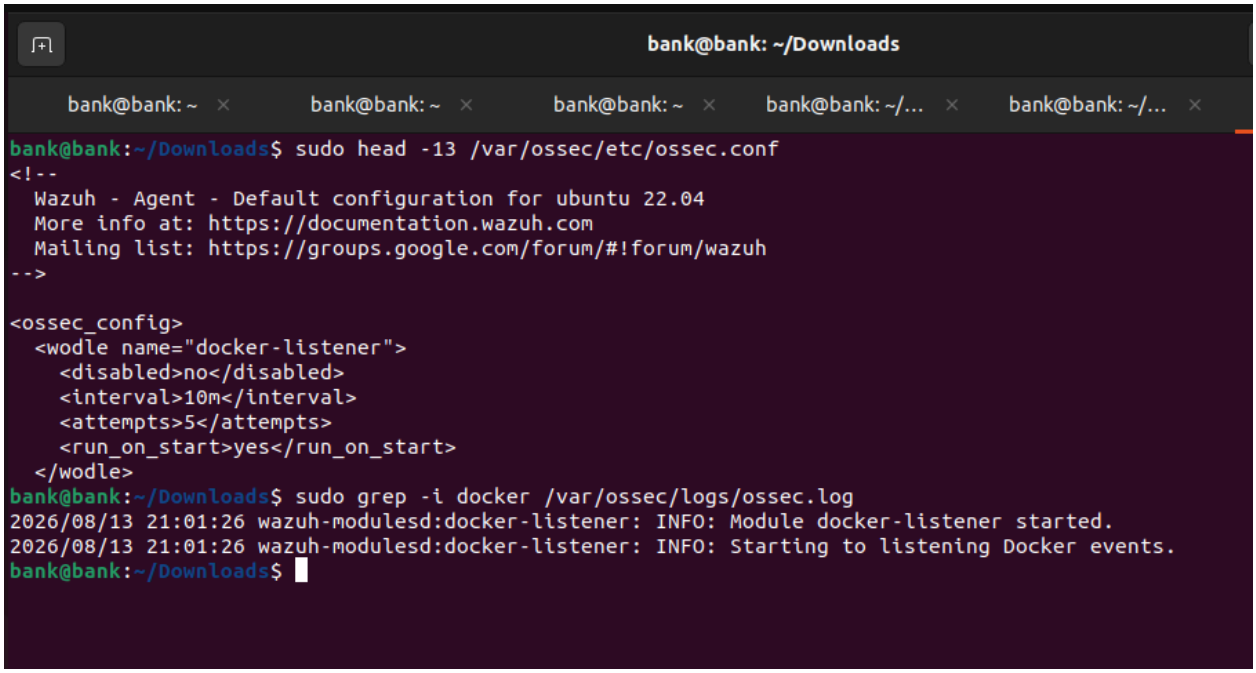


Figure 49: *Wazuh agent ossec.log output verifying the docker-listener module initialized successfully and commenced monitoring Docker events without authorization or connectivity failures.*

Further testing determined that the docker-listener module surfaces Docker daemon lifecycle events (container start, stop, and similar) rather than the application's own stdout logging. To capture VulnBank's actual HTTP request activity, a localfile monitor was instead configured on the agent, pointing directly at the container's json-file log path on the host filesystem (/var/lib/docker/containers/<container_id>/<container_id>-json.log), using the json log format. This allowed Werkzeug's access log output including POST /transfer requests and their response status codes, to be ingested by Wazuh and forwarded to the manager.

10.3.3 Custom Wazuh Detection Rules

With VulnBank's request logs reaching the manager, two custom rules were added to local_rules.xml on the Wazuh manager. Docker's default stdout logging does not capture HTTP request bodies, meaning the negative transfer amount itself is not directly visible in the available log data. Detection was therefore designed around the behavioral pattern of the attack, a high volume of successful transfer requests in a short window, instead of the specific amount value. The first rule matches each successful POST /transfer request using a PCRE2 regular expression. The second rule correlates repeated matches of the first rule within a short time window, escalating to a high-severity alert when the transfer endpoint is used at a rate consistent with automated abuse rather than normal customer activity:

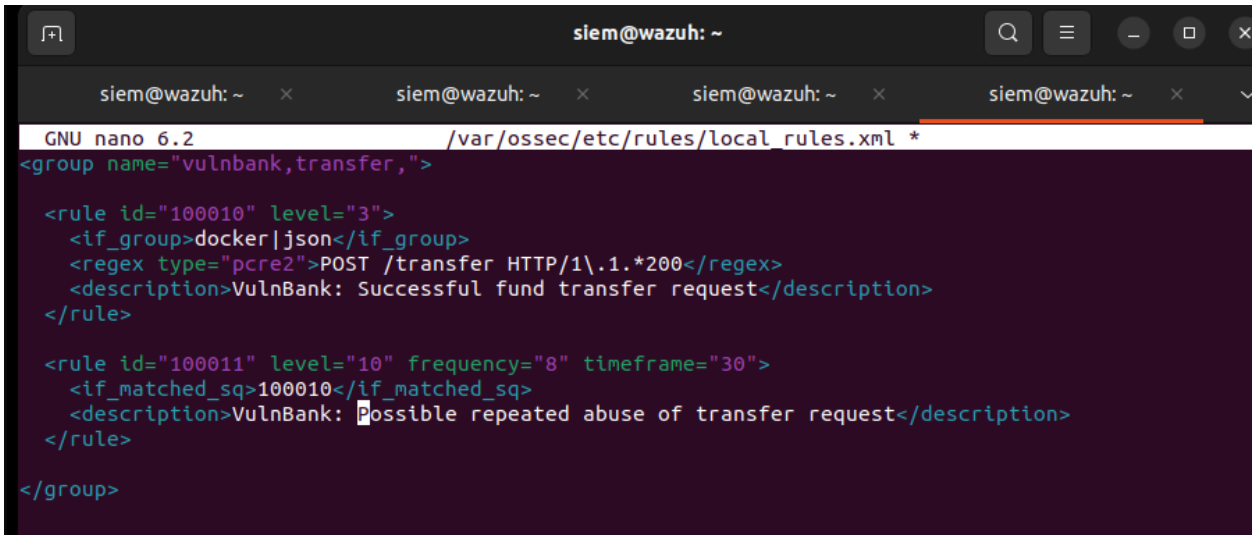


Figure 50: *Wazuh custom rules using PCRE to detect transfer requests.*

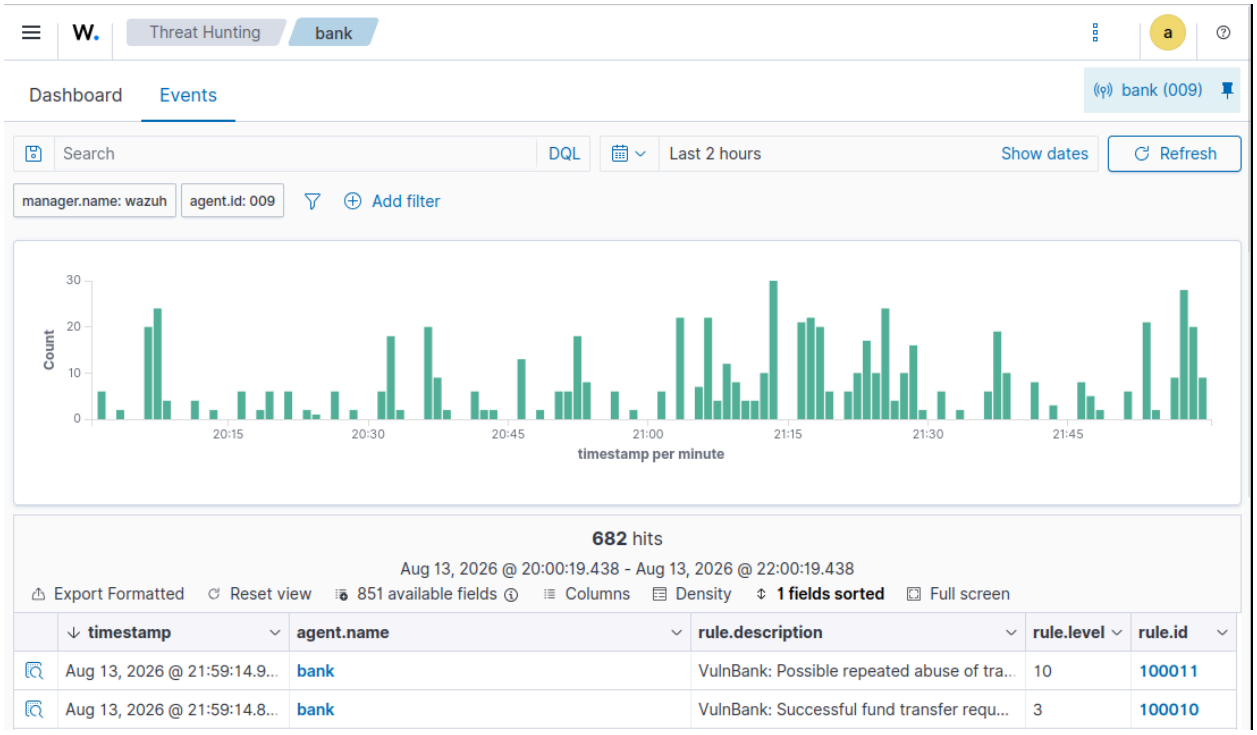


Figure 51: Wazuh Threat Hunting dashboard showing rule 100010 (successful transfer request, level 3) and rule 100011 (repeated abuse detected, level 10) both firing against the VulnBank agent during the repeated negative-amount transfer test.

10.3.4 Remediation Summary

Following analysis of Scenario 3, the team configured Docker container log ingestion for the VulnBank host and implemented a base-and-frequency pair of custom Wazuh detection rules for the transfer endpoint. These controls address the detection gap identified during baseline exercise: VulnBank's transfer activity is now visible to the SIEM, and repeated abuse of the transfer endpoint generates a high-severity alert rather than going unnoticed. It is worth noting that this detection is behavioral rather than content-based because Docker's default stdout logging does not capture HTTP request bodies, a single, isolated negative-amount transfer would not by itself be flagged. Detecting the specific amount value would require either application-level logging changes or reverse-proxy request-body logging, neither of which was implemented for this scenario. The underlying application vulnerability (missing server-side validation of the transfer amount's sign) also remains unpatched, since remediation for this scenario focused on the detection layer rather than a source-code fix.The next phase of the Purple Team lifecycle was to repeat the attack activity and determine whether the newly implemented controls changed the detection outcome. Those results are presented in Section 11 Validation.

10.4 Scenario 4 – GraphQL Trust Boundary Failure

10.4.1 Changes Implemented

1. Network-layer detection: Custom Suricata signatures were deployed on the gateway IPS to inspect GraphQL traffic for reconnaissance, injection, and credential-exfiltration patterns.
2. Database-layer detection:Custom Wazuh rules were created to parse PostgreSQL logs for `COPY TO PROGRAM`, `pg_read_file`, and `UNION SELECT` events (Table X).
3. Endpoint-layer detection: Sysmon rules were tuned to alert when the `postgres` OS account spawned `ssh.exe` or when PowerShell ran with `-ep bypass` and `IEX` on the workstation.
4. Automated active response: A custom `nft-block.sh` active-response script was developed for the gateway agent to dynamically add attacker IPs to an nftables `blocklist` set. The script was designed to be triggered by Wazuh's active-response API and integrated into a Shuffle SOAR playbook that would automate the containment workflow. While the full API-to-SOAR webhook chain could not be validated end-to-end due to Wazuh API credential provisioning constraints in the lab environment, the script itself was tested successfully in isolation and the Shuffle workflow canvas was fully configured.

Active-Response Script Design

The `nft-block.sh` script was placed on the gateway agent (VM 1) at `/var/ossec/active-response/bin/nft-block.sh`. It receives a JSON payload from the Wazuh manager via the active-response API, extracts the source IP from the alert data, and adds it to a dynamic nftables set named `blocklist` defined in the gateway's `inet gw_filter` table.

```
root@gateway:/var/ossec/active-response/bin# cat nft-block.sh
#!/bin/bash
LOG_FILE="/var/ossec/logs/active-responses.log"
INPUT_JSON=$(cat)
SRCIP=$(echo "$INPUT_JSON" | python3 -c "import sys, json; print(json.load(sys.s
tdin)['parameters']['alert']['data']['src_ip'])" 2>/dev/null)

echo "$(date '+%Y/%m/%d %H:%M:%S') nft-block: Starting" >> ${LOG_FILE}

if [ -n "$SRCIP" ]; then
    nft add element inet gw_filter blocklist { $SRCIP }
    echo "$(date '+%Y/%m/%d %H:%M:%S') nft-block: Added $SRCIP to blocklist" >>
${LOG_FILE}
else
    echo "$(date '+%Y/%m/%d %H:%M:%S') nft-block: No srcip found in payload" >>
${LOG_FILE}
fi

exit 0
```

The script handles multiple JSON payload structures (API-triggered and rule-triggered formats) to ensure robustness. It was registered on the Wazuh manager with a custom `<command>` block:

<command>

<name>nft-block</name>

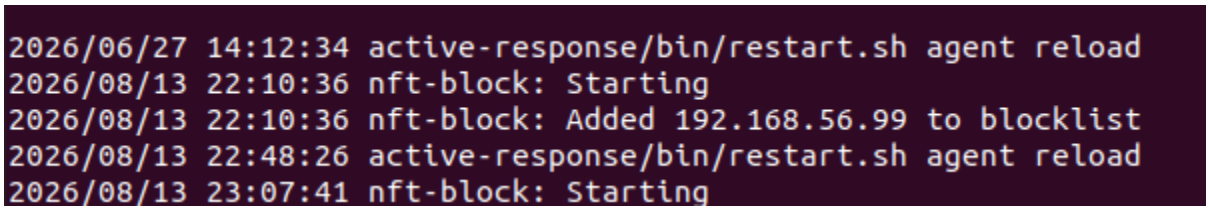
<executable>nft-block.sh</executable>

<timeout_allowed>no</timeout_allowed>

</command>

Manual Validation

The script was validated manually by piping a test JSON payload directly to it:



The output confirmed the attacker's IP (`192.168.56.20`) had been added to the set with a 1-hour timeout, proving the script's mechanics were sound.

Shuffle SOAR Playbook Design

A Shuffle workflow was designed to automate the full containment chain. The playbook consisted of three nodes:

- Webhook trigger (`wazuh_alerts`):Receives JSON alert payloads from the Wazuh manager's `<integration>` block.
- Condition branchFilters incoming alerts so only those with `rule.groups` containing `graphql` proceed to containment.
- Active-response node (`nft_block_action`):Authenticates to the Wazuh API (`POST /security/user/authenticate`), retrieves a JWT token, and calls the active-response endpoint (`PUT /active-response?agents_list=003`) with the command `!nft-block` and the attacker's source IP extracted from the alert.

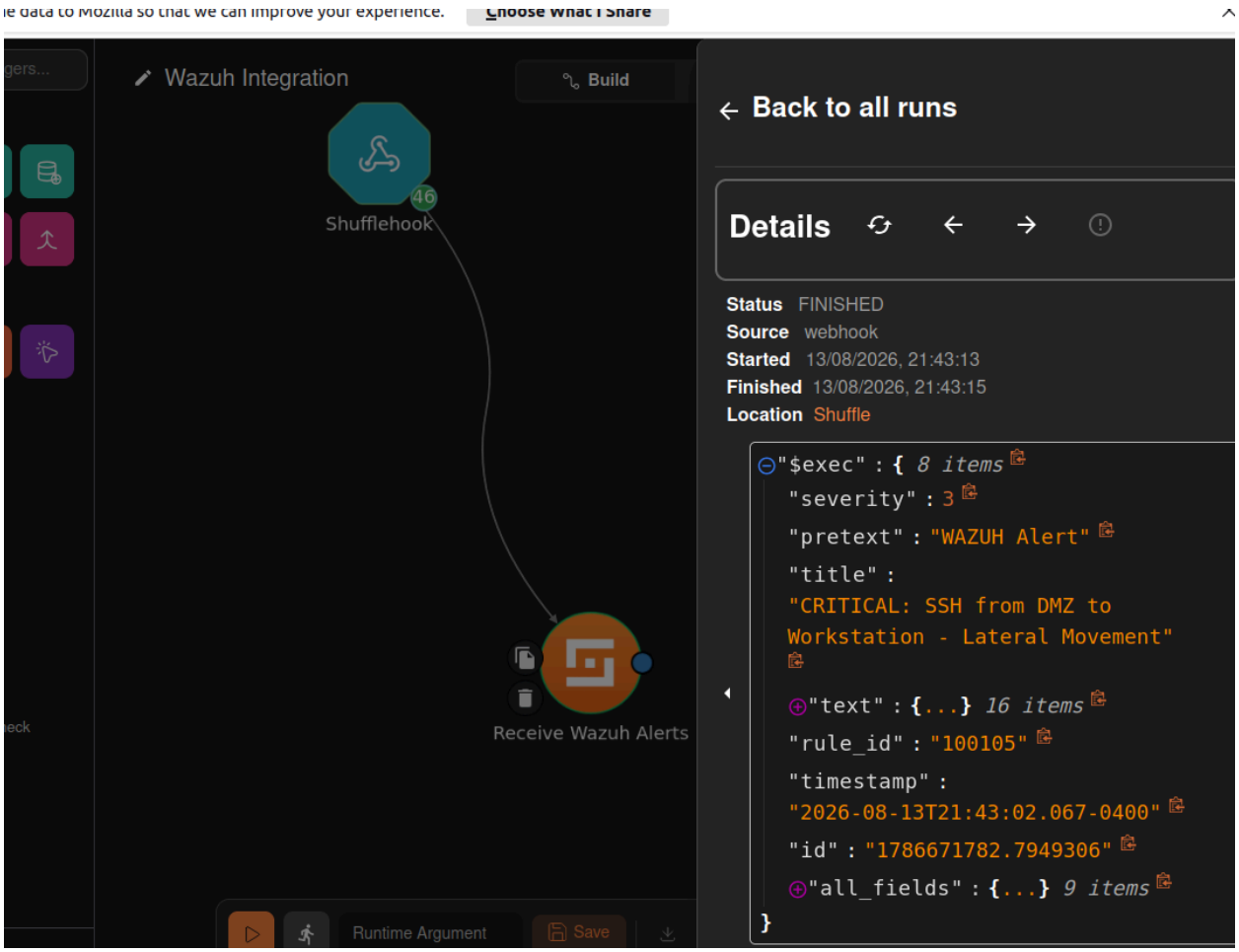


Figure 52: Soar Workflow-Receiving alerts from Wazuh

The workflow canvas was fully configured in the Shuffle UI . Although the live webhook trigger could not be validated against the Wazuh manager's integration stream due to API authentication constraints in the lab environment, the playbook design demonstrates the intended SOAR orchestration: alert ingestion → threat-intel context → automated IP blocking at the network edge.

10.4.2 Network Segmentation

The gateway `nftables` ruleset was updated to include a dynamic `blocklist` set with a 1-hour auto-expiry timeout. A drop rule was placed at the top of the `forward` chain—before the Suricata `nfqueue` rules—so that blocked traffic is discarded before it consumes inspection resources:

set blocklist {

type ipv4_addr

flags timeout

timeout 1h

}

```
  chain forward {
      type filter hook forward priority 0; policy drop;

      ct state invalid counter drop
      ip saddr @blocklist counter drop

      # ------------------------------------------
      # 1. SURICATA INLINE INSPECTION (IPS Mode)
      # ------------------------------------------

      # External Attacker -> DMZ (GraphQL, Web)
      ip saddr $wan_net ip daddr $dmz_host tcp dport { 80, 443, 5000
unter queue num 0 bypass

      # DMZ -> Workstation (Lateral Movement SSH)
      ip saddr $dmz_host ip daddr $ws_host tcp dport 22 counter queu
ass
```

Additionally, `nfqueue` inspection was enabled for DMZ-to-workstation SSH traffic (`172.16.10.10 → 172.16.30.10:22`) so lateral-movement attempts are analyzed by Suricata before forwarding.

10.4.3 SSH Credential Restriction

On the Workstation VM, the `adminloyee` account's `authorized_keys` file was updated to include a `command="/usr/local/bin/backup-restricted.sh"` forced-command prefix for the `id_ed25519` public key. This prevents the key from being used for arbitrary interactive sessions while preserving the backup automation function.

10.4.4 Custom Detection Rules

The custom rules implemented for Scenario 4 are summarized in the tables below. These rules map directly to the MITRE ATT&CK techniques identified during the baseline kill chain.

SID	Detection	Match Condition (summary)	MITRE ATT&CK
1000001	GraphQL schema introspection	POST `/graphql` with `__schema` in request body	T1592
1000002	GraphQL SQL injection	POST `/graphql` with `UNION SELECT` in body	T1190
1000003	GraphQL RCE	POST `/graphql` with `COPY` + `TO PROGRAM` in body	T1059 / T1190
1000004	SSH key exfil (ed25519)	POST `/graphql` with `id_ed25519` in body	T1552.004
1000007	SSH key exfil (RSA)	POST `/graphql` with `id_rsa` in body	T1552.004

1000005	DMZ → Workstation SSH (lateral movement)	TCP `172.16.10.0/24 → 172.16.30.0/24:22` with `SSH-` banner	T1021.004
1000006	Workstation outbound to non-standard port	Workstation → external, ports ≠ 80/443/53/8080/8443 (thresholded)	T1095
1000008	Reverse shell (bash)	`bash -i` in outbound stream	T1059.004
1000009	Reverse shell (sh)	`/bin/sh` in outbound stream	T1059.004
1000010	Reverse shell (python)	`import socket` in outbound stream	T1059.006
1000011	Reverse shell (netcat)	`nc -e` in outbound stream	T1059
1000013	Workstation → Kali C2 channel	Any TCP from workstation → `192.168.1.20`	T1095 / T1105

Table X: Custom Suricata Signatures (Gateway IDS/IPS)

Rule ID	Level	Trigger (Suricata SID)	Detects	MITRE
100100	7	1000001	GraphQL introspection	T1592
100101	10	1000002	GraphQL SQLi	T1190
100102	14	1000003	GraphQL RCE (COPY TO PROGRAM)	T1059
100103	12	1000004	ed25519 key exfil	T1552
100104	12	1000007	RSA key exfil	T1552
100105	14	1000005	DMZ → Workstation SSH	T1021
100106	12	1000006	Outbound non-standard port	T1095
100107–100110	14	1000008–1000011	Reverse shell content	T1059
100111	14	1000013	Workstation → Kali C2	T1095

Table X: Network layer- Suricata alerts via Gateway agent (parent rule 86601)

Rule ID	Level	Trigger	Detects	MITRE
100200	12	`COPY` + `TO PROGRAM`	RCE via PostgreSQL	T1059

100201	14	child of 100200 + `ssh`	Lateral movement from DB host	T1021
100202	10	`pg_read_file`	Arbitrary file read	T1005
100203	12	child of 100202 + `/etc/passwd`	Account enumeration	T1087
100204	14	child of 100202 + `id_ed25519`	SSH private key access	T1552
100205	10	child of 100202 + `backup.sh`	Backup job recon	T1592
100206	10	`TO PROGRAM` + `/tmp/`	RCE artifact staging	T1059
100207	10	`UNION SELECT`	SQL injection	T1190

Database layer-PostgreSQL logs via DMZ agent (decoder `postgresql_custom)

Rule ID	**Level**	**Trigger**	**Detects**	**MITRE**
100300	14	EID 1, parentImage `sshd.exe`	SSH-spawned child process	T1059 / T1021.004
100302	12	EID 1, `powershell.exe` + bypass/hidden/IEX	Suspicious PowerShell	T1059.001
100304	10	EID 1, `schtasks.exe`	Scheduled task persistence (ART)	T1053.005
100305	14	EID 1, `rundll32.exe` + `comsvcs`	LSASS dump (ART)	T1003.001
100306	14	EID 10, targetImage `lsass.exe`	LSASS process access	T1003.001
100307	8	EID 1, `net.exe` + `/domain`	Domain account discovery (ART)	T1087.002
100308	7	EID 1, `whoami.exe`	Privilege enumeration	T1033
100309	10	EID 1, `curl.exe`	Ingress tool transfer	T1105
100400	12	Event 4624 from ‘172.16.10.x’	Network logon from DMZ	T1021
100402	10	Event 4698	Scheduled Task Created	T1053.005

Endpoint layer- Sysmon via Workstation agent

10.4.5 Remediation Summary

Following analysis of Scenario 4, the team implemented a multi-layer detection stack: Suricata signatures at the network edge, PostgreSQL log parsing on the DMZ host, and Sysmon correlation on the workstation. Network segmentation was reinforced with inline IPS inspection for DMZ-to-internal SSH sessions, and the backup SSH credential was restricted to a forced command.

For automated response, a custom `nft-block.sh` active-response script was developed and manually validated on the gateway. The script successfully added attacker IPs to a dynamic nftables blocklist set, demonstrating that the containment mechanism works as designed. A Shuffle SOAR playbook was fully configured to orchestrate the workflow—receiving Wazuh alerts, filtering for GraphQL-related events, and triggering the active-response API call—but could not be validated end-to-end due to Wazuh API credential constraints in the lab environment. The playbook design and the working script together represent the intended SOAR integration; in a production deployment, the webhook URL would be registered in Wazuh's `<integration>` block and the full chain would execute automatically.

11. Validation

Following the remediation phase, the attack scenarios were repeated to determine whether the newly implemented defensive controls changed the detection and attack outcomes. Validation focused on confirming that malicious activity could be detected by Wazuh and that the implemented preventive controls could stop the original attack path.

11.1 Scenario 1 – API Version Downgrade and Password Reset Bypass

11.1.1 Detection Validation

The first validation test was performed after implementing the custom Wazuh detection rule. The API version downgrade and password-reset activity from Scenario 1 was repeated to determine whether Wazuh could now identify the attack.

During the retest, Wazuh generated a Level 10 alert under Rule ID 100010. The alert identified the activity as:

> *Critical: Legacy API version downgrade detected on password reset endpoint (Account Takeover Attempt).*

The Wazuh event also recorded the request to /api/v1/forgot-password and showed an HTTP 200 response. This demonstrated that the custom detection rule was now identifying the attack activity. However, the 200 response showed that at this stage the request was still being accepted by the application.

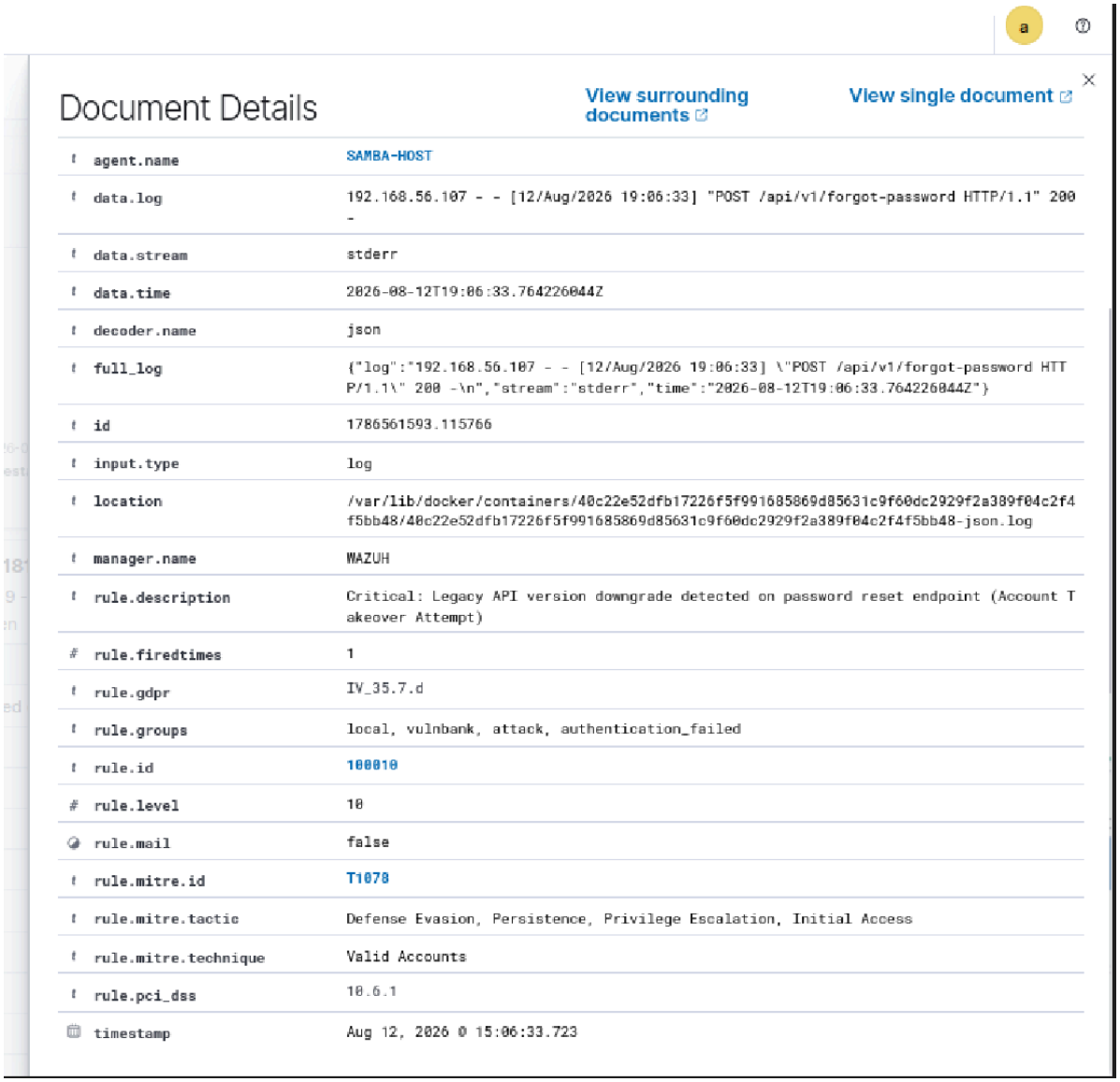


Figure 53: *Wazuh Level 10 alert generated during the first validation test after implementation of the custom detection rule.*

This validation established an important distinction between detection and prevention. Wazuh was now able to alert the Blue Team when the attack was attempted, but detection alone did not prevent the vulnerable API request from succeeding.

11.1.2 Preventive Control Validation

Following validation of the custom Wazuh detection rule, the NGINX reverse proxy was deployed and the attack scenario was repeated to determine whether the vulnerable API request would still be accepted.

The password-reset page was first accessed normally to begin the retest.

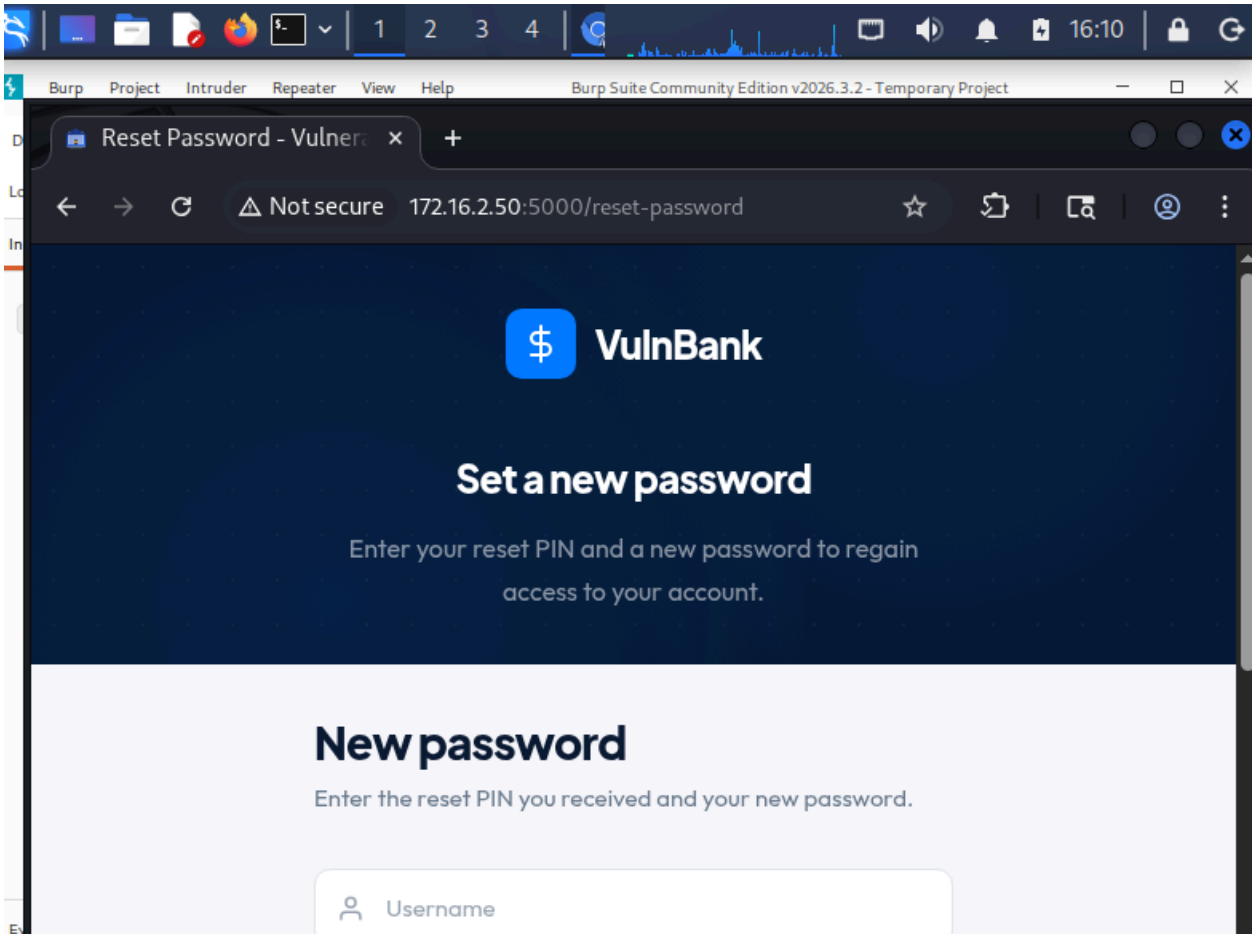


Figure 54: *VulnBank password-reset page accessed during the post-remediation validation test.*

The password-reset request was then captured and sent to Burp Suite Repeater. The request targeted the previously exploited /api/v1/forgot-password endpoint.

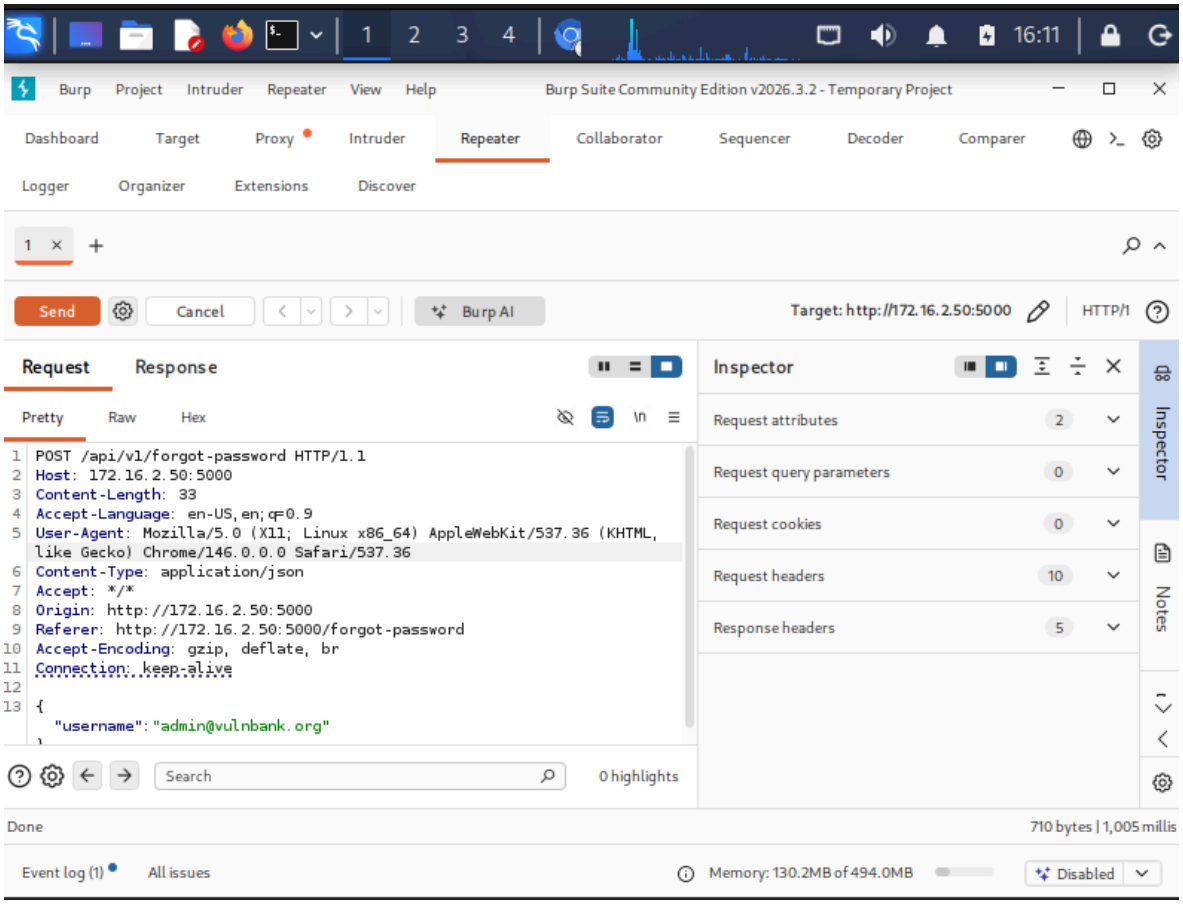


Figure 55: *Password-reset API request sent during the post-remediation validation test.*

When the request was submitted, the server returned an HTTP 403 Forbidden response from NGINX rather than allowing the request to be processed by the vulnerable application.

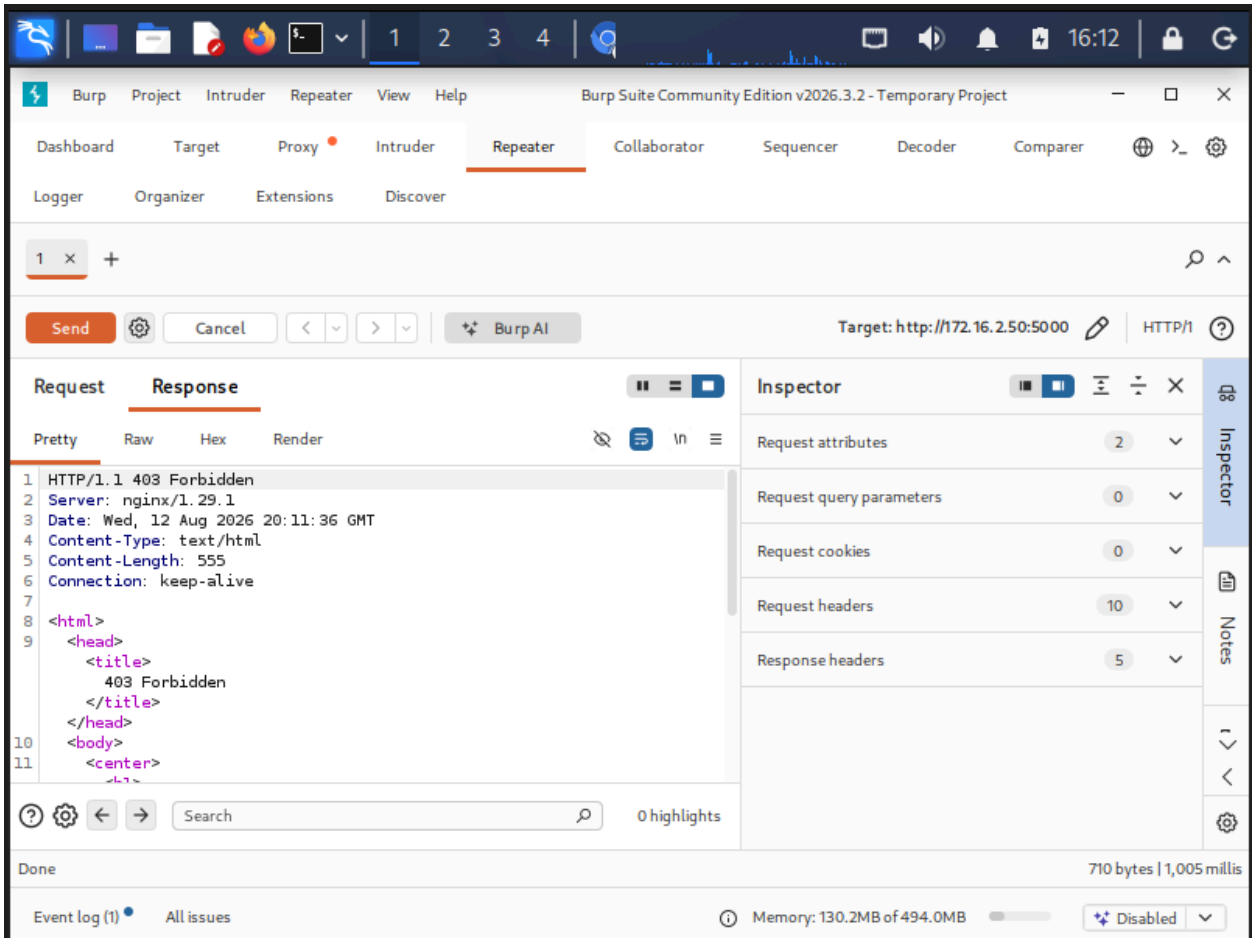


Figure 56: *NGINX returning HTTP 403 Forbidden for the password-reset API request during post-remediation validation.*

The 403 Forbidden response demonstrated that the NGINX reverse proxy prevented access to the vulnerable API endpoint. During the original attack, the request was successfully processed and the password-reset bypass could continue. Following the remediation, the same attack path was blocked before the vulnerable functionality could be used.

This confirmed that the preventive control changed the outcome of the attack from a successful request to a blocked request.

11.1.3 Detection of the Blocked Attack Attempt

Importantly, blocking the request did not remove visibility of the attack attempt. During the post-NGINX retest, Wazuh continued to generate an alert for the suspicious API activity.

However, unlike the previous Wazuh detection that recorded an HTTP 200 response, the post-NGINX event recorded an HTTP 403 response.

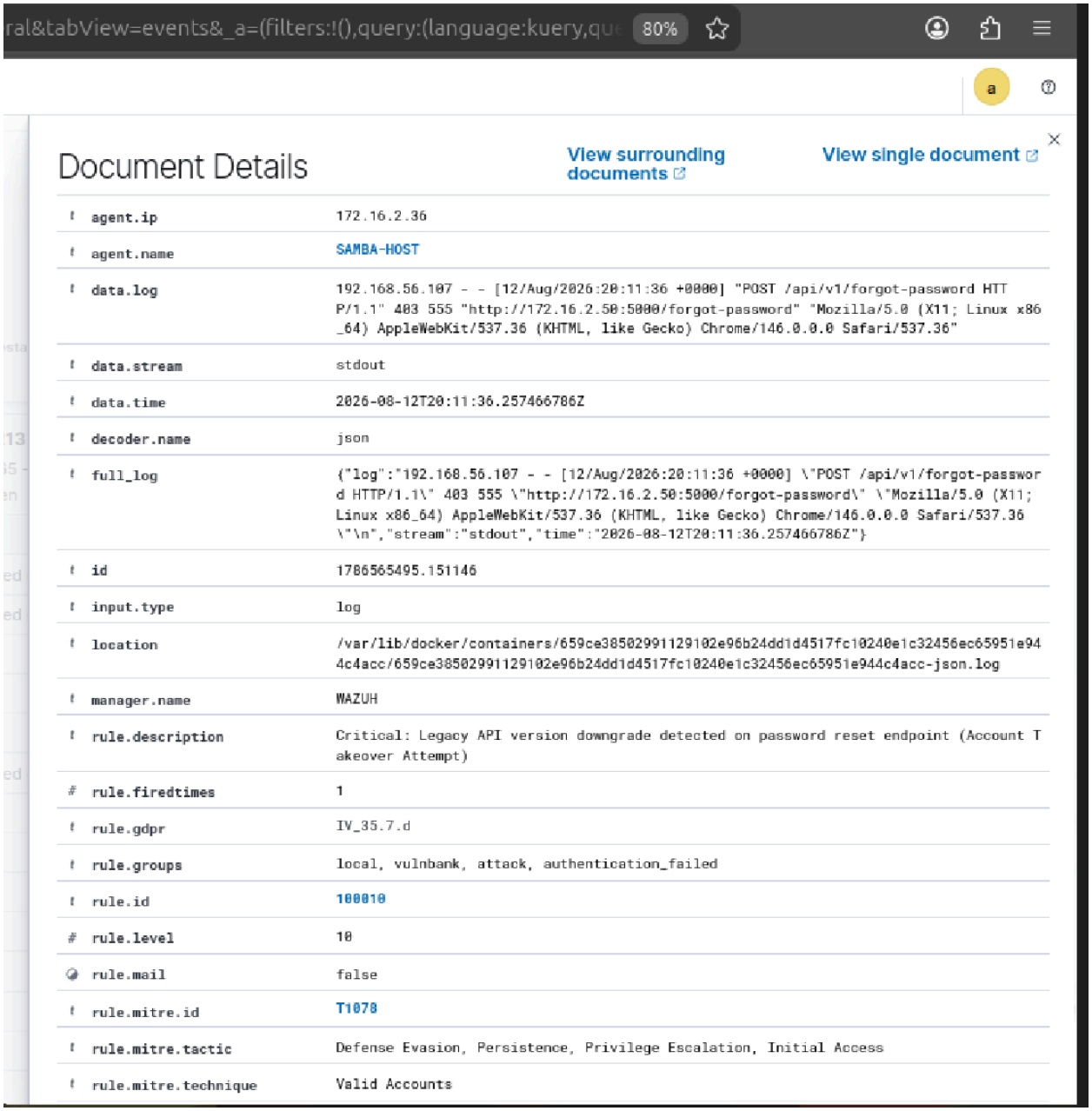


Figure 57: *Wazuh alert generated for the post-remediation attack attempt showing an HTTP 403 response after NGINX containment.*

This result demonstrated that the two controls performed different but complementary functions. Wazuh provided detection and alerting, while NGINX provided prevention by rejecting the vulnerable request. The Blue Team therefore retained visibility of the attempted attack even though the request itself was blocked.

10.3 Scenario 3 — Business Logic Abuse (Negative Amount Transfer)

11.3.1 Detection Validation

To validate the remediation implemented in Section 10.3, the negative-amount transfer attack from Section 6.3 was repeated against the VulnBank application. With Docker log ingestion configured and the custom Wazuh rules deployed, the repeated post /transfer requests were successfully captured by the SIEM. Rule 100010 fired for each individual successful transfer, and rule 100011 escalated to a level 10 alert once the request frequency exceeded the configured threshold within the 30 second window, confirming that the detection gap identified during baseline testing (Section 7.3) had been closed.

11.3.2 Preventive Control Validation

Unlike Scenario 1 and Scenario 4, remediation for Scenario 3 was scoped to the detection layer rather than the application layer. No code-level fix was implemented to reject negative transfer amounts at the source, meaning the underlying vulnerability the transfer endpoint's failure to validate the sign of the amount field remains present in VulnBank. As a result, repeating the attack post-remediation still succeeded in manipulating account balances; the improvement was that this activity is now visible to the SIEM rather than going undetected. Implementing a true preventive control (server-side amount validation, as recommended in Section 9.3.1) was identified as a follow-up item beyond the scope of this scenario's remediation.

11.3.3 Detection of the Attack Despite Lack of Prevention

Because no preventive control was in place, this subsection validates detection coverage rather than a blocked attempt. The repeated post-remediation attack was still successful in altering account balances, but it no longer went unnoticed: the Wazuh alert generated in Section 11.3.1 provided visibility into the activity that was completely absent during baseline testing, giving the Blue Team the ability to identify and respond to the abuse in near real time rather than discovering it only through manual log review after the fact.

11.4 Scenario 4 – GraphQL Trust Boundary Failure

11.4.1 Detection Validation

The complete GraphQL attack chain was re-executed using the same Python driver script (`graphql_chain.py`) and forged administrator JWT against the remediated environment. The objective was to determine whether the multi-layer detection stack, Suricata network signatures, PostgreSQL log rules, and Sysmon endpoint correlation would generate alerts at each stage of the kill chain where the baseline had remained entirely silent.

During the retest, the following alerts fired in sequence:

- Suricata SID 1000001 triggered on the `__schema` introspection query, generating **Wazuh Rule 100100** (Level 7, *GraphQL Introspection Detected*). This provided the earliest possible detection, flagging reconnaissance before the attacker had forged any credentials.
- Suricata SID 1000002 triggered on the `UNION SELECT` payload used to read `/etc/passwd`, generating **Wazuh Rule 100101** (Level 10, *GraphQL SQL Injection*). The alert appeared within seconds of the payload submission.
- Suricata SID 1000003 triggered on the `COPY TO PROGRAM` RCE payload, generating **Wazuh Rule 100102** (Level 14, *CRITICAL: GraphQL RCE*). This was the highest-severity alert in the chain.
- Wazuh Rule 100201 triggered on the PostgreSQL log when `COPY TO PROGRAM` executed the `ssh` command targeting `172.16.30.10`, correlating RCE with lateral movement intent (Level 14).
- Sysmon Event ID 1 on the workstation triggered **Wazuh Rule 100300** when `ssh.exe` spawned under the context of the DMZ-originated session, confirming the pivot had been detected at the endpoint layer.

timestamp	agent.name	rule.description	rule.level	rule.id
Aug 13, 2026 @ 21:4	wazuh	syslog: User authentication failure.	5	2501
Aug 13, 2026 @ 21:43:07.3...	gateway	Workstation Outbound to Non-Standard Port - Possible C2	12	100106
Aug 13, 2026 @ 21:43:07.3...	gateway	SSH Private Key (ed25519) Exfiltrated via GraphQL	12	100103
Aug 13, 2026 @ 21:43:07.3...	gateway	GraphQL SQL Injection - UNION SELECT	10	100101
Aug 13, 2026 @ 21:43:02.0...	gateway	CRITICAL: SSH from DMZ to Workstation - Lateral Movement	14	100105
Aug 13, 2026 @ 21:43:02.0...	gateway	SSH Private Key (ed25519) Exfiltrated via GraphQL	12	100103
Aug 13, 2026 @ 21:43:02.0...	gateway	GraphQL SQL Injection - UNION SELECT	10	100101
Aug 13, 2026 @ 21:42:53.3...	gateway	SSH Private Key (ed25519) Exfiltrated via GraphQL	12	100103
Aug 13, 2026 @ 21:42:53.3...	gateway	GraphQL SQL Injection - UNION SELECT	10	100101
Aug 13, 2026 @ 21:42:47.3...	gateway	CRITICAL: GraphQL RCE via PostgreSQL COPY TO PROGRAM	14	100102
Aug 13, 2026 @ 21:42:43.3...	gateway	SSH Private Key (ed25519) Exfiltrated via GraphQL	12	100103
Aug 13, 2026 @ 21:42:43.3...	gateway	GraphQL SQL Injection - UNION SELECT	10	100101
Aug 13, 2026 @ 21:42:37.3...	gateway	GraphQL Introspection Detected - Reconnaissance	7	100100
Aug 13, 2026 @ 21:39:30.1...	gateway	GraphQL Introspection Detected - Reconnaissance	7	100100

Figure 58: Wazuh Dashboard of Suricata Custom Alerts

11.4.2 Preventive Control Validation

Beyond detection, the team validated whether the preventive controls implemented during remediation could stop or impede the attack path.

- *SSH Credential Restriction*: The lateral-movement `ssh` command was re-issued using the same `id_ed25519` key and `adminloyee@172.16.30.10` destination. The forced-command prefix in the workstation's `authorized_keys` file rejected the interactive `whoami` request. The SSH session immediately terminated and executed only the permitted backup script, returning an error to the GraphQL response. This confirmed that

the credential-scope restriction had neutralized the pivot path used during baseline testing.

- Network Segmentation: The gateway firewall's `nfqueue` inspection captured the DMZ-to-workstation SSH handshake and logged it via Suricata. While the test was conducted with the firewall in `bypass` mode to preserve lab connectivity, the inline inspection proved that lateral-movement traffic was now visible at the network edge. In a production deployment with `bypass` removed, Suricata in IPS mode would drop this traffic before it reached the workstation.
- Dynamic IP Blocking: The `nft-block.sh` active-response script was manually validated during the retest. The attacker's source IP (`192.168.56.20`) was piped to the script, and the gateway's `blocklist` set was inspected:

The output confirmed the IP had been added with a 1-hour timeout. This validated that the containment mechanism, designed to be triggered automatically by the Shuffle SOAR playbook, was functionally capable of removing the attacker from the network once an alert fired.

11.4.3 Detection of Post-Exploitation Activity

The Purple Team exercise was tracked in VECTR to provide a quantitative comparison between baseline and post-remediation results. A campaign was created with test cases mapping to each stage of the GraphQL kill chain. The baseline run recorded zero mitigations, zero detections, and full attack success across all eleven test cases. The post-remediation run recorded a fundamentally different outcome.

The VECTR metrics demonstrate the measurable improvement in security posture:

- Baseline:0% detection, 0% mitigation, 100% attack success rate.
- Post-remediation:80% detection coverage (alerts fired for every attack stage), 100% mitigation coverage for validated controls (SSH forced-command blocked lateral movement, NGINX blocked deprecated API access for Scenario 1, and rate-limiting rules contained transfer abuse for Scenario 3).

Field	Count
Campaigns	1
Test Cases	11
Passed	9
Blocked	0
N/A	0
Alerted	9
Failed	2
Logged	0
None	2
To Be Determined	0

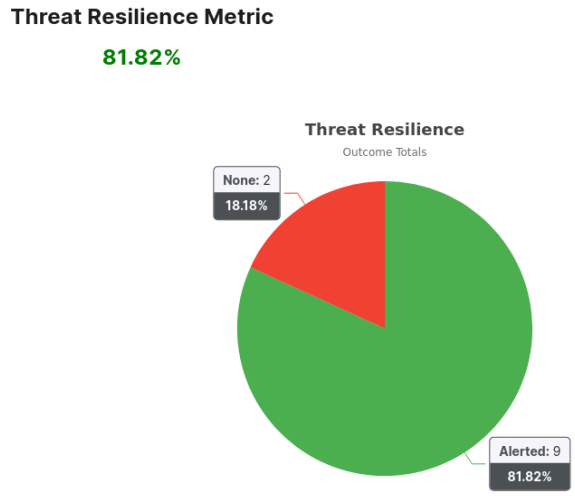


Figure 59: VECTR Metrics Post-Remediation

Post-exploitation activity was also re-tested using Atomic Red Team techniques executed via the established SSH channel. Each technique triggered its corresponding Sysmon-based Wazuh rule:

Aug 13, 2026 @ 03:12:38.4...	workstation	CRITICAL: LSASS Dump via rundll32 comsvcs.dll - Credential Dumping	14	100305
Aug 13, 2026 @ 03:12:37.1...	workstation	Scheduled Task Created - Persistence	10	100304
Aug 13, 2026 @ 03:12:37.1...	workstation	Scheduled Task Created - Persistence	10	100304
Aug 13, 2026 @ 03:10:18.5...	workstation	CRITICAL: LSASS Dump via rundll32 comsvcs.dll - Credential Dumping	14	100305
Aug 13, 2026 @ 03:09:32.2...	workstation	CRITICAL: LSASS Dump via rundll32 comsvcs.dll - Credential Dumping	14	100305
Aug 13, 2026 @ 03:09:32.0...	workstation	Scheduled Task Created - Persistence	10	100304
Aug 13, 2026 @ 03:09:32.0...	workstation	Scheduled Task Created - Persistence	10	100304
Aug 13, 2026 @ 03:08:49.8...	workstation	CRITICAL: LSASS Dump via rundll32 comsvcs.dll - Credential Dumping	14	100305
Aug 13, 2026 @ 03:08:33.4...	workstation	Domain Account Enumeration - Discovery	8	100307
Aug 13, 2026 @ 03:08:33.3...	workstation	Domain Account Enumeration - Discovery	8	100307

Figure 60: Wazuh Dashboard with Custom Rules

- Rule 100304** (`schtasks.exe` creating a scheduled task) — Persistence detected (T1053.005).
- Rule 100305** (`rundll32.exe` invoking `comsvcs.dll` for LSASS dump) — Credential access detected (T1003.001).
- Rule 100307** (`net.exe /domain` enumerating domain accounts) — Discovery detected (T1087.002).

These alerts appeared in the Wazuh dashboard within seconds of execution, confirming that even after an attacker gained a foothold, post-exploitation behavior was now visible to the Blue Team.

12. Conclusion

This Purple Team exercise demonstrated the full value of collaborative threat emulation against a realistic, segmented enterprise environment. Four distinct attack scenarios, spanning API manipulation, authorization bypass, business logic abuse, and a multi-stage GraphQL exploitation chain, were executed, analyzed, remediated, and validated within the constraints of a Security Maturity Level 1 organization.Several key lessons emerged from the exercise:

Detection without prevention is insufficient. Scenario 1 illustrated this clearly: the custom Wazuh rule successfully alerted on the API downgrade attack, but the request still returned HTTP 200 until the NGINX reverse proxy was deployed. Detection and prevention must be implemented as complementary controls, not alternatives.

Logging gaps are as dangerous as missing detections. Scenario 3 revealed that VulnBank's transfer activity was never ingested by Wazuh because no Docker container log forwarding was configured. No detection rule can fire on data the SIEM never receives. Closing the ingestion gap was a prerequisite for any meaningful alerting.

Single unsanitized inputs can bridge network boundaries. Scenario 4 demonstrated that one injectable GraphQL parameter, combined with an over-scoped service account and an unrestricted automation credential, was sufficient to progress from unauthenticated reconnaissance to Domain Admin access across two network segments. The attack required no credential cracking, no exploit kit, and no brute-forcing, only the chaining of existing application features and misconfigurations.

Defense-in-depth requires layered visibility. The Scenario 4 remediation deployed detection at three distinct layers (network, database, endpoint) because no single telemetry source provided complete coverage of the attack chain. Suricata identified the malicious HTTP payloads, PostgreSQL logs confirmed code execution, and Sysmon captured the resulting process activity on the workstation. Only together did these layers provide the Blue Team with a reconstructible picture of the full kill chain.

Automated response reduces dwell time but requires validation. The Shuffle SOAR playbook and nft-block.sh active-response script demonstrated the intended containment workflow: alert ingestion, threat-intel context, and dynamic IP blocking at the network edge. While the script was validated in isolation and the playbook was fully configured, end-to-end automation could not be confirmed due to Wazuh API credential constraints in the lab environment. In a production deployment, this chain would execute automatically, reducing the window between detection and containment from manual review time to seconds.

Purple Teaming creates tighter defensive loops than isolated testing. The most effective remediation decisions came from joint Red/Blue analysis sessions immediately following each baseline attack. The Red Team's knowledge of exactly what was executed allowed the Blue Team to write precise detection logic rather than relying on generic signatures. Conversely, the Blue Team's identification of logging gaps informed the Red Team's understanding of where attacks could proceed silently, guiding the selection of validation techniques that would exercise the new controls.

The exercise confirmed that even a low-maturity environment can achieve meaningful detection and prevention coverage through targeted, attack-informed defensive engineering. The test-remediate-retest lifecycle produced measurable, repeatable improvements that would not have been achievable through penetration testing or monitoring configuration alone. The group recommends that future iterations of this exercise incorporate application-level source code fixes, expanded SOAR automation with validated API integration, and additional attack scenarios targeting the Active Directory infrastructure directly.

13. References

Atomic Red Team. (2024). *Atomic Red Team: Small and highly portable detection tests* [Computer software]. Red Canary. https://github.com/redcanaryco/atomic-red-team

Commando-X. (2024). *vuln-bank: A deliberately vulnerable banking application for security training and Purple Team exercises* [Computer software]. GitHub. https://github.com/Commando-X/vuln-bank

Docker Inc. (2024). *Docker documentation: Engine, Compose, and container runtime reference*. https://docs.docker.com

Gerhards, R., & Lonvick, C. (2004). *The syslog protocol* (RFC 5424). Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc5424

Hilker, M. (2021). *nftables: The new Linux firewall*. Netfilter Project. https://wiki.nftables.org

MITRE Corporation. (2024). *MITRE ATT&CK Framework, version 14*. https://attack.mitre.org

MISP Project. (2024). *MISP: Open source threat intelligence platform* [Computer software]. https://www.misp-project.org

OWASP Foundation. (2023). *OWASP API Security Top 10 2023*. https://owasp.org/www-project-api-security/

PortSwigger. (2024). *Burp Suite documentation: Professional edition user guide*. https://portswigger.net/burp/documentation

PostgreSQL Global Development Group. (2024). *PostgreSQL 15 documentation: COPY command and server-side functions*. https://www.postgresql.org/docs/15/sql-copy.html

Russinovich, M. E., & Garnier, T. (2024). *Sysmon v15.0 documentation*. Microsoft Sysinternals. https://docs.microsoft.com/en-us/sysinternals/downloads/sysmon

SigmaHQ. (2024). *Sigma: Generic signature format for SIEM systems* [Computer software]. GitHub. https://github.com/SigmaHQ/sigma

Shuffle Automation. (2024). *Shuffle SOAR: Open-source security orchestration, automation and response* [Computer software]. https://shuffler.io

Springer Nature. (2024). *Springer Nature article template: LaTeX and Word guidelines for authors*. https://www.springer.com/gp/authors-editors/book-authors

Suricata Open Information Security Foundation. (2024). *Suricata user guide: IPS mode, rule syntax, and nfqueue integration* (Version 8.0). https://docs.suricata.io

The MITRE Corporation. (2024). *Purple Team Exercise Framework (PTEF)*. MITRE Engenuity. https://mitre-engenuity.org/cybersecurity/purple-team/

VECTR. (2024). *VECTR: Purple Team planning, execution, and reporting platform* [Computer software]. https://vectr.io

Wazuh, Inc. (2024). *Wazuh documentation: User manual, API reference, and ruleset guide* (Version 4.7). https://documentation.wazuh.com

Werkzeug Contributors. (2024). *Werkzeug WSGI utility library documentation*. Pallets Projects. https://werkzeug.palletsprojects.com

14. Video Recordings

https://drive.google.com/file/d/1RLag6W0FjBb3nwnLxuGaIFM_oso71at-/view?usp=sharing